High-Performance Computer Architecture

A Companion to ECE 4100/6100

ECE 4100/6100

Spring 2026

Contents

Before We Begin **v**

1 Introduction **1**
1.1 What Computer Architecture Is . . . 1
1.2 Core Methods: Parallelism, Speculation, and Prediction . . . 1
1.3 Parallelism by Grain Size . . . 2
1.4 Flynn's Taxonomy . . . 3
1.5 Vectors: Packed-Word SIMD vs. Pipelined Vector Units . . . 4
1.6 Common Chapter 1 Pitfalls . . . 5
1.7 Chapter 1 Summary . . . 5
1.8 Practice Questions . . . 6

2 Performance Evaluation **7**
2.1 Why Performance Evaluation Matters . . . 7
2.2 Throughput vs. Runtime . . . 7
2.3 CPU Runtime Equation . . . 7
2.4 RISC vs. CISC Through the Performance Equation . . . 9
2.5 Amdahl's Law . . . 9
2.6 Gustafson's Law and Why It Does Not Contradict Amdahl . . . 10
2.7 Combining Performance Numbers Correctly . . . 11
2.8 Workload Dependence and Benchmarking . . . 12
2.9 Cost-Aware Performance Comparison . . . 12
2.10 Chapter 2 Summary . . . 12
2.11 Practice Questions . . . 13

3 Cache Memories **14**
3.1 Memory Technology and the Hierarchy . . . 14
3.2 Typical Hierarchy Levels . . . 14
3.3 Locality: Why Caches Work . . . 15
3.4 Cache Geometry and Address Partitioning . . . 15
3.5 Direct-Mapped, Set-Associative, and Fully Associative . . . 17
3.6 Hit, Miss, and Average Access Time . . . 18
3.7 The Three C's of Misses . . . 18
3.8 Replacement Policies and Their Tradeoffs . . . 19
3.9 Write Policies . . . 20
3.10 Reducing Miss Ratio . . . 21
3.11 Reducing Miss Penalty . . . 22
3.12 Reducing Hit Time . . . 24
3.13 Virtual Memory Technology, Terminology, and Analogy to Caching . . . 25
3.14 Compiler and Data-Layout Techniques . . . 25

3.15 Chapter 3 Summary . . . 26
3.16 Practice Questions . . . 27

4 Pipelining **29**
4.1 Pipeline Motivation . . . 29
4.2 Five-Stage Example Pipeline . . . 29
4.3 Pipeline Speedup Calculation . . . 29
4.4 CPI Model and Stall Accounting . . . 30
4.5 Program Characteristics Plus Hazards Produce Stalls . . . 31
4.6 Structural Hazards . . . 31
4.7 Data Hazards . . . 31
4.8 Control Hazards . . . 32
4.9 Branch Target Buffer (BTB) . . . 33
4.10 Whether-To Branch: Direction Prediction . . . 34
4.11 Simple Dynamic Direction Predictors . . . 34
4.12 Global and Local History Predictors . . . 35
4.13 Perceptron Direction Prediction . . . 37
4.14 Hybrid and Tournament Predictors . . . 37
4.15 Timing-Diagram Interpretation . . . 38
4.16 Chapter 4 Summary . . . 39
4.17 Practice Questions . . . 40

5 Integrated Synthesis and Cache Simulation Concepts **42**
5.1 From an Event to an Explanation . . . 42
5.2 Integrated Themes from Chapters 1 and 2 . . . 42
5.3 Integrated Themes from Chapter 3 . . . 43
5.4 Integrated Themes from Chapter 4 . . . 43
5.5 Cache Simulation Conceptual Model . . . 45
5.6 Hierarchy Semantics in the Model . . . 45
5.7 Prefetching Concepts . . . 45
5.8 Statistics and AAT Interpretation . . . 46
5.9 Conceptual Tradeoffs Across Parameters . . . 46
5.10 Chapter 5 Summary . . . 46
5.11 Practice Questions . . . 46

6 Dynamic ILP and Out-of-Order **48**
6.1 Why Cycles per Instruction Can Go Below One . . . 48
6.2 Historical Motivation . . . 48
6.3 Superscalar Pipeline View . . . 49
6.4 Dependency Types and Why They Matter . . . 49
6.5 Fire/Complete Taxonomy . . . 50
6.6 Scoreboarding Intuition (FICO Style) . . . 51
6.7 Function Units and Latency . . . 51
6.8 Worked FICO Timing Example . . . 52
6.9 Register Renaming as the Next Step . . . 53
6.10 Tomasulo's Algorithm . . . 54

6.11 Result Buses and Physical Register Files . . . 55
6.12 RATs, Ready Bits, and Dispatch in Renamed Machines . . . 56
6.13 Precise State and Recovery . . . 56
6.14 Reorder Buffers and Retirement . . . 57
6.15 Future Files and Checkpoint/Repair . . . 60
6.16 Speculation Without and With Prediction . . . 61
6.17 Security Implications of Speculative Execution . . . 61
6.18 Implementation Pressure . . . 61
6.19 Interpreting Out-of-Order Processor Statistics . . . 61
6.20 Chapter 6 Summary . . . 62
6.21 Practice Questions . . . 63

7 Compiler-Scheduled ILP 65
7.1 VLIW as a Compiler-Scheduled Alternative . . . 65
7.2 Dependence Graphs and List Scheduling . . . 66
7.3 Multi-Op Encoding, Tail Bits, and Pause Fields . . . 67
7.4 Software Pipelining and Cyclic Scheduling . . . 68
7.5 Iterative Modulo Scheduling . . . 70
7.6 Worked Software-Pipelining Example . . . 71
7.7 Memory Disambiguation . . . 72
7.8 Chapter 7 Summary . . . 72
7.9 Practice Questions . . . 72

8 Multiprocessors and Coherence 74
8.1 From Instruction-Level to Thread-Level Parallelism . . . 74
8.2 Memory Organization and System Structure . . . 75
8.3 Interconnects: Shared Buses and Point-to-Point Links . . . 75
8.4 The Cache-Coherence Problem . . . 76
8.5 Coherence Solution Families . . . 77
8.6 Snooping Coherence and the MSI Protocol . . . 77
8.7 Transient States and a Worked Two-Cache Trace . . . 78
8.8 MESI, MOESI, and MOESIF Refinements . . . 79
8.9 Directory-Based Coherence . . . 81
8.10 Directory Scaling and Distributed Organizations . . . 84
8.11 Sharing Patterns and False Sharing . . . 86
8.12 Programming Models: Shared Memory and Message Passing . . . 87
8.13 Synchronization Primitives and Coherence Traffic . . . 88
8.14 Scaling and Architectural Limits . . . 89
8.15 Chapter 8 Summary . . . 90
8.16 Practice Questions . . . 90

9 Synchronization and Memory Consistency 93
9.1 Protocol States Versus Protocol Behavior . . . 93
9.2 Atomic Primitives for Shared-Memory Synchronization . . . 93
9.3 Reducing Synchronization Contention . . . 95
9.4 Transactional Memory . . . 96

9.5 Memory Consistency Beyond Coherence . . . 97
9.6 Chapter 9 Summary . . . 101
9.7 Practice Questions . . . 101

10 Parallel Interconnection Networks 103
10.1 Why Parallel Interconnects Are Their Own Topic . . . 103
10.2 Direct and Indirect Topologies . . . 103
10.3 Direct Networks: Hypercubes, Meshes, and Tori . . . 104
10.4 Indirect Networks and Multi-Stage Fabrics . . . 106
10.5 Routing Algorithms . . . 107
10.6 Flow Control and Congestion Management . . . 108
10.7 From Store-and-Forward to Virtual Channels . . . 109
10.8 Switch Architecture . . . 110
10.9 Chapter 10 Summary . . . 110
10.10Practice Questions . . . 111

A Pipeline Timing-Diagram Interpretation Checklist 113
A.1 How to Use This Checklist . . . 113

B Formula Compendium (Grouped and Labeled) 117

C Glossary of Core Definitions 123

D Advanced Topics 135
D.1 Quantum Computing Architecture . . . 135
D.2 Superconducting Processor Architecture . . . 136
D.3 Final Review Checkpoints . . . 138
D.4 Practice True/False Prompts . . . 139

E Comprehensive Final Review Checklist 140
E.1 Core Topic Checklist . . . 140
E.2 Review Clarifications . . . 142
E.3 Fast Self-Check Prompts . . . 143

F Lecture Sources and Historical Notes 144

Before We Begin

The Thread Through the Book

A fast computer is a machine that spends less time waiting. It keeps useful data nearby, overlaps independent work, and sometimes starts work before it knows whether that work will be needed. Each improvement creates another question: what happens when the guess is wrong, the data is missing, or two processors want to change the same value? Those questions connect the chapters that follow.

We begin with the kinds of parallel work a machine can exploit and the measurements that tell us whether an improvement helped. Caches and pipelines then show how a single core hides or avoids waiting. Dynamic scheduling and compiler scheduling offer two ways to find more overlap. The later chapters move beyond one core, where keeping copies consistent, coordinating threads, and carrying messages become part of the computation itself.

This is a course companion, not a replacement for the lectures or the current assignment specification. Read the prose for the mechanism, work through the examples for its consequences, and use the end-of-chapter questions to test whether you can reason without the explanation in front of you. The separate answer sheet follows the same chapter and question numbering. The formula compendium and glossary are reference tools; they are most useful after the underlying ideas are familiar. Appendix D extends the discussion to quantum and superconducting architectures, and Appendix E supports a final review.

Stories, Sources, and Simplified Machines

Short lecture examples appear where they explain an architectural choice. An attributed example is tied to the supplied course material in Appendix F; an uncited everyday analogy is an illustration written for this book. New analogies explain mechanisms without being attributed to the instructor. Historical machines provide context, while cycle counts and small diagrams describe explicitly simplified models unless identified otherwise. Always carry the stated assumptions into a calculation: forwarding timing, queue width, memory policy, and the meaning of a counter can change the answer.

Cross-Chapter Reading Paths

1. Readers who want the performance-and-memory foundation should begin with Chapters 1 through 3 and then read the first part of Chapter 4.
2. Readers focused on control hazards, prediction, and data hazards should concentrate on the middle and later sections of Chapter 4.
3. Readers focused on instruction-level parallelism should read Chapter 6 for dynamic ILP and Chapter 7 for compiler-scheduled ILP.

4. Readers focused on simulator-style interpretation and consolidated worked checks should use Chapter 5 alongside the detailed discussions in Chapters 3, 4, and 6.
5. Readers focused on thread-level parallelism, coherence, and shared-memory multiprocessors should continue from Chapters 1 and 2 into Chapters 8 through 10.
6. Readers reviewing the advanced-topics material should read Appendix D.
7. Readers doing a last comprehensive review should use Appendix E to identify weak concepts, then return to the relevant chapter sections.

Topic Coverage Map

Chapter 1: Introduction Coverage includes speculation with and without prediction, grain sizes of parallelism (bit, instruction, thread, and inter-program), Flynn taxonomy, single instruction, multiple data (SIMD) versus multiple instruction, multiple data (MIMD), pipelining as temporal parallelism, and the distinction between packed-word SIMD and pipelined vector operations.

Chapter 2: Performance Evaluation Coverage includes throughput versus runtime, the central processing unit (CPU)-time equation in terms of instruction count (IC), cycles per instruction (CPI), and cycle time, reduced instruction set computer (RISC) versus complex instruction set computer (CISC) tradeoffs, Amdahl's and Gustafson's laws, and correct use of arithmetic versus harmonic means.

Chapter 3: Cache Memories Coverage includes static random-access memory (SRAM) versus dynamic random-access memory (DRAM) and bit-line delay, locality, hierarchy levels, virtual-memory terminology and its analogy to caching, average access time (AAT), the three C's of misses, cache geometry and construction, replacement and write policies, insertion-policy choices such as MIP and LIP, prefetching strategies including Markov and hybrid schemes, techniques to improve miss rate, miss penalty, and hit time, simulator-style hierarchy statistics, and compiler/data-layout effects.

Chapter 4: Pipelining Coverage includes instruction fetch / instruction decode / execute / memory / write back (IF/ID/EX/MEM/WB) behavior, speedup equations, cycles per instruction (CPI) stall modeling, the principle that program behavior plus hazards produces stalls, structural and control hazards, delay and squash-slot ideas, branch target buffer (BTB) behavior with and without tags, return-address stack (RAS) behavior, direction-prediction heuristics, and dynamic predictors including one-bit and Smith-counter schemes, GShare, GSelect, Yeh/Patt, perceptrons, hybrid tournament selection, and delayed predictor-update timing.

Chapter 5: Integrated Synthesis and Cache Simulation Concepts Coverage includes integrated synthesis, worked pipeline calculations, hierarchy semantics, prefetching, and interpretation of simulator-style cache, branch, and average access time (AAT) statistics.

Chapter 6: Dynamic ILP, Superscalar Scheduling, and Out-of-Order Execution Coverage includes cycles per instruction (CPI) versus instructions per cycle (IPC) interpretation, instruction-level parallelism (ILP), historical superscalar context, fire/complete ordering models,

scoreboarding and busy-bit scheduling, function-unit latency considerations, Tomasulo-style reservation stations, physical register files, result buses, register renaming intuition, precise versus imprecise exceptions, reorder buffer (ROB) and future-file organization, checkpoint/repair recovery, simulator-style fetched/retired and queue-pressure counter interpretation, and the security consequences of speculative out-of-order execution.

Chapter 7: Compiler-Scheduled ILP and Software Pipelining Coverage includes compiler-scheduled very long instruction word (VLIW) execution, dependence-graph and list-scheduling ideas, zero-no-op multi-op encoding with tail and pause fields, software pipelining and iterative modulo scheduling, and memory disambiguation.

Chapter 8: Multiprocessors, Interconnects, and Cache Coherence Coverage includes shared-memory versus message-passing organization, standalone multiprocessors versus accelerator systems, buses versus point-to-point links, the cache-coherence problem and its formal properties, shared-cache versus directory-based versus snooping solutions, MSI coherence states and transactions, MESI/MOESI/MOESIF refinements including `E`, `O`, and `F`, transient coherence states, directory-entry structure and message flow, limited and distributed directory organizations, sharing patterns and false sharing, and shared-memory programming support through locks, barriers, and atomic synchronization primitives.

Chapter 9: Atomic Synchronization, Transactional Memory, and Memory Consistency Coverage includes the distinction between coherence states and full protocol behavior, abort-versus-intervene transfer policies, load-link / store-conditional, fetch-and-op and fetch-and-add primitives, dynamic work claiming, tree and tournament locks, sense-reversing barriers, hardware barrier intuition, transactional-memory execution and aborts, and the distinction between coherence and memory consistency, including strict consistency, sequential consistency, relaxed consistency, data races, and synchronization ordering through fences and acquire/release style semantics.

Chapter 10: Parallel Interconnection Networks Coverage includes the distinction between direct and indirect topologies, hypercube organization and Gray-code intuition, k-ary n-dimensional meshes and tori, physical-layout constraints and wire-length balance, bisection bandwidth, crossbars and dance-hall style switching, multi-stage interconnects such as omega networks, perfect-shuffle wiring, deterministic versus adaptive versus oblivious routing, packets versus flits, and the progression from store-and-forward to cut-through, wormhole, and virtual-channel flow control.

Appendix D: Advanced Topics Coverage includes quantum computers as accelerators, decoherence, imperfect control, NISQ limits, fault-tolerant logical qubits, reversible phase-oracle intuition, superconducting-circuit constraints, compiler-scheduled dataflow execution, operand queues, barrel processing, and combined single-thread/throughput-oriented superconducting processor organization.

Appendix E: Final Review Checklist Coverage includes a comprehensive concept checklist, review clarifications for tiling, iterative modulo scheduling, memory consistency, barriers versus locks, and bisectional bandwidth, plus fast self-check prompts.

Chapter 1

Introduction

A processor can have three adders and still finish only one useful addition at a time. If each addition needs the preceding result, the extra hardware must wait. Give those same adders three independent calculations and all three can work together. This small contrast, used in the introductory lecture, is the starting point for the book: performance comes from matching a machine to the work it can actually overlap.[1]

1.1 What Computer Architecture Is

Computer architecture asks how a processor and memory system should be organized to finish useful work efficiently. "Efficiently" might mean a shorter wait for one result, more completed jobs per second, or less energy per calculation. Circuits, fabrication, software, and algorithms set the possibilities; the architect chooses how to arrange the machine within them.

Every architectural decision is a tradeoff among competing goals. A design that minimizes latency for one task may reduce throughput for a server-style workload. A design that aggressively improves performance may consume more energy or require expensive cooling. A design that is elegant on paper may be too costly in area or too difficult to verify at scale. For this reason, architecture is best treated as constrained optimization: choose structures and policies that best match the target application while respecting implementation limits.

1.2 Core Methods: Parallelism, Speculation, and Prediction

Most performance techniques in modern processors can be traced to three closely linked ideas: *parallelism*, *speculation*, and *prediction*. *Parallelism* increases performance by doing multiple useful operations at the same time. *Speculation* increases performance by doing work early, before the machine is fully certain that this work will be needed. *Prediction* estimates uncertain future behavior (for example branch direction or branch target) so speculative work is directed toward the most likely path. Prediction makes speculation more selective: a good estimate directs early work toward a result the program will actually need.

Classifying common optimizations This distinction is useful because many architectural features combine these ideas differently. Packed-word SIMD instructions and multicore processors are primarily forms of parallelism. The course uses *speculation without prediction* broadly for locality-based choices such as retaining cache blocks in case they are needed again. That is a useful teaching category, but a cache hit does not necessarily execute an uncommitted instruction, and ordinary buffering need not involve a guess. Branch prediction is speculation with prediction.

[1] Course handout `n1_intro.pdf`, slide INTRO-6. Lecture-source references use the identifiers collected in Appendix F.

Out-of-order superscalar execution combines both parallelism and speculation because the machine overlaps independent work while also allowing some work to proceed before the final architectural outcome is known.

1.2.1 Speculation Without Prediction

Speculation does not require a learned predictor. A processor can use a fixed optimistic rule rather than a learned estimate. For example, it may execute a load before the addresses of older stores are known, assuming that the load will not read a location one of those stores changes. If the assumption fails, the machine must detect the ordering violation and replay the affected work. No history-based predictor is required, though one could make that decision more selective.

Related idea: a write buffer A write buffer lets the core continue while a store waits to reach the next memory level. The important gain is overlap, and the buffer alone does not prove that the store is speculative. In a machine with precise exceptions, translation and permission faults must be resolved before a store becomes irrevocable. An uncommitted store can wait in a speculative store queue; a committed store can drain from a buffer later without being a guess about whether the instruction should execute.

1.2.2 Speculation With Prediction

When prediction is added, speculation becomes more selective and usually more efficient. The processor estimates likely future behavior, executes along the estimated path, and commits the work only if the estimate is correct.

Example: branch prediction Branch prediction is the canonical case. If the predictor guesses that a branch is taken, instruction fetch and decode proceed down the predicted target path immediately. A correct guess keeps the front end busy and reduces control-hazard stalls. An incorrect guess forces the machine to squash the speculative instructions and restart on the correct path, which introduces a misprediction penalty. The basic tradeoff is straightforward: high prediction accuracy yields high performance; low accuracy can erase the benefit of speculation.

1.3 Parallelism by Grain Size

Parallelism is best understood by *grain size*, meaning the amount of work grouped into one concurrent unit. At the smallest scale, bit-level parallelism appears in wide datapaths and arithmetic logic that operate on many bit positions at once. A 64-bit adder, for example, processes far more state per cycle than an 8-bit design because the hardware width itself exposes more fine-grained parallel work.

At *instruction-level parallelism (ILP)*, the machine overlaps independent instructions within a thread. Superscalar issue, out-of-order scheduling, and pipelining all exploit ILP. Consider two arithmetic instructions: `R1=R2+R3` and `R4=R5+R6`. Because neither depends on the other, a dual-issue machine can dispatch them together. In contrast, `R1=R2+R3` followed by `R4=R1+R6` introduces a data dependence that limits overlap. This illustrates the central ILP limitation: dependencies, control hazards, and hardware resource conflicts bound achievable parallel speedup. Figure 1.1 makes the introductory lecture's comparison explicit.

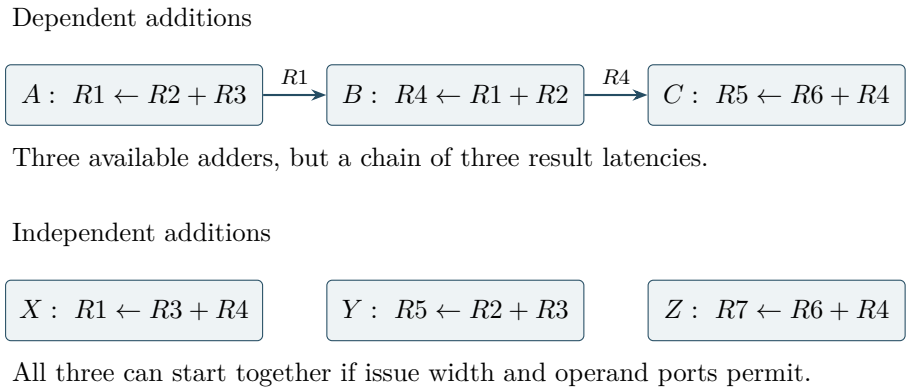


Figure 1.1: Hardware width is useful only when the work is independent. Assume equal add latency, ready inputs, three adders, and enough issue bandwidth; ignore fetch and write-back overhead.

At *thread-level parallelism (TLP)*, multiple threads execute concurrently on multithreaded or multicore hardware. TLP can provide substantial throughput gains, but only if the workload contains enough independent work and synchronization overhead remains controlled. Inter-program parallelism is the coarsest common level, where independent processes run simultaneously under an operating system scheduler. This is the dominant form in desktop multitasking and cloud services, where many programs share the same physical platform.

1.4 Flynn's Taxonomy

Flynn's taxonomy classifies parallel machines by counting instruction streams and data streams. In *single instruction, single data (SISD)*, one instruction stream acts on one data stream, matching the traditional scalar uniprocessor model. In *single instruction, multiple data (SIMD)*, one instruction stream is applied across many data elements in lockstep, which is highly effective for dense data-parallel computations such as vector math, image processing, and many graphics processing unit (GPU) kernels. The data values may differ across lanes, but the issued operation is the same across the whole SIMD group.

In *multiple instruction, multiple data (MIMD)*, multiple processing elements execute different instruction streams on different data streams. This is the dominant model for modern multicore central processing units (CPUs), distributed servers, and cluster systems because it supports general-purpose, irregular workloads. *multiple instruction, single data (MISD)* exists mostly in specialized contexts, such as certain fault-tolerant designs, and is uncommon in mainstream systems.

1.4.1 SIMD vs. MIMD

SIMD and MIMD solve different problems. SIMD is efficient when the same operation must be applied repeatedly to large collections of data. Its lockstep execution naturally provides synchronization across lanes, which reduces control overhead. If different cores or lanes truly follow different code paths at the same time, the machine is no longer acting like classical SIMD for that region; that behavior belongs to MIMD or to a SIMT-style execution model with divergence handling. MIMD is more flexible because each processor can follow its own control path, but this flexibility introduces synchronization and communication costs that must be managed explicitly by hardware and software.

Modern GPUs often use a hybrid model sometimes called *single instruction, multiple threads (SIMT)*. SIMT provides a SIMD-like execution efficiency for the hardware while allowing a MIMD-like programming model where each thread appears to have its own scalar program counter. The hardware manages the divergence and convergence of these threads, grouping those on the same path into SIMD-style execution units.

Representative workload mappings Simple examples help fix the taxonomy. A small embedded controller such as a thermostat is usually best viewed as SISD. A dense image transform that applies the same arithmetic to many pixels is a classic SIMD workload. A cloud-service cluster is naturally MIMD because many processors or machines execute different control paths for different requests. Some tasks, such as brute-force hash search, can be built in several ways: a lockstep array of identical workers looks SIMD-like, while a farm of largely independent workers looks MIMD. The important step is to identify whether the machine is reusing one instruction stream across many data elements or allowing many instruction streams to proceed independently.

1.4.2 Where Pipelining Fits

Pipelining is orthogonal to Flynn's taxonomy. It is a temporal form of parallelism in which different stages operate on different instructions concurrently. Because pipeline structure is an internal execution organization, pipelining can appear inside SISD cores, SIMD engines, and MIMD processors alike.

1.5 Vectors: Packed-Word SIMD vs. Pipelined Vector Units

Packed-word SIMD and classic vector pipelines both exploit data parallelism, but they do so with different execution structures. Packed-word SIMD divides a fixed-width register into lanes (for example, four 32-bit lanes in 128 bits) and applies one instruction across all lanes in lockstep. An operation such as `ADD_PACKED` performs multiple elementwise additions per instruction, and lane boundaries isolate carries so one lane's overflow does not spill into neighboring lanes. This style underlies CPU SIMD extensions such as streaming SIMD extensions (SSE), advanced vector extensions (AVX), and Arm NEON.

Vector pipelines, in contrast, stream long vectors through deep arithmetic pipelines. A floating-point multiply can be viewed conceptually as operand preparation, multiply, and result normalization/writeback stages. If a pipeline has k stages and processes n elements, a serial implementation needs nk cycles, while a pipelined implementation needs approximately $k + n - 1$ cycles after fill effects are accounted for (Eq. (1.1)):

$$\text{serial cycles} = nk, \qquad \text{pipeline cycles} \approx k + n - 1. \tag{1.1}$$

The result is substantial throughput gain for long vectors even though single-element latency is not eliminated.

Worked vector-throughput example Suppose a vector arithmetic pipeline has 6 stages and the program processes 100 elements. A non-pipelined implementation would spend

$$nk = 100 \times 6 = 600 \text{ cycles} \tag{1.2}$$

to process the vector. The pipelined form instead needs approximately

$$k + n - 1 = 6 + 100 - 1 = 105 \text{ cycles.} \tag{1.3}$$

The difference is the standard vector-pipeline gain: the pipeline still pays 6-stage latency for the first element, but after fill it can accept and complete additional elements at a much higher steady-state rate.

Chaining Vector chaining improves this further. If a second vector operation consumes results from a first operation, the hardware can forward early produced elements directly into the consumer pipeline instead of waiting for the full vector to complete. Chaining reduces producer-consumer delay and improves sustained throughput in vectorized loops.

Why the Cray example belongs here The introductory slides use the Cray-1 to make vector processing concrete.[2] Its vector registers could hold 64 elements, so one vector instruction described a stream of work rather than a single scalar operation. With chaining, an early element could enter the next functional unit while later elements were still passing through the first. The useful comparison is to a line of workstations: the second workstation can start on the first item without waiting for the entire batch. The six-stage calculation above is a teaching model, not a claim about the Cray-1's exact functional-unit latencies.

1.6 Common Chapter 1 Pitfalls

A common mistake is to conflate speculation with prediction. The correct view is that prediction is one way to make speculation more effective, not the definition of speculation itself. Another recurring error is to assume that additional cores guarantee proportional speedup; real speedup is limited by dependencies, communication, and serial regions of code. It is also common to treat SIMD and MIMD as interchangeable, even though they target different workload structures and have different programming implications.

A final source of confusion is the difference between packed-word SIMD and long-vector pipelining. Both process multiple data elements, but packed SIMD exploits lane-level parallel arithmetic within instruction-width registers, while vector pipelines exploit staged streaming over long element sequences. Related to this, pipelining primarily improves throughput rather than the latency of a single operation.

1.7 Chapter 1 Summary

Chapter 1 introduces computer architecture as disciplined performance engineering under practical constraints. The chapter frames nearly all speedup mechanisms as forms of parallelism or speculation, then organizes parallelism by grain size from bits through instructions, threads, and entire programs. Flynn's taxonomy provides a clean language for distinguishing SISD, SIMD, MISD, and MIMD machines, especially the practical differences between SIMD efficiency and MIMD flexibility. Finally, the chapter distinguishes packed-word SIMD from pipelined vector execution and shows why chaining is valuable in vector data paths. Together, these ideas form the conceptual base used throughout the rest of the book.

[2] `n1_intro.pdf`, INTRO-13–14; see also the Cray manual listed in Appendix F.

1.8 Practice Questions

The following questions reinforce the main ideas of Chapter 1 and are intended for self-checking after the summary.

1. For each of the following techniques, state whether it primarily exploits *parallelism*, *speculation without prediction*, *speculation with prediction*, or more than one of these ideas: packed-word SIMD, a cache, a branch predictor, and out-of-order execution with branch prediction.
2. Classify each of the following workloads using Flynn's taxonomy, and give a one-sentence justification for each answer: a dense image convolution engine, a cloud-service cluster that handles independent requests, a small thermostat controller, and a brute-force hash search implemented as many independent workers.
3. Explain why pipelining is orthogonal to Flynn's taxonomy. Give one example of a pipelined SISD design and one example of pipelining inside a wider SIMD or MIMD system.
4. A vector arithmetic pipeline has 6 stages and processes a vector of 100 elements. Compute the approximate cycle count for a serial implementation and for a pipelined implementation. Then explain how chaining could reduce producer-consumer delay for two dependent vector operations.
5. Distinguish packed-word SIMD from classic pipelined vector execution. In particular, explain what is fixed by register width in packed SIMD and what is streamed through the pipeline in classic vector execution.

Chapter 2

Performance Evaluation

Imagine choosing between two computers by the number on the clock-speed label. The higher-frequency machine seems the obvious choice until its program executes more instructions or spends more cycles waiting on each one. This chapter gives us a way to follow the work from instructions to elapsed time, so that a convincing sales pitch and a faster computer do not become the same thing.

2.1 Why Performance Evaluation Matters

Architecture decisions are only meaningful if they are evaluated with the right metric. A design that improves one metric can degrade another, and a design that looks fast on one workload can be slow on the workload that actually matters. The goal of this chapter is therefore methodological: define performance correctly, choose representative workloads, and use mathematically correct summaries when comparing alternatives.

2.2 Throughput vs. Runtime

Two primary performance metrics appear repeatedly in computer architecture. *Runtime* (also called response time or turnaround time) measures how long one job takes from start to completion. *Throughput* measures how many jobs complete per unit time. These metrics are related but not interchangeable.

For a single-user interactive application, runtime is usually the dominant concern: the user wants one operation to finish quickly. For servers and shared systems, throughput is often dominant: the owner cares about how many requests the system can process per second. The same machine can be better under one metric and worse under the other, so performance claims must always specify which metric is being optimized.

This distinction also explains why pipelining can be valuable even when individual operation latency does not fall dramatically. A deeper pipeline can increase the number of completed operations per unit time, improving throughput, while leaving single-operation latency unchanged or slightly worse.

2.3 CPU Runtime Equation

The fundamental central processing unit (CPU) time equation used throughout performance analysis is (Eq. (2.1)):

$$\text{CPU time} = \text{Instruction Count (IC)} \times \text{Cycles per Instruction (CPI)} \times \text{Cycle Time (CT)}. \quad (2.1)$$

Using clock frequency $f = 1/\text{CT}$, the same equation is (Eq. (2.2)):

$$\text{CPU time} = \frac{\text{IC} \times \text{CPI}}{f}. \quad (2.2)$$

This decomposition is useful because each factor is controlled by different mechanisms. *Instruction count* is influenced by algorithm choice, compiler quality, and instruction set architecture (ISA) expressiveness. *Cycles per instruction (CPI)* is influenced by microarchitecture, hazard behavior, memory hierarchy behavior, and available parallelism. *Cycle time* is influenced by circuit and pipeline timing constraints.

For instruction mixes, total cycles are computed directly (Eq. (2.3)):

$$\text{Total cycles} = \sum_i (\text{IC}_i \times \text{CPI}_i), \qquad \text{CPI}_{avg} = \frac{\text{Total cycles}}{\text{Total instructions}}. \tag{2.3}$$

Worked example Consider a workload that executes 76 loads at 6 cycles each, 30 stores at 4 cycles each, 300 adds at 1 cycle each, and 58 multiplies at 5 cycles each. Then total cycles are

$$76 \cdot 6 + 30 \cdot 4 + 300 \cdot 1 + 58 \cdot 5 = 1166. \tag{2.4}$$

At 2.2 MHz, runtime is

$$\frac{1166}{2.2 \times 10^6} \approx 5.30 \times 10^{-4}\text{s} = 0.53\text{ms}. \tag{2.5}$$

If multiply latency is reduced from 5 cycles to 3 cycles, new total cycles become

$$76 \cdot 6 + 30 \cdot 4 + 300 \cdot 1 + 58 \cdot 3 = 1050, \tag{2.6}$$

which yields a speedup of

$$\frac{1166}{1050} \approx 1.11. \tag{2.7}$$

The example illustrates a recurring theme: accelerating only one instruction class can help, but total improvement depends on how often that class occurs.

Worked machine-comparison example Now consider three machines that all execute the same source-level program. Machine J runs at 5.1 GHz with $CPI = 3.0$. Machine K runs at 5.5 GHz with $CPI = 3.7$ and the same dynamic instruction count as J. Machine G runs at 6.1 GHz with $CPI = 3.2$ but requires 1.5× as many dynamic instructions. Using Eq. (2.2), the J-versus-K comparison is

$$\frac{T_K}{T_J} = \frac{3.7/5.5}{3.0/5.1} \approx 1.146, \tag{2.8}$$

so K is slower even though its clock is faster. For J versus G, runtimes are proportional to

$$T_J \propto \frac{1.0 \times 3.0}{5.1}, \qquad T_G \propto \frac{1.5 \times 3.2}{6.1}. \tag{2.9}$$

Numerically,

$$T_J \propto 0.588, \qquad T_G \propto 0.787. \tag{2.10}$$

So the faster clock does not rescue Machine G: the larger dynamic instruction count dominates and Machine J is faster overall. This is the central Chapter 2 discipline: compare complete CPU time, not any single factor in isolation.

2.4 RISC vs. CISC Through the Performance Equation

Reduced instruction set computer (RISC) and *complex instruction set computer (CISC)* are best understood as design tradeoffs, not as a simple "better/worse" ranking. Historically, CISC designs emphasized richer, denser instructions to reduce dynamic instruction count and code size. RISC designs emphasized simpler instructions and more regular pipelines, often improving CPI and enabling higher clock rates. In CPU-time terms, this is a direct trade (Eq. (2.11)):

$$\text{IC} \downarrow \text{ may come with} \text{CPI} \uparrow \text{ or} \text{CT} \uparrow, \quad \text{or vice versa.} \tag{2.11}$$

In modern systems the boundary is less rigid than textbook definitions suggest, and x86 implementations, for example, decode complex instructions into simpler internal micro-operations and then execute them with highly RISC-like back ends. Arm designs, traditionally associated with RISC, also include increasingly rich instruction features to improve efficiency for modern workloads. The practical conclusion is that ISA style should be evaluated by end-to-end performance, power, and software ecosystem impact rather than by labels alone.

2.5 Amdahl's Law

Amdahl's Law quantifies the speedup limit when only part of a workload is accelerated. If fraction f of execution time benefits from a speedup factor s, total speedup is

$$S_{Amdahl} = \frac{1}{(1-f) + \frac{f}{s}}. \tag{2.12}$$

The limit as $s \to \infty$ is

$$S_{\text{upper}} = \frac{1}{1-f}. \tag{2.13}$$

This bound is the key insight: the unaccelerated fraction eventually dominates.

Worked example The performance lecture turns Amdahl's law into a purchasing decision: you simulate jet designs, and a spectacularly large machine promises 100,000 processors.[1] It is tempting to divide the old runtime by 100,000. But suppose only 80% of the original runtime can be parallelized, with ideal acceleration of that part and no extra communication cost. Then

$$S = \frac{1}{0.2 + \frac{0.8}{100000}} \approx 5.0. \tag{2.14}$$

If the old simulation took 100 hours, 20 hours remain serial regardless of the processor count. The other 80 hours shrink to 0.0008 hours, leaving just over 20 hours in total. Even 100 ideal processors would finish in 20.8 hours, already close to that limit. Figure 2.1 shows where the time goes. The purchasing question is now concrete: how much is that last fraction of an hour worth?

[1] `n2_perf.pdf`, PERF-18–20. The numerical baseline here is simplified to 100 hours.

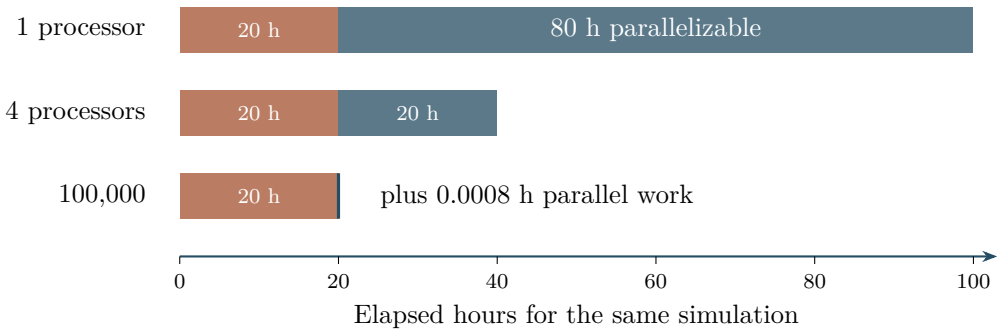


Figure 2.1: Amdahl's fixed-workload limit. The serial 20 hours do not shrink. Assume ideal division of the remaining work and no parallel overhead; the final blue segment is too short to show to scale.

Maximum-speedup interpretation When a problem states that a new mechanism speeds up "25% of a benchmark program," the 25% must be interpreted as a fraction of the *original execution time* of the whole benchmark unless stated otherwise. If that accelerated fraction were made infinitely fast, the maximum possible speedup would be

$$S_{\max} = \frac{1}{1 - 0.25} = \frac{4}{3} \approx 1.33. \tag{2.15}$$

This is the limiting-case form of Amdahl's bound: once the improved portion becomes arbitrarily fast, the remaining 75% of the original runtime dominates completely.

2.6 Gustafson's Law and Why It Does Not Contradict Amdahl

Gustafson's Law addresses a different question: instead of fixing problem size, it assumes the workload scales with available processors. Let α be the sequential fraction measured on an N-processor run. Scaled speedup is

$$S_{Gustafson} = \alpha + (1 - \alpha)N. \tag{2.16}$$

This often grows nearly linearly with N when α is small.

Amdahl and Gustafson do not contradict each other because their assumptions differ. Amdahl is *strong-scaling* analysis (fixed total work). Gustafson is *weak-scaling* analysis (larger total work as resources grow). Both are valid, but they answer different engineering questions.

Example With $\alpha = 0.15$ and $N = 16$,

$$S_{Gustafson} = 0.15 + 0.85 \cdot 16 = 13.75. \tag{2.17}$$

This can exceed Amdahl's fixed-size bound because the scaled workload includes more total useful work.

Return to the jet simulation. Perhaps the real goal is a more detailed airflow model within the same overnight run, rather than finishing the old model as quickly as possible. Additional processors can support that extra parallel work. This is the change of question in the lecture's transition to Gustafson: measure the serial fraction in the scaled parallel run, rather than quietly reusing the old fixed-workload fraction.

2.7 Combining Performance Numbers Correctly

Single-number summaries are useful only when the averaging method matches the quantity being averaged. For times (seconds per task), *arithmetic mean* is appropriate (Eq. (2.18)):

$$\bar{T}_{arith} = \frac{1}{n}\sum_{i=1}^{n} T_i. \tag{2.18}$$

For rates describing *equal amounts of work* in each workload, use the *harmonic mean* (Eq. (2.19)):

$$\bar{R}_{harm} = \frac{n}{\sum_{i=1}^{n} \frac{1}{R_i}}. \tag{2.19}$$

Using arithmetic mean on rates can select the wrong system. Suppose machine A runs two workloads at 4 and 4 transactions/s, while machine B runs them at 2 and 7 transactions/s. Arithmetic means are 4.0 and 4.5, which appears to favor B. But for one transaction of each workload, total throughput is determined by total work divided by total time (Eq. (2.20)):

$$R_A = \frac{2}{\frac{1}{4}+\frac{1}{4}} = 4, \qquad R_B = \frac{2}{\frac{1}{2}+\frac{1}{7}} = \frac{28}{9} \approx 3.11. \tag{2.20}$$

Under equal-work comparison, A is better. This is the lecture's warning about buying the slower machine by averaging the wrong quantity.[2] The result depends on equal work per workload. If instead the machine spends equal *time* at each rate, its total completed work divided by total time is the arithmetic mean of the rates. Start with total work and total time; the appropriate mean follows from what is held equal.

Weighted workload mixes Real benchmark suites are often not uniform. If workload i contributes fraction w_i of the jobs or requests, with $\sum_i w_i = 1$, then the weighted average runtime is

$$\bar{T}_{weighted} = \sum_i w_i T_i. \tag{2.21}$$

For rates under the corresponding fixed-work interpretation, the comparable aggregate rate is

$$R_{eq} = \frac{1}{\sum_i \frac{w_i}{R_i}}. \tag{2.22}$$

These weighted forms are the correct way to combine benchmark results when workloads do not occur equally often.

Worked weighted-runtime example Suppose three workloads occur with weights 0.50, 0.30, and 0.20, and one machine runs them in 4 s, 9 s, and 6 s, respectively. Then the weighted average runtime is

$$\bar{T}_{weighted} = 0.50(4) + 0.30(9) + 0.20(6) = 5.9 \text{ s}. \tag{2.23}$$

The same arithmetic form should *not* be applied blindly to rates. If the reported numbers were throughputs, the fixed-work interpretation would instead require the weighted harmonic form in Eq. (2.22).

[2] `n2_perf.pdf`, PERF-9–11.

2.8 Workload Dependence and Benchmarking

Performance numbers are workload-dependent. Miss ratio, CPI, and even effective clock behavior under thermal constraints depend on the executed program mix. A claim such as "processor X is faster" is incomplete unless it states the benchmark set, input sizes, and aggregation method.

Representative benchmarking therefore requires matching the measured workloads to expected deployment behavior. A web service, a game engine, and a scientific simulation stress very different execution paths and memory behaviors. Chapter 2 methodology demands that architects evaluate on relevant traces and report assumptions explicitly; otherwise conclusions will not transfer to real systems.

2.9 Cost-Aware Performance Comparison

In the January 15 lecture, the question "What is a supercomputer?" leads to a joke about the most expensive machine one could buy in a generation.[3] The joke connects the pursuit of maximum performance to the price of reaching it. An architect still has to ask what useful work that money buys for the intended workload. Raw speedup is therefore only one decision metric. If machine m costs $Cost_m$ and delivers speedup S_m relative to a chosen baseline, two useful value measures are

$$\text{speedup per dollar} = \frac{S_m}{Cost_m}, \qquad \text{cost per unit speedup} = \frac{Cost_m}{S_m}. \tag{2.24}$$

The first form is convenient when maximizing performance for a fixed budget. The second form is convenient when minimizing cost for a required performance gain. Both require the same discipline as every other Chapter 2 metric: use one baseline, one workload definition, and one aggregation method throughout the comparison.

Worked value-comparison example Suppose Server A costs \$4,000 and delivers speedup 1.25, while Server B costs \$5,400 and delivers speedup 1.55. Their speedup per dollar values are

$$\frac{1.25}{4000} = 3.125 \times 10^{-4}, \qquad \frac{1.55}{5400} \approx 2.87 \times 10^{-4}, \tag{2.25}$$

so Server A gives slightly more performance per dollar. Their cost per unit speedup values are

$$\frac{4000}{1.25} = 3200, \qquad \frac{5400}{1.55} \approx 3483.87, \tag{2.26}$$

which leads to the same conclusion under the reciprocal metric. This example is intentionally simple: the point is to show that cost-aware comparisons are still just ratio comparisons once a common baseline is fixed.

2.10 Chapter 2 Summary

Chapter 2 provides the framework for disciplined performance reasoning. Runtime and throughput are distinct objectives and must not be mixed casually. CPU time decomposes into IC, CPI, and

[3] January 15, 2026 lecture, Notability note *2. ECE 6100, Recording 2*, approximately 06:03–06:32. This retells the teaching point rather than quoting the speaker.

cycle time, which allows precise reasoning about optimization impact. RISC and CISC are best evaluated as tradeoff points in Eq. (2.1) rather than as absolute categories. Amdahl's Law gives hard limits for fixed-size speedup, while Gustafson's Law explains scaled-workload growth without contradiction. Finally, performance summaries require a stated workload mix: arithmetic means combine job times, harmonic means combine rates for equal work, and weights express unequal work or job frequencies. These tools form the quantitative core used in later chapters.

2.11 Practice Questions

The following questions reinforce the main ideas of Chapter 2 and are intended for self-checking after the summary.

1. Machine A executes a workload with $IC = 1.2 \times 10^9$, $CPI = 1.5$, and clock rate 2.4 GHz. Machine B executes the same workload with $IC = 0.9 \times 10^9$, $CPI = 1.9$, and clock rate 3.0 GHz. Compute the runtime of both machines and the speedup of the faster machine over the slower one.
2. Two machines execute the same program. Machine J runs at 5.1 GHz with $CPI = 3.0$. Machine K runs at 5.5 GHz with $CPI = 3.7$. Compute the speedup of K over J. Then consider Machine G, which runs at 6.1 GHz with $CPI = 3.2$ but requires $1.5\times$ as many dynamic instructions as J for the same work. Compute the speedup of G over J.
3. A benchmark mix contains three workloads with weights 0.50, 0.30, and 0.20. On one machine, their runtimes are 4 s, 9 s, and 6 s. Compute the weighted average runtime. Then explain why the same weighted arithmetic mean would be inappropriate if the reported quantities were throughputs instead of runtimes.
4. A proposed optimization accelerates the branch-recovery portion of a program, which currently accounts for 30% of total runtime, by a factor of 4. Use Amdahl's Law to compute the overall speedup. Then compute the maximum possible speedup if that portion were made infinitely fast.
5. A program spends 20% of its time in vectorizable code. Suppose an added accelerator makes only that portion run four times faster. Compute the overall speedup. Then explain why adding more parallel hardware gives diminishing returns when the accelerated fraction is fixed.
6. A parallel machine runs a scaled problem with serial fraction 0.10 on 16 processors. Compute the speedup predicted by Gustafson's Law. Then explain why this result does not contradict Amdahl's Law.
7. For each of the following, state whether arithmetic mean, harmonic mean, or a weighted variant is the correct aggregation method, and explain why: runtimes for an equal-work benchmark suite, throughputs for an equal-work benchmark suite, and runtimes for a production workload mix with known fractions.
8. Two candidate servers cost \$4,000 and \$5,400 and deliver speedups of 1.25 and 1.55 relative to the same baseline. Compute speedup per dollar and cost per unit speedup for both systems. State which system is preferable under each metric.

Chapter 3

Cache Memories

The next value a program needs may already be only a few millimeters from the execution unit, or it may require a much longer trip through the memory system. The program still issues a load in either case. The cache decides how often that simple instruction becomes a long wait. Its success rests on a practical observation: programs tend to return to the same data and to nearby addresses.

3.1 Memory Technology and the Hierarchy

Modern cache design begins with a technology reality: fast memory is expensive and dense memory is slower. *Static random-access memory (SRAM)* cells use more transistors per bit, provide lower access latency, and do not require refresh, so SRAM is the natural choice for on-chip caches. *Dynamic random-access memory (DRAM)* cells are denser and cheaper per bit but slower and refresh-driven, so DRAM is the natural choice for main memory capacity. This is why systems are built as hierarchies rather than as one uniform memory.

A useful mental model is that each level trades speed for size as distance from the core increases. L1 and often L2 are SRAM-based and optimized for very low hit latency. Lower levels and main memory prioritize capacity and cost. Architecture work in Chapter 3 is mostly about exploiting this hierarchy so that accesses are absorbed by fast levels as often as possible.

At the circuit level, one practical limiter is wire delay. Longer bit lines have higher effective capacitance and resistance, so sense amplifiers need more time to resolve values. This is one reason very large single-level caches become slow: making arrays taller increases bit-line delay even if logic depth is unchanged.

3.2 Typical Hierarchy Levels

In mainstream systems, the first level is a small L1 cache optimized for minimum hit time. A larger L2 cache is usually next, trading some latency for much lower miss ratio. Below L2 sits physical memory (DRAM), which is much larger but much slower. Beneath DRAM is virtual-memory backing store (for example solid-state drive (SSD)), managed by the operating system through paging.

From the central processing unit (CPU) perspective, each lower level is accessed only when the upper level misses. This staged access is exactly why average-access-time analysis composes recursively across levels.

Think of the few books open on a desk and the many books on a shelf. Keeping a useful book within reach saves repeated trips, but covering the desk with everything makes the working area harder to use. The analogy captures the capacity-versus-access-time tradeoff. The hardware adds a precise rule the desk does not have: each address maps to a particular set of candidate locations.

3.3 Locality: Why Caches Work

Cache behavior is dominated by *locality*. *Temporal locality* means that if data is used now, it is likely to be used again soon. *Spatial locality* means that if one address is accessed, nearby addresses are likely to follow. *Sequential locality* is a common special case of spatial locality in which accesses proceed in order, such as instruction fetch or array traversal.

These properties justify block-based transfer. Instead of moving one byte at a time from DRAM, the system moves an entire cache block, betting that adjacent bytes will be useful soon. The same properties also justify replacement policies that retain recently used data and evict older candidates.

3.4 Cache Geometry and Address Partitioning

To reason cleanly about cache structure, use the triplet (C, B, S) (Eq. (3.1)):

$$\text{cache capacity} = 2^C \text{ bytes}, \qquad \text{block size} = 2^B \text{ bytes}, \qquad \text{associativity} = 2^S \text{ ways}. \tag{3.1}$$

Address fields in (C, B, S) notation In this notation, S is **not** an extra address field beyond tag/index/offset. Instead, S changes the number of sets and therefore the index width.

$$\text{sets} = \frac{2^C}{2^B \cdot 2^S} = 2^{C-B-S}. \tag{3.2}$$

Total line count follows directly from sets and ways (Eq. (3.3)):

$$\text{lines} = \text{sets} \cdot \text{ways} = 2^{C-B-S} \cdot 2^S = 2^{C-B}. \tag{3.3}$$

For direct-mapped caches ($S = 0$), ways $= 1$ so lines $=$ sets. For set-associative caches ($S > 0$), lines $>$ sets because there are multiple ways per set.

$$|\text{Offset}| = B, \qquad |\text{Index}| = \log_2(\text{sets}) = C - B - S. \tag{3.4}$$

Because index plus offset consume $(C - S)$ low-order bits (Eq. (3.5)):

$$|\text{Index}| + |\text{Offset}| = (C - B - S) + B = C - S. \tag{3.5}$$

For an address width of AW bits (Eq. (3.6)):

$$|\text{Tag}| = AW - (|\text{Index}| + |\text{Offset}|) = AW - (C - S). \tag{3.6}$$

So the split remains (Eq. (3.7)):

$$\text{Address} = [\text{Tag} \mid \text{Index} \mid \text{Offset}], \tag{3.7}$$

with

$$|\text{Offset}| = B, \qquad |\text{Index}| = C - B - S, \qquad |\text{Tag}| = AW - (C - S). \tag{3.8}$$

Where the S bits "go" Higher associativity means fewer sets, so index width decreases by S relative to a direct-mapped organization with the same C and B. That is why S appears in index/tag equations but is not carried as an address field.

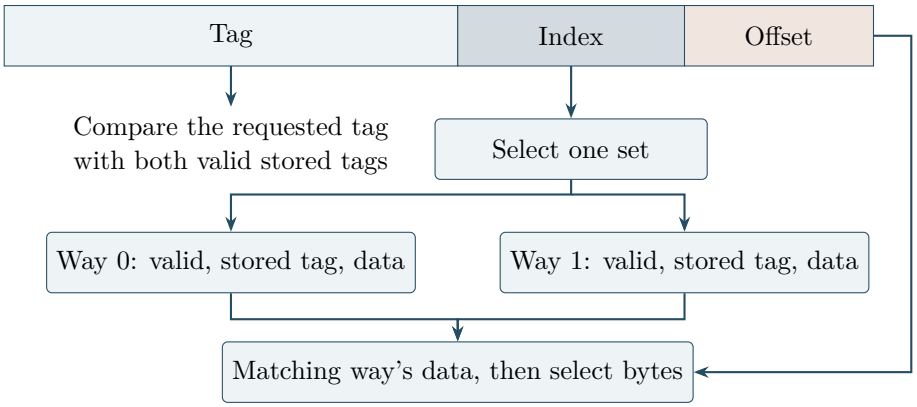


Figure 3.1: A conceptual two-way cache lookup. The index chooses a set, valid tag matches choose the way, and the offset chooses bytes within its block. Field widths are illustrative. If neither valid tag matches, the access misses.

What selects the way? The address tells the cache where to look, but it does not name a way. Figure 3.1 follows one access from the address fields to the matching line. The way is selected by hardware after tag comparison, not by an explicit address field:

1. Index selects one set.
2. Tags for all 2^S ways in that set are compared in parallel.
3. Match logic identifies the hitting way (if any).
4. Internal way-select control drives the data mux (conceptually an S-bit code since $\log_2(2^S) = S$).

Quick example: 32 KB, 32 B blocks, 4-way With $(C, B, S) = (15, 5, 2)$ (Eq. (3.9)):

$$|\text{Index}| = C - B - S = 15 - 5 - 2 = 8. \tag{3.9}$$

For the same C, B but direct-mapped ($S = 0$) (Eq. (3.10)):

$$|\text{Index}| = 15 - 5 - 0 = 10. \tag{3.10}$$

So moving to 4-way associativity reduces index width by 2 bits (equivalently reduces sets by 4×).

32-bit address specialization If $AW = 32$ (Eq. (3.11)):

$$|\text{Offset}| = B, \qquad |\text{Index}| = C - B - S, \qquad |\text{Tag}| = 32 - (C - S) = 32 - C + S. \tag{3.11}$$

Worked example (metadata sizing) Consider a 64-bit machine with $(C, B, S) = (13, 5, 0)$, an 8 KB direct-mapped cache with 32-byte blocks. The cache has $2^{13-5} = 256$ blocks and $2^{13-5-0} = 256$ sets. Tag width is $64 - (13 - 0) = 51$ bits. If each block stores one valid bit and one dirty bit, metadata per block is 53 bits. Total tag-store plus metadata is (Eq. (3.12)):

$$256 \times 53 = 13{,}568 \text{ bits}. \tag{3.12}$$

If B is reduced to 3 while C and S are fixed, the number of blocks rises to $2^{10} = 1024$, so tag overhead increases to $1024 \times 53 = 54{,}272$ bits. This illustrates an important Chapter 3 point: smaller blocks can increase tag overhead significantly.

Concrete Tag-Store Accounting The quickest reliable procedure is:

1. determine the number of lines from 2^{C-B},
2. determine tag width from the relevant physical or virtual address width,
3. add only the metadata bits the problem explicitly asks for, and
4. multiply per-line metadata by line count.

For example, suppose a cache is *write-back, write-allocate* and *physically indexed, physically tagged* with $(C, B, S) = (18, 7, 3)$, 64-bit virtual addresses, 4 KB pages, and 32 GB of physical memory. The physical address width is 35 bits because $32\,\text{GB} = 2^{35}$ bytes. The cache therefore has $2^{18-7} = 2^{11} = 2048$ lines, and the physical tag width is

$$|\text{Tag}| = 35 - (18 - 3) = 20 \text{ bits}. \tag{3.13}$$

Because the level is write-back, write-allocate, each line also needs one valid bit and one dirty bit, so per-line tag-store plus required metadata is 22 bits. Total tag-store size is therefore

$$2048 \times (20 + 1 + 1) = 45{,}056 \text{ bits}. \tag{3.14}$$

If the problem says to ignore replacement metadata such as LRU state, then it should not be added.

Address partitioning example Consider the same $(13, 5, 0)$ 64-bit cache. For the virtual address `0000_1234_ABCD_5678`: **Offset (5 bits):** the lowest 5 bits of `78` (`0111_1000`) are `11000`, which is `0x18` (decimal 24). **Index (8 bits):** bits 5 through 12 come from hex digits `5`, `6`, and `7`. In binary these digits are `0101`, `0110`, and `0111`; extracting bits 5–12 yields `1011_0011` (binary), which is `B3`. **Tag (51 bits):** the remaining upper bits are the tag. This direct bit-mapping is the fundamental operation performed by hardware on every memory access. It illustrates how physical cache structure determines exactly which address bits are used for indexing and comparison.

3.5 Direct-Mapped, Set-Associative, and Fully Associative

A *direct-mapped cache* has one way per set, so each block address maps to exactly one location. Lookup is simple and fast, but conflict misses can be severe. A *set-associative cache* allows each index to hold multiple candidate blocks; it reduces conflicts at the cost of parallel tag comparisons and a way-select multiplexer. A *fully associative cache* removes index constraints entirely and minimizes conflict misses, but hardware search complexity and power become substantial for large capacities.

Under (C, B, S) notation, direct-mapped means $S = 0$. Fully associative means one set, so $C - B - S = 0$ and therefore $S = C - B$. This distinction is critical, because confusing these two conditions leads to incorrect address math and wrong simulator behavior.

How each cache is constructed Direct-mapped construction uses one data line and one tag entry per index. The index decoder selects one row, then one tag compare and one word select complete the access. A 2^S-way set-associative cache replicates tag and data entries 2^S times per index; all tags in the selected set are compared in parallel, and a way-select mux picks the winning line. A fully associative cache has a single set containing all lines, so every access performs an associative search across the entire tag store and relies heavily on replacement metadata.

3.6 Hit, Miss, and Average Access Time

For a cache hit, access latency is the *hit time* HT. For a miss, the processor first spends hit-time logic discovering the miss, then pays *miss penalty* MP to fetch from the next level. The *average access time* is (Eq. (3.15)):

$$AAT = HT + MR \cdot MP, \tag{3.15}$$

where MR is *miss ratio*. In Chapter 3 analysis, Eq. (3.15) is the cache analogue of the CPU-time equation in Eq. (2.1).

If multiple workloads share one machine and workload i accounts for fraction w_i of the accesses, then weighted average access time is

$$AAT_{weighted} = \sum_i w_i \, AAT_i, \tag{3.16}$$

with $\sum_i w_i = 1$. This weighted form is the correct way to compare cache organizations against a non-uniform workload mix.

Hit time is not a single gate delay. It includes row decoding, word-line drive, bit-line sensing, tag comparison, way selection, and word/byte selection. Cache-geometry choices alter these terms. Taller arrays hurt bit-line delay, wider arrays increase mux and comparator complexity, and higher associativity increases tag-compare and selection cost.

Which change affects which AAT term? When a problem asks which of HT, MR, or MP changes, the most useful first-pass mapping is usually:

1. **data-layout or tiling changes** primarily move *miss ratio* because they change locality and conflict behavior,
2. **VIPT versus PIPT** primarily moves *hit time* because it changes whether TLB lookup can overlap cache indexing, and
3. **write buffers** primarily reduce effective *miss penalty* or visible lower-level service time on writes by decoupling cache progress from memory completion.

This classification is approximate rather than absolute, but it is the right first-pass answer for most design analyses.

3.7 The Three C's of Misses

Misses are classed as *compulsory*, *capacity*, or *conflict*. Compulsory misses are first-touch misses and are unavoidable unless data is brought in before first demand access. Capacity misses happen because the working set exceeds what the cache can hold over the reuse interval. Conflict misses happen when mapping restrictions force evictions even though total capacity would have been enough under freer placement.

In practice, this taxonomy provides a diagnosis strategy. If misses are mainly conflict-driven, associativity or victim structures can help. If misses are mainly capacity-driven, larger effective storage or better blocking can help. If misses are mainly compulsory, prefetching and layout changes are primary tools.

Sizing for a repeating working set For a repeating access pattern, the smallest cache that avoids capacity misses must hold every block that remains live across the reuse interval. If N_{live} distinct blocks must stay resident until reuse, then the cache needs at least N_{live} lines. That condition is necessary but not always sufficient in a direct-mapped or low-associativity cache, because several of those live blocks may still collide onto the same set. This is why trace analysis must separate "enough total lines" from "enough placement freedom."

3.8 Replacement Policies and Their Tradeoffs

When a set is full and a miss occurs, one resident block must be evicted. *Belady's optimal policy* is a theoretical lower bound that requires future knowledge and is not implementable directly. *Least recently used (LRU)* approximates this by evicting the least recently touched entry, and usually performs well under temporal locality. *First in, first out (FIFO)* ignores reuse and can evict hot lines. *Random* is simple and sometimes surprisingly competitive but unpredictable. *Least frequently used (LFU)* tracks frequency but can lag phase changes.

For high associativity, exact LRU state grows costly. Practical designs use approximations, tree-based encodings, or hybrid insertion policies. A notable policy distinction is insertion behavior: a line can be installed as *most recently used (MRU)* (default in many demand-miss paths) or near LRU (useful for speculative prefetch lines to reduce pollution). Related insertion-policy abbreviations are *most recently used insertion policy (MIP)* and *least recently used insertion policy (LIP)*.

Simulating LRU with timestamps One exact way to simulate LRU is to attach a "last used" timestamp to each line in a set. On every hit, update the touched line's timestamp to the current access time. On a miss, fill an invalid line if one exists; otherwise evict the valid line with the smallest timestamp because that line was touched least recently. This method is conceptually simple and is often the most direct way to hand-simulate LRU on short traces.

Worked cache-trace decode Consider a cache with $(C, B, S) = (6, 4, 1)$. Then the cache has $2^{6-4} = 4$ lines, arranged as $2^{6-4-1} = 2$ sets with 2 ways per set. So the offset is 4 bits, the index is 1 bit, and the remaining upper bits form the tag. Now consider the access sequence `0x24`, `0x34`, `0x2C`, `0x44`. Because the offset is 4 bits, the block numbers are `0x2`, `0x3`, `0x2`, and `0x4`. Using the low-order bit of the block number as the set index, those accesses map to set 0, set 1, set 0, and set 0, with tags 1, 1, 1, and 2, respectively. The first two accesses are compulsory misses, the third is a hit on the already resident block for `0x24`, and the fourth is another miss because it introduces a different tag into set 0. This is the basic hand-simulation loop: decode block, split into set and tag, check the resident ways, then update replacement state.

Worked timestamp-LRU trace Consider a 2-way set with two accesses already resident:

Tag	Timestamp
A	4
B	7

If the next access is a hit on A at time 8, then A's timestamp becomes 8 while B remains at 7. If the following access is a miss at time 9 and no invalid line exists, then B is evicted because it has the smaller timestamp and is therefore the least recently used line. This is the mechanical rule to

remember: every hit refreshes exactly one timestamp, and every full-set miss evicts the smallest timestamp.

Exact LRU with a bit matrix Another exact representation for an n-way set is an $n \times n$ recency matrix. Entry $M[i][j] = 1$ means way i is more recent than way j. When way k is accessed, set row k to 1, set column k to 0, and keep the diagonal in its fixed convention. After the update, the least-recently used way is the one whose off-diagonal row bits are all 0, because every other way is marked as newer than it.

Worked bit-matrix update For a 4-way set, suppose the current access is to way 2. Then row 2 is set to 1 against the other ways and column 2 is cleared:

	0	1	2	3
0	–	?	0	?
1	?	–	0	?
2	1	1	–	1
3	?	?	0	–

The question marks keep their previous values because the relative recency order among the untouched ways does not change. After several accesses, the LRU way is the row whose off-diagonal entries are all 0. That row is the one that every other way claims to be newer than.

3.9 Write Policies

Write behavior has two independent decisions. On a write hit, either update only the cache and defer memory update (*write-back*), or update both cache and memory immediately (*write-through*). On a write miss, either fetch the block then write it (*write-allocate*), or write to lower memory without installing in cache (*write-no-allocate*).

Write-back reduces memory traffic when repeated writes target the same line, but requires a dirty bit and careful eviction handling. Because dirty data can live in cache before eviction, abrupt power loss can drop recently written values that have not yet reached lower levels. Write-through simplifies coherence visibility and durability behavior because lower memory is updated on each write hit, and it does not require dirty metadata at that cache level, but it increases memory bandwidth pressure. The common pairings are write-back plus write-allocate for reuse-heavy data paths, and write-through plus write-no-allocate for streaming paths with little read-after-write reuse.

Reading write traces Write traces are easiest to interpret by separating three questions. First, does the access hit or miss in the current cache state? Second, does the policy allocate the block on a miss? Third, if a valid victim is evicted, is it dirty? In a write-back, write-allocate cache, a write miss usually installs the block and marks it dirty immediately. Each later dirty eviction contributes one writeback event. Dirty lines that remain in the cache at the end of the trace are not counted as writebacks unless the problem explicitly asks for a final flush.

Worked dirty-eviction trace Consider a write-back, write-allocate cache whose relevant set can hold only two lines. Suppose the set is initially empty and the access sequence is

$$W(A),\ W(B),\ W(A),\ R(C),$$

with C mapping to the same full set as A and B. The first write to A is a miss, so A is allocated and marked dirty. The first write to B does the same for B. The second write to A is a hit and leaves A dirty. When C is later accessed, one of the dirty lines A or B must be evicted; that eviction produces a writeback because the victim holds modified data that has not yet reached the next level.

Valid and dirty bits: meaning, need, and size impact Each cache line typically carries at least one *valid bit*. Valid= 1 means the tag/data entry contains a meaningful memory block; valid= 0 means the entry must not hit, even if tag bits happen to match after reset or replacement. A *dirty bit* is needed when that cache level uses *write-back*: dirty= 1 means the line was modified in cache and must be written to the next level before eviction; dirty= 0 means eviction can discard the line without write-back traffic. In a pure write-through level, dirty bits are not required because lower memory is updated on every write hit.

If line count is $N_{lines} = 2^{C-B}$, valid-bit storage is N_{lines} bits. Dirty-bit storage is another N_{lines} bits only when write-back is enabled at that level. Relative to data-array size 2^C bytes, overhead from these two bits alone is (Eq. (3.17)):

$$\text{overhead fraction} = \frac{n_v + n_d}{2^{B+3}}, \tag{3.17}$$

where $n_v = 1$ (valid always) and $n_d \in \{0, 1\}$ (dirty only for write-back). Example: with 32-byte blocks ($B = 5$), valid-only overhead is $1/256 \approx 0.39\%$, and valid+dirty overhead is $2/256 \approx 0.78\%$ (excluding tag, replacement, and coherence bits).

3.10 Reducing Miss Ratio

The first lever is **cache size (C)**. Increasing C reduces capacity misses because more of the working set fits in the hierarchy. However, larger caches increase hit time due to longer bit lines and decoder delays, and they consume more area and power.

The second lever is **associativity (S)**. Increasing S reduces conflict misses by providing more locations for a given index. The benefit is most significant when moving from direct-mapped to 2-way or 4-way, after which returns diminish. The cost includes increased hit time (due to the way-select multiplexer and parallel tag comparisons) and increased power consumption.

The third lever is **block size (B)**. Increasing B reduces compulsory misses by exploiting spatial locality (fetching more useful data per miss). However, if B becomes too large, it can increase conflict misses (by reducing the total number of blocks) and will always increase miss penalty (longer transfer time). So the statement "increasing block size cannot improve miss ratio" is false. With a fixed cache capacity, a larger block often improves miss ratio at first by capturing nearby useful data, but the improvement is not monotonic because fewer total lines remain available.

Hardware Prefetching Hardware prefetching targets compulsory and capacity effects by pulling data before demand. A +1 *prefetcher* fetches block $i+1$ when block i is accessed. A *stride prefetcher* detects constant deltas (for example $i, i+k, i+2k$) and fetches the next predicted address. A *Markov prefetcher* uses a correlation table to learn non-linear transitions; if address A is often followed by B then C, the predictor learns and issues those future blocks when A appears. A *hybrid prefetcher* combines a richer predictor such as Markov correlation with a simpler fallback such as next-block prefetching, so the machine can still make a reasonable guess when the correlation table has no useful entry. To prevent *cache pollution* (evicting useful data for speculative data that might not be

used), many designs use a *prefetch buffer*. This is a small, separate storage area where prefetched lines sit until they are actually demanded by the CPU. If demanded, they are moved into the main cache.

Prefetch insertion policy matters Prefetched lines are often installed with a low-priority policy such as *least recently used insertion policy (LIP)* rather than as most-recently used lines. The goal is to avoid immediately evicting useful demand data for speculative lines that may never be touched. This helps in set-associative caches, but it does not eliminate the placement limitations of a direct-mapped cache. In a direct-mapped design, a +1 prefetch can still evict useful demand data if the prefetched block maps to the same line.

Trace-driven prefetch statistics Detailed simulator outputs often distinguish three prefetch counters. *Prefetches issued* count speculative requests launched by the prefetcher. A *prefetch hit* means a later demand access found a line that was present only because of an earlier prefetch. A *prefetch miss*, sometimes called a useless prefetch, means the prefetched line was evicted before any demand access used it. These counters matter because a prefetcher can lower demand misses and still be a poor design if it wastes bandwidth or pollutes the cache badly.

3.11 Reducing Miss Penalty

Miss penalty is shaped strongly by transfer mechanics. If a block has 2^B bytes and bus transfer width is W bytes, then the number of sub-block transfers is $N_{sb} = 2^B/W$. With first-transfer startup latency T_1 and per-following transfer time T_f, block transfer time is (Eq. (3.18)):

$$MP_{full} = T_1 + (N_{sb} - 1)T_f. \tag{3.18}$$

Early restart returns data to the CPU as soon as the requested sub-block arrives rather than after full-block completion. If critical-word position is uniformly distributed and transfer order is fixed from sub-block 0 upward, average CPU-visible penalty becomes (Eq. (3.19)):

$$MP_{ER} \approx T_1 + \frac{N_{sb} - 1}{2} T_f. \tag{3.19}$$

Critical-word-first improves further by requesting the demanded sub-block first, then wrapping to complete the rest in the background. That benefit is only CPU-visible if the machine can resume execution as soon as the critical word arrives. Without early restart or an equivalent partial-consume mechanism, critical-word-first alone does not reduce the stall seen by the processor, because the core would still wait for the full block to complete before continuing.

Non-uniform critical-word distributions The uniform approximation in Eq. (3.19) is only one special case. If sub-block k is demanded with probability p_k and fixed-order transfer would deliver that word after k follow-up transfers, then the early-restart penalty is

$$MP_{ER} = \sum_{k=0}^{N_{sb}-1} p_k \left(T_1 + kT_f\right). \tag{3.20}$$

Under the same simple model, critical-word-first with early restart reduces the CPU-visible penalty to approximately T_1 because the demanded sub-block is requested first and the rest of the block finishes in the background.

Worked miss-penalty comparison Consider a 32-byte block on an 8-byte bus, so the miss repairs 4 words. Suppose each requested word transfer takes 70 ns and transfers are serialized. Baseline repair waits for all 4 words, so the visible miss penalty is

$$MP_{full} = 4 \times 70 = 280 \text{ ns}. \tag{3.21}$$

With early restart under a uniform critical-word position, the demanded word arrives after 1, 2, 3, or 4 transfers with equal probability, so the average visible penalty is

$$MP_{ER} = \frac{70 + 140 + 210 + 280}{4} = 175 \text{ ns}. \tag{3.22}$$

With critical-word-first plus early restart, the demanded word is requested first, so the processor-visible penalty returns to approximately one transfer time, 70 ns, while the rest of the block completes in the background.

Another major lever is adding lower cache levels. If L1 misses probe L2 before DRAM, L1 penalty becomes L2 average access time (Eq. (3.23)):

$$AAT_{L1} = HT_{L1} + MR_{L1} \cdot AAT_{L2}. \tag{3.23}$$

This recursive form extends naturally to deeper hierarchies.

Worked two-level AAT example Suppose $HT_{L1} = 1$ cycle, $MR_{L1} = 0.08$, $HT_{L2} = 8$ cycles, $MR_{L2,read} = 0.25$, and $DRAM_TIME = 60$ cycles. Then

$$AAT_{L2} = 8 + 0.25 \times 60 = 23 \text{ cycles}. \tag{3.24}$$

That lower-level average time becomes the miss penalty seen by L1, so

$$AAT_{L1} = 1 + 0.08 \times 23 = 2.84 \text{ cycles}. \tag{3.25}$$

The main reasoning pattern is recursive rather than additive-by-guess: first solve the lower level, then substitute that average behavior upward.

Inclusive policy: subset, not equality If L2 is *inclusive* of L1, then every line present in L1 must also be present in L2. The reverse is not required: L2 can contain many lines that are not currently in L1. So inclusivity means "L1 is a subset of L2," not "L1 and L2 contain exactly the same data."

Why many L2 ratios are read-only In a hierarchy where L1 uses write-back, write-allocate and L2 uses write-through, write-no-allocate, simulators often report separate L2 read and write traffic. Read hits and read misses form a meaningful reuse ratio because reads probe the resident L2 contents. Writes, by contrast, may simply reflect L1 writebacks or pass-through stores and may not allocate lines on a miss. For that reason, many reports define L2 hit ratio and miss ratio over *reads only* rather than over total L2 traffic.

Victim caches are also miss-penalty tools. A small fully associative buffer captures lines evicted from the primary cache. On a later conflict-driven re-reference, data can be swapped back quickly without going to DRAM. Conceptually, a victim cache provides a small amount of flexible associativity exactly where pressure appears.

Victim cache versus L2 A victim cache and an L2 do not solve exactly the same problem. A victim cache is usually the better value when misses are dominated by a small number of conflict patterns near the L1 boundary, because a few fully associative entries can undo repeated index collisions cheaply. A larger L2 is usually the better value when misses reflect broader capacity pressure or many different miss streams, because it filters a much wider set of misses rather than only recently evicted L1 lines. The right choice depends on whether the workload is mainly suffering from local conflicts or from a deeper shortage of effective capacity.

Write buffers reduce stalls by decoupling cache writes from lower-memory completion. They are especially useful when a following miss would otherwise wait behind pending writes. Correct implementation requires forwarding from the newest matching buffered write and careful ordering semantics.

3.12 Reducing Hit Time

Hit-time optimization frequently conflicts with miss-rate optimization. Smaller arrays and lower associativity usually improve hit latency. However, overly aggressive simplification can raise miss ratio enough to hurt total performance. In general, small caches and direct-mapped organizations are fast on the hit path, but they are often unacceptable unless miss ratio remains controlled by workload behavior or lower-level caches.

Address translation complicates this tradeoff. A naive *physically indexed, physically tagged* path can serialize translation lookaside buffer (TLB) lookup before cache access, increasing L1 hit latency. *Virtually indexed, physically tagged (VIPT)* caches overlap TLB translation with set indexing and preserve physical-tag correctness.

PIPT vs. VIPT timing view With virtual memory, the CPU generates a virtual address split into virtual page number (VPN) and page offset. In a *physically indexed, physically tagged (PIPT)* L1, both index and tag use physical address bits. That means the TLB must produce the physical address before cache indexing can begin. So a first-order timing model is (Eq. (3.26)):

$$HT_{L1,\mathrm{PIPT}} \approx T_{TLB} + T_{L1,\mathrm{index+array+tag}}. \tag{3.26}$$

In a *VIPT* L1, indexing starts immediately using low-order virtual bits while the TLB is translating in parallel. When the physical address arrives, the physical tag compare finalizes hit/miss. A first-order model is (Eq. (3.27)):

$$HT_{L1,\mathrm{VIPT}} \approx \max(T_{TLB},\, T_{L1,\mathrm{index+array}}) + T_{\mathrm{phys_tag_compare}}. \tag{3.27}$$

The practical effect is that VIPT can hide much of TLB latency on the L1 hit path, while PIPT often exposes it directly.

Why the VIPT constraint exists For VIPT to work cleanly, index-plus-offset bits must fit within page-offset bits (Eq. (3.28)):

$$C - S \leq P, \tag{3.28}$$

where P is page-offset width in bits. If $C - S > P$, then some index bits come from VPN bits. Different virtual addresses that map to the same physical page (*synonyms*) can then index different sets, allowing duplicated physical data in L1. That creates alias-management problems: writes

through one synonym may not immediately update the other cached copy, risking stale reads until extra coherence/synchronization machinery repairs it. This constraint often forces minimum associativity for a given L1 size and page size. For example, with $P = 12$ (4 KB pages) and $C = 15$ (32 KB L1), one needs $S \geq 3$, i.e., at least 8-way associativity. If $S = 2$ (4-way), then $C - S = 13 > 12$, so VIPT indexing would extend beyond page offset and synonym risk appears.

Concrete VIPT Associativity Check Suppose $C = 18$ and page size is 4 KB, so $P = 12$. To make the cache VIPT without indexing beyond the page offset, require

$$C - S \leq P \Rightarrow 18 - S \leq 12 \Rightarrow S \geq 6. \tag{3.29}$$

So the minimum acceptable associativity field is $S = 6$, which corresponds to $2^6 = 64$ ways. The arithmetic is simple; the main thing to remember is that the constraint is on *index plus offset*, which in (C, B, S) notation is exactly $C - S$.

Write buffers can also improve effective hit behavior for writes by decoupling write completion from the immediate critical path and by forwarding recent write data when needed. This prevents some write-after-read and read-after-write timing stalls from stretching apparent hit latency.

3.13 Virtual Memory Technology, Terminology, and Analogy to Caching

Virtual-memory terminology mirrors cache terminology closely. A virtual address is typically split into a *virtual page number (VPN)* and page offset. Translation maps VPN to a *physical frame number (PFN)* through a page table. A *translation lookaside buffer (TLB)* is a cache of recent translations. A missing translation in the TLB causes a page-table walk; if the page is not in DRAM, a page fault triggers operating system (OS) handling and secondary-storage transfer.

The analogy to caching is direct. VPN behaves like a tag, page offset behaves like block offset, and the TLB behaves like a small associative cache of mapping metadata. Likewise, DRAM behaves as a cache for backing store, while on-chip caches behave as caches for DRAM. This layered view is useful because it applies one conceptual framework to both hardware cache hierarchy and OS-managed virtual memory.

3.14 Compiler and Data-Layout Techniques

Software organization strongly affects cache performance. **Compiler layout of instructions** places frequently executed basic blocks close together in memory to improve I-cache hit rate and reduce fetch stalls. **Compiler-directed prefetching** issues non-binding prefetch instructions ahead of use, which is effective for predictable array accesses. **Data-layout choices**, including structure-of-arrays (SoA), field reordering, and padding, can reduce cache-line waste and false sharing.

Tiling (blocking) re-organizes loops to process data in smaller cache-resident regions. For example:

```
// Before tiling:
for (i=0; i<N; i++) for (j=0; j<N; j++) A[i][j] = B[j][i];
// After tiling (where b is the tile size):
for (ii=0; ii<N; ii+=b) for (jj=0; jj<N; jj+=b)
    for (i=ii; i<ii+b; i++) for (j=jj; j<jj+b; j++) A[i][j] = B[j][i];
```

Choosing the tile factor is a working-set problem. The largest profitable tile is usually the largest one whose active data footprint still fits in the effective cache space while avoiding severe mapping conflicts. For direct-mapped or low-associativity caches, this means a tile can fail even when the byte count appears to fit, because the active blocks may still alias onto the same few sets. The practical rule is therefore: first check footprint size, then check placement conflicts. This is why tiled matrix multiplication often shows much better cache behavior than a naive matrix multiply, even when both perform the same arithmetic work.

Concrete Tile-Factor Estimate Consider a cache with $(C, B, S) = (10, 6, 2)$, so total capacity is $2^{10} = 1024$ bytes, block size is 64 bytes, and associativity is 4-way. If each matrix element is 8 bytes and the tiled working set keeps TF rows of 32 elements active, then one tiled row occupies

$$32 \times 8 = 256 \text{ bytes.} \tag{3.30}$$

A footprint estimate for the tile is therefore $256 \times TF$ bytes. The largest tile that still fits in 1024 bytes is

$$256 \times TF \leq 1024 \Rightarrow TF \leq 4. \tag{3.31}$$

Here $TF = 4$ is also structurally reasonable because each row occupies four 64-byte blocks, and the 4-way organization provides enough placement freedom to avoid immediate within-tile self-conflict.

Review Emphasis: What Tiling Is Really Buying Tiling does not change the arithmetic performed by the program. It changes the order in which the program visits data so that reuse happens while the relevant blocks are still cache-resident. The review-problem reasoning is usually: identify the active data touched by one tile, multiply by bytes per element, require that footprint to fit in the effective cache capacity, and then remember that associativity can still matter. If the tile fits by byte count but all active blocks map to too few sets, conflict misses can still defeat the transformation.

Loop fusion combines independent loops that touch related data to improve temporal locality. For example:

```
for (i=0; i<N; i++) a[i] = b[i] + 1; // loop 1
for (i=0; i<N; i++) c[i] = a[i] * 2; // loop 2
// Fused:
for (i=0; i<N; i++) { a[i] = b[i] + 1; c[i] = a[i] * 2; }
```

In practice, these compiler and data-layout techniques frequently determine whether hardware cache features help or saturate.

3.15 Chapter 3 Summary

Chapter 3 develops cache memory from first principles to practical optimization. The chapter links SRAM/DRAM technology limits to hierarchical design, formalizes cache geometry with (C, B, S) math, and uses AAT to connect structure to performance. Misses are interpreted through the three C's, while replacement and write policies define behavioral tradeoffs. Improvement strategies are organized by target term: miss ratio, miss penalty, or hit time. Prefetching, victim caches, write buffers, multi-level hierarchies, simulator-style hierarchy and prefetch counters, and VIPT constraints

show how modern designs balance speed, capacity, and correctness. Finally, compiler and data-layout techniques complete the picture by showing that cache performance is a hardware-software co-design problem.

3.16 Practice Questions

The following questions reinforce the main ideas of Chapter 3 and are intended for self-checking after the summary.

1. A physically indexed, physically tagged cache has total capacity 32 KiB, block size 64 B, associativity 4-way, and 48-bit addresses. Compute the number of sets, the number of offset bits, the number of index bits, and the number of tag bits. Then compute the total tag-store size if each line stores a valid bit and a dirty bit in addition to the tag.
2. For each of the following changes, state whether it primarily targets *hit time*, *miss ratio*, or *miss penalty*: adding a victim cache, adding a write buffer, increasing associativity, increasing block size, and using critical-word-first with early restart.
3. Explain the difference between critical-word-first and early restart. Why does critical-word-first by itself not guarantee a reduction in processor-visible miss penalty?
4. A system uses 32-byte blocks, an 8-byte memory bus, and serialized memory transfers. A miss first incurs a 70 ns round trip for the requested word, and each additional word also requires 70 ns. The cache hit time is 10 ns and the miss ratio is 15%. Compute the average access time under three cases: baseline repair with no early use of returned data, early restart with a uniform probability of needing any of the four words in the block, and critical-word-first with early restart.
5. Consider a $(C, B, S) = (6, 4, 2)$ cache with LRU replacement and the access stream `0x589e`, `0xa7a1`, `0x3767`, `0xcce2`, `0xa7aa`, `0xff4d`, `0x376c`, `0x5892`. For each access, determine the set, tag, hit or miss outcome, and final contents of the tag store after the full trace.
6. A 4-way set uses an exact LRU bit-matrix implementation. Starting from a state in which way 0 is least recently used and way 3 is most recently used, update the matrix for the hit sequence $2 \to 0 \to 3 \to 1 \to 2$. Which way is least recently used at the end?
7. A VIPT L1 cache has capacity 64 KiB, block size 64 B, and page size 4 KiB. Compute the minimum associativity required so that the set index fits entirely within the page offset.
8. A miss stream repeatedly alternates between two blocks that map to the same direct-mapped L1 set. Would a small victim cache or a simple +1 prefetcher be the more natural first fix? Explain why.
9. A tiled matrix kernel keeps three $b \times b$ tiles of double-precision data active at once. If the effective L1 capacity available to the tiles is 24 KiB, what is the largest integer tile size b that satisfies the footprint constraint $3b^2 \cdot 8 \leq 24$ KiB?
10. A loop has reuse across a large array, but the full working set is much larger than the L1 cache. Explain why tiling can reduce misses even though the program performs the same loads, stores, and arithmetic overall.
11. A two-level hierarchy has $HT_{L1} = 1$ cycle, $MR_{L1} = 0.08$, $HT_{L2} = 8$ cycles, $MR_{L2,read} = 0.25$, and $DRAM_TIME = 60$ cycles. Compute AAT_{L2} and then AAT_{L1}.

12. A write-back, write-allocate cache receives the sequence: write block A, write block B, write block A again, then access block C, which maps to the same full set as A and B and forces an eviction. Describe when dirty state is created and whether the eviction causes a writeback.

Chapter 4

Pipelining

An instruction passes through several kinds of work: finding the instruction, reading operands, calculating, accessing memory, and recording a result. If each part has its own hardware, the machine need not finish the first instruction before starting the second. Like successive items moving through workstations, instructions can occupy different stages at once. The gain comes from a regular flow of finished work; a dependency or a wrong turn interrupts that flow.

4.1 Pipeline Motivation

Pipelining improves throughput by overlapping instruction execution stages in time. Instead of waiting for one instruction to complete all work before starting the next, the processor places stage registers between major datapath functions and allows multiple instructions to be in flight simultaneously. The key gain is not that one instruction becomes instantly shorter in latency, but that completed instructions per unit time rise after the pipeline fills.

4.2 Five-Stage Example Pipeline

The canonical integer pipeline has five stages (Eq. (4.1)):

$$IF \rightarrow ID \rightarrow EX \rightarrow MEM \rightarrow WB. \tag{4.1}$$

Instruction Fetch (IF) reads the instruction from the instruction memory hierarchy and advances the program counter (PC). Instruction Decode (ID) decodes opcode and operands and reads source registers. Execute (EX) performs ALU work and effective-address calculation. Memory (MEM) performs data-memory access for loads and stores. Write Back (WB) writes results to the architected register file.

Even instructions that do not logically need memory often still occupy the MEM stage in a basic in-order five-stage design so that all instructions remain aligned in a uniform stage progression. This simplifies control and preserves in-order retirement semantics. Figure 4.1 shows the overlap before hazards complicate it.

4.3 Pipeline Speedup Calculation

Suppose an unpipelined design executes one instruction in N stage-equivalent delays, each of delay T_s. Its instruction time is approximately NT_s. A pipelined design with stage registers has clock (Eq. (4.2)):

$$T_{clk,pipe} \approx T_s + T_{reg}, \tag{4.2}$$

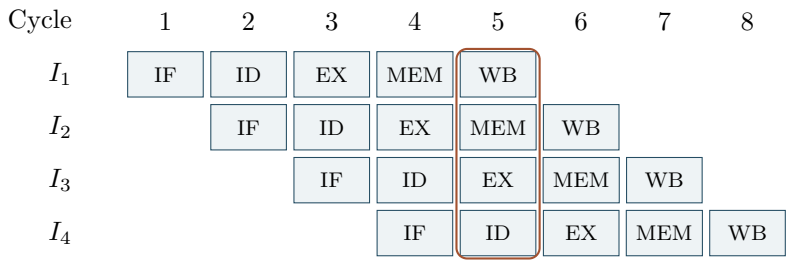


Figure 4.1: Four instructions take eight cycles in an ideal five-stage pipeline. Highlighted cycle 5 contains work from all four instructions. Assume one cycle per stage, no hazards, and separate resources wherever stages overlap.

where T_{reg} is latch/flip-flop overhead. For a long stream, ideal CPI approaches 1, so time per instruction approaches $T_{clk,pipe}$ and ideal speedup becomes (Eq. (4.3)):

$$S_{ideal} \approx \frac{NT_s}{T_s + T_{reg}}. \tag{4.3}$$

If T_{reg} were zero, this approaches N. In practice, register overhead and stalls reduce realized speedup.

Finite-length form For M instructions in an N-stage pipeline (ignoring hazards), total cycles are $N + M - 1$, so (Eq. (4.4)):

$$T_{pipe} \approx (N + M - 1)T_{clk,pipe}, \qquad T_{unpiped} \approx MNT_s. \tag{4.4}$$

This shows fill and drain overhead is more visible for short instruction sequences.

Worked pipeline-speedup calculation Suppose an unpipelined implementation has effective $CPI = 5$, and a 5-stage pipelined version keeps the same clock period. The ideal maximum speedup is therefore 5. If 25% of instructions now experience one extra stall cycle, the pipelined CPI becomes

$$CPI_{pipe} = 1 + 0.25 \times 1 = 1.25. \tag{4.5}$$

The corresponding speedup relative to the original $CPI = 5$ design is then

$$\frac{5}{1.25} = 4, \tag{4.6}$$

which is below the ideal bound because hazards consume part of the pipelined throughput gain.

4.4 CPI Model and Stall Accounting

In steady state without hazards, ideal CPI is 1 (Eq. (4.7)):

$$CPI_{ideal} = 1. \tag{4.7}$$

With hazards, the practical model is (Eq. (4.8)):

$$CPI = 1 + \text{stalls}_{avg}, \tag{4.8}$$

where stalls_{avg} is average stall cycles per instruction. This form is central in Chapter 4: almost every pipeline optimization is either reducing stall frequency, reducing stall duration, or both.

4.5 Program Characteristics Plus Hazards Produce Stalls

Hardware hazards are potential conflicts. A stall occurs only when the running program triggers that conflict. This yields a guiding principle (Eq. (4.9)):

$$\text{Program characteristics} + \text{pipeline hazards} \Rightarrow \text{stalls}. \tag{4.9}$$

For example, a branch hazard matters only when branches occur in the dynamic instruction stream. A structural hazard from a shared memory port matters only when fetch and data access overlap in a conflicting cycle.

4.6 Structural Hazards

Structural hazards arise from insufficient hardware resources. A classic example is a single-ported memory system that must serve both IF and MEM stages. If one instruction needs data memory while another needs instruction fetch in the same cycle, one must wait, causing a stall.

Common avoidance strategies are straightforward architectural provisioning: split L1 into separate instruction and data caches, add ports, duplicate critical units, or schedule around known bottlenecks. In modern designs, many obvious structural hazards are eliminated in hardware because their stall cost is predictable and severe.

CPI impact example If 20% of instructions incur one extra stall cycle due only to a structural hazard, then the average CPI becomes

$$CPI = 1 + 0.2 \times 1 = 1.2. \tag{4.10}$$

This is the same accounting pattern used for other hazards: identify the frequency of the event and multiply by its average stall contribution.

4.7 Data Hazards

Data hazards occur when the pipeline changes the order of read/write accesses to operands so that the order differs from the order in the original sequential program.

Read After Write (RAW) A *read-after-write (RAW)* hazard occurs when an instruction refers to a result that has not yet been calculated or stored by a preceding instruction. This is the most common hazard in the classic 5-stage pipeline.

```
I1:  ADD R1, R2, R3    I2:  SUB R4, R1, R5
```
(4.11)

In this case, `I2` needs `R1` in its ID/EX stage, but `I1` does not write `R1` until its WB stage. Without intervention, `I2` would read a stale value.

Write After Read and Write After Write *Write-after-read (WAR)* and *write-after-write (WAW)* hazards do not typically occur in the simple 5-stage in-order pipeline because instructions read early (ID) and write late (WB) in a fixed order. However, they become critical in out-of-order execution or pipelines with multiple completion units of varying latencies.

4.7.1 Forwarding (Bypassing)

Forwarding allows a result to be used as an input to a subsequent instruction before it is written to the register file. If the result is already available in a pipeline register (such as EX/MEM or MEM/WB), hardware can forward it directly to the ALU inputs. This eliminates many RAW stalls.

4.7.2 Load-Use Stalls

Forwarding cannot solve every RAW hazard. If a load instruction is followed immediately by an instruction that uses the loaded value, the data is not available until the end of the MEM stage, which is too late for the EX stage of the following instruction.

```
I1:  LW R1, 0(R2)    I2:  ADD R3, R1, R4
```
(4.12)

This case requires a one-cycle load-use stall even with forwarding. Compilers attempt to schedule independent instructions into these "load-delay" slots to maintain performance.

4.7.3 Same-Cycle Register-File Write Then Read

One subtle pipeline detail is the timing of register-file writes relative to reads in the same cycle. If the register file writes in the early part of the cycle and decode reads in the later part, then an instruction in ID can sometimes see a value written back by an older instruction in WB in that same cycle. If that timing is not supported, the machine needs either an explicit WB-to-ID bypass path or an extra stall. This detail matters when reading timing diagrams because two pipelines with the same five visible stage names can still differ by one cycle on closely spaced dependences.

4.8 Control Hazards

Control hazards occur because the fetch stream does not immediately know the correct next PC for branches and jumps. The branch creates uncertainty in both target and direction (taken/not taken), and the pipeline may fetch wrong-path instructions that must later be squashed.

CPI impact example If branches occur every five instructions on average (branch frequency $= 0.2$) and each branch introduces one average stall cycle in a simple pipeline, then

$$CPI = 1 + 0.2 \times 1 = 1.2. \tag{4.13}$$

This simple calculation captures why even modest branch penalties have visible throughput cost.

4.8.1 Delay Slots

A *delay slot* ISA semantic says that one or more instructions after a branch execute before control transfer takes effect. The compiler tries to fill these slots with useful independent work. This can reduce explicit branch stalls when suitable instructions exist, but becomes increasingly difficult as pipelines deepen and dependence constraints tighten.

How software fills delay slots Delay-slot instructions execute whether the branch is taken or not. So a compiler may place an instruction into a delay slot only if doing so is safe on *both* outcomes. The usual search order is:

1. move an independent instruction from before the branch down into the slot,
2. if control-flow structure permits, move an instruction from a successor block only when it is safe for both paths, or
3. leave the slot as a `NOP` if no safe candidate exists.

This is why branch schedules often still leave some slots unfilled: correctness takes priority over slot utilization.

4.8.2 Squash Slots with Likely Bits

Some designs expose *squash-slot* behavior via branch-likely style semantics. The compiler places a candidate instruction near the branch and encodes likely behavior. Hardware keeps or squashes that instruction based on actual branch resolution and likely-bit policy. This improves performance when heuristics are good but complicates ISA semantics and compiler scheduling.

How squash-slot filling differs Squash slots are more permissive than delay slots because wrong-path instructions may be discarded. If the machine assumes a branch is likely taken, the compiler can place instructions from the predicted taken path into those slots. If the prediction is wrong, hardware squashes them. So the key distinction is:

- **delay slot:** instruction must be correct regardless of branch outcome,
- **squash slot:** instruction only needs to be correct on the predicted path because hardware may cancel it.

4.9 Branch Target Buffer (BTB)

The *branch target buffer (BTB)* predicts where a branch goes (target prediction) early enough to guide fetch.

4.9.1 BTB with Tags

A tagged BTB stores branch PC tags and associated targets. On fetch, the BTB is indexed and tag-compared; a valid tag hit yields a predicted target. Tagging detects when the indexed entry belongs to a different branch, reducing destructive *aliasing* at the cost of more hardware area and lookup work.

4.9.2 BTB without Tags

A tagless BTB behaves like a hashed direct lookup table from index to target metadata. It is fast and compact but prone to *aliasing*: unrelated branches may map to the same entry and pollute each other's predictions. This can still be acceptable if the table is sized well and collision rate is low for target workloads, trading accuracy for substantial area savings.

4.9.3 Hardware Return Address Stack

Function returns are indirect branches whose targets are difficult for generic BTB logic. A small hardware *return-address stack (RAS)* records call return PCs and pops on return, producing very high return-target accuracy in common call/return patterns. This is one of the most cost-effective branch-target optimizations.

Worked BTB/RAS trace Assume the BTB is indexed by $PC[7:4]$ and the RAS is initially empty. Now consider the short control-flow sequence

```
0x4000:  CALL 0x4A00
0x4A00:  BREQZ R1,0x4AF0 (taken)
0x4AF8:  RET
```

The call pushes return address `0x4004` onto the RAS and creates or refreshes a BTB entry for the call target `0x4A00`. The conditional branch updates its BTB entry with the taken target `0x4AF0`. The return then pops the top of the RAS and predicts target `0x4004`. The main bookkeeping distinction is that the BTB remembers branch and jump targets by PC, while the RAS handles the nested call-return discipline explicitly.

BTBs still matter for unconditional control transfers A BTB is not useful only for conditional branches. Even if every control transfer is unconditional, fetch still benefits from knowing the target early. Without a BTB or some equivalent early-target mechanism, the pipeline may need to wait until later decode or execute logic computes the jump target, creating avoidable fetch bubbles.

4.10 Whether-To Branch: Direction Prediction

Target prediction answers where a branch goes. *Direction prediction* answers whether it is taken. A simple static method is software heuristics, especially *backward taken / forward not taken (BTFNT)*: loop back-edges are often taken, many forward conditionals are often not taken. Compilers can also use profile-guided statistics to set likely-direction metadata (for example branch-likely annotations).

Dynamic predictors improve further by learning runtime behavior (for example saturating counters and history-based schemes). The essential Chapter 4 lesson is that deeper pipelines require higher direction accuracy because misprediction penalty grows with the number of in-flight wrong-path stages.

A very simple baseline is the *always-taken* predictor. It ignores PC and history entirely and predicts that every conditional branch will be taken. This is crude, but it remains a useful reference point because many loop back-edges are taken most of the time. More sophisticated predictors should therefore be judged not only against one another, but also against simple baselines such as backward taken / forward not taken and always taken.

4.11 Simple Dynamic Direction Predictors

The simplest dynamic predictors store a small amount of state per static branch or per BTB entry.

4.11.1 One-Bit Prediction

A *1-bit predictor* records only the last observed direction for a branch. In a compact BTB-style design, this may appear as a single prediction bit associated with the target entry. If the last outcome was taken, predict taken next time; if the last outcome was not taken, predict not taken. This adapts quickly, but it also flips too easily. For example, a loop-closing branch that is taken many times and then falls through once on loop exit will usually be mispredicted again on the next loop entry because the last outcome was not taken.

4.11.2 Smith Two-Bit Saturating Counters

A *Smith predictor* replaces the single bit with a small saturating counter, most commonly two bits. The four counter states are commonly interpreted as strongly not taken, weakly not taken, weakly taken, and strongly taken. Prediction comes from the high-order tendency of the counter, while update moves the counter only one step toward the actual outcome. This hysteresis avoids overreacting to one unusual branch result. In the loop example above, a two-bit counter usually mispredicts only on the loop exit rather than on both the exit and the next re-entry.

4.12 Global and Local History Predictors

History-based predictors use recent branch outcomes to select among several learned taken/not-taken tendencies rather than relying on one counter per static branch.

4.12.1 GShare

GShare uses a global history register (GHR) like other global-history predictors, but instead of concatenating PC bits and history bits it combines them with an exclusive-or (XOR). If the predictor uses n index bits, then a common form is

$$index = PC_{low}[n-1:0] \ \oplus \ GHR[n-1:0]. \tag{4.14}$$

The XOR can reduce some destructive aliasing patterns by spreading correlated PCs across the counter table more evenly than a plain concatenation. Conceptually, GShare and GSelect use the same ingredients; they differ mainly in how PC and history are mixed to form the table index.

4.12.2 GSelect

GSelect is a simple global-history predictor. It keeps a *global history register (GHR)* containing the most recent branch outcomes and combines that history with a small hash of the current PC to index a table of saturating counters. If the PC hash contributes p bits and the GHR contributes h bits, then the predictor selects one of 2^{p+h} counters. Prediction comes from the selected counter, usually by treating the upper half of the counter states as taken and the lower half as not taken. After the true outcome is known, the predictor updates that same counter and then shifts the actual outcome into the GHR. The key idea is that one branch can learn differently depending on the recent behavior of other branches.

Worked GSelect trace The following trace makes the lookup order explicit. Assume a 2-bit GHR whose rightmost bit is the newest outcome, a 1-bit PC hash equal to $PC[2]$, and 2-bit Smith counters where `00/01` predict not taken and `10/11` predict taken. Let every counter begin in state `10`, and let the initial GHR be `01`. For the four resolved branches below, prediction always uses the *current* PC hash and the *current* GHR. Only after that prediction is the selected counter updated and the actual outcome shifted into the GHR.

Time	PC	Act.	$PC[2]$	GHR_T	Ctr_T	Pred.	Ctr_{T+1}	GHR_{T+1}
T_0	`0x2017`	N	`1`	`01`	`10`	T	`01`	`10`
T_1	`0x2009`	T	`0`	`10`	`10`	T	`11`	`01`
T_2	`0x2017`	T	`1`	`01`	`01`	N	`10`	`11`
T_3	`0x2009`	N	`0`	`11`	`10`	T	`01`	`10`

After T_3, the final GHR is `10`. The visited counter at $(PC[2], GHR) = (0, \texttt{10})$ has been trained up to `11`, and the counter at $(0, \texttt{11})$ has been trained down to `01`. This is the mechanical rule to remember: read the counter indexed by the *old* GHR, make the prediction, update that same counter with the actual outcome, then shift the actual outcome into the GHR.

4.12.3 Yeh/Patt

Yeh/Patt separates branch history into two levels. A first table, indexed by the branch PC, stores a short per-branch history pattern. That recorded history then indexes a second table of saturating counters. In other words, GSelect uses one shared global history for every branch, while Yeh/Patt gives each branch its own history entry before consulting the prediction counters. Prediction therefore proceeds in two steps:

1. Use the branch PC to read the branch's recorded history.
2. Use that history to select the saturating counter that predicts taken or not taken.

Update proceeds in the same order in reverse: first train the selected counter toward the actual outcome, then shift the actual outcome into that branch's local history register. This structure is especially useful when different static branches follow different repetitive patterns.

Worked Yeh/Patt trace Now assume a 2-bit history table indexed by $PC[3:2]$ and a shared pattern table of 2-bit Smith counters. Let every history-table entry begin at `00`, and let every pattern-table counter begin at `01`. The sequence below uses two PCs, `0x3B98` and `0x3B9F`, whose hashes are `10` and `11`, respectively.

Time	PC	$PC[3:2]$	Hist_T	Ctr_T	Pred.	Act.	Ctr_{T+1}	Hist_{T+1}
T_0	`0x3B98`	`10`	`00`	`01`	N	N	`00`	`00`
T_1	`0x3B9F`	`11`	`00`	`00`	N	T	`01`	`01`
T_2	`0x3B98`	`10`	`00`	`01`	N	N	`00`	`00`
T_3	`0x3B9F`	`11`	`01`	`01`	N	N	`00`	`10`

This trace makes the two-level structure explicit. At T_1, branch `0x3B9F` still reads local history `00`, so it trains the shared pattern entry for history `00`, even though it is a different static branch from `0x3B98`. At T_3, however, the same branch now reads local history `01`, so it consults and updates a different pattern-table counter. The history table is branch-specific; the pattern table is shared by history value.

4.13 Perceptron Direction Prediction

Perceptron prediction replaces small counter tables with a simple linear classifier. For one static branch, let the history bits be encoded as

$$x_i = \begin{cases} +1, & \text{if history bit } i \text{ is taken} \\ -1, & \text{if history bit } i \text{ is not taken} \end{cases} \tag{4.15}$$

and let w_0 be a bias weight. The predictor forms

$$y = w_0 + \sum_{i=1}^{n} w_i x_i. \tag{4.16}$$

If $y \geq 0$, it predicts taken; otherwise it predicts not taken. After the real outcome is known, a common training rule is to update when the prediction was wrong or when the confidence $|y|$ is below a threshold θ. If the target value is $t = +1$ for taken and $t = -1$ for not taken, then the update is

$$w_0 \leftarrow w_0 + t, \qquad w_i \leftarrow w_i + t x_i. \tag{4.17}$$

The perceptron predictor can learn longer correlations than small counter tables, but it costs more arithmetic and storage per prediction.

Worked perceptron update Suppose the active perceptron has weights $[1, 2, -1, 0, 1]$, where the first value is the bias w_0, and suppose the current 4-bit global history is `1010`. Using taken $= +1$ and not taken $= -1$, that history becomes

$$x = [+1, -1, +1, -1].$$

The perceptron output is therefore

$$y = 1 + 2(+1) + (-1)(-1) + 0(+1) + 1(-1) = 3. \tag{4.18}$$

Because $y \geq 0$, the prediction is taken. If the true outcome is not taken, then the prediction is wrong and training must occur with target $t = -1$. The updated weights become

$$w_0 \leftarrow 1 - 1 = 0, \qquad [w_1, w_2, w_3, w_4] \leftarrow [2, -1, 0, 1] + [-1, +1, -1, +1] = [1, 0, -1, 2].$$

After the same outcome is shifted into the history register, the new GHR becomes `0100` if the newest bit is placed on the right.

4.14 Hybrid and Tournament Predictors

A *hybrid predictor* combines two or more component predictors and uses a chooser to decide which component to trust for a branch context. A common *tournament predictor* stores a small table of saturating counters indexed by branch PC or by a short hash of PC and history. When the chooser leans one way, the machine uses one predictor family; when it leans the other way, it uses the competing family.

The usual training rule is simple. If the component predictors disagree and one is correct while the other is wrong, the chooser is updated toward the correct component. If both are correct or both

are wrong, the chooser often remains unchanged because that outcome did not distinguish which component was better for that context. This organization is useful when one predictor handles short repetitive patterns well while another captures longer correlations better. For example, a tournament design may arbitrate between a Yeh/Patt predictor and a perceptron predictor.

Predictor-quality metrics Two branch-prediction ratios appear repeatedly in simulator output. If N_{inst} is total dynamic instruction count, N_{branch} is dynamic branch count, and $N_{correct}$ is the number of correct branch predictions, then

$$f_{branch} = \frac{N_{branch}}{N_{inst}} \tag{4.19}$$

is the dynamic branch fraction and

$$Acc_{branch} = \frac{N_{correct}}{N_{branch}} = 1 - \frac{N_{mispred}}{N_{branch}} \tag{4.20}$$

is branch-prediction accuracy. These metrics answer different questions: branch fraction tells how often control-flow prediction matters, while prediction accuracy tells how often the predictor was correct when it mattered.

Prediction and update timing Prediction happens near fetch, but the true branch outcome becomes known only later when the branch resolves or retires. A pipelined predictor therefore cannot always train itself using only its *current* tables, because the history pattern, perceptron sum, or chooser state seen at update time may differ from the state that existed when the prediction was originally made. Practical hardware and simulators solve this by preserving a small prediction-time snapshot, such as the selected history entry or chooser index, until the branch is ready to update the predictor.

4.15 Timing-Diagram Interpretation

Timing diagrams are not just bookkeeping artifacts. They are often enough to infer what forwarding paths, register-file timing rules, and structural resources a pipeline must have. A reliable reading order is:

1. Identify true, anti-, and output dependences from the program text before looking at the diagram.
2. Mark every repeated stage occupancy in the diagram as a stall candidate.
3. For each stall candidate, decide whether the cause is data, structural, or control.
4. Infer which hardware feature would be required to remove that stall.

Common structural cues If one instruction is fetching while another is accessing data memory in the same cycle without conflict, then the design must have either separate instruction and data paths or enough memory-porting to support both operations. If fetch pauses exactly when a load or store reaches MEM, a shared memory path is the likely cause.

Common forwarding cues If a dependent ALU instruction reaches EX before the producer reaches WB, some bypass path must exist. If a load-use pair still suffers one bubble but longer ALU-to-ALU chains do not, the machine likely has EX/MEM-style forwarding but still cannot make load data available early enough for the immediately following EX stage. If a consumer in ID proceeds only when the producer is in WB, the design may be relying on WB-to-ID forwarding or on same-cycle register-file write-then-read timing rather than on an earlier bypass path.

Register-file timing cue Suppose a producer writes back in cycle k and a dependent consumer reads registers in ID in that same cycle k. If the consumer does not stall further, then the machine must support either same-cycle write-before-read behavior or an equivalent WB-stage bypass into decode. If the consumer waits one more cycle, then neither mechanism is present.

Control-hazard cue Wrong-path instructions that appear briefly and are then crossed out or squashed indicate speculative fetch. A gap after a branch with no wrong-path work suggests conservative branch resolution rather than aggressive prediction. Likely-bit or squash-slot behavior is visible when an instruction after the branch is fetched speculatively but only conditionally allowed to complete.

A fuller checklist remains in Appendix A, but these rules are the minimum set needed to infer pipeline structure from the kinds of timing traces used throughout this text.

4.16 Chapter 4 Summary

Chapter 4 frames pipelining as throughput engineering with explicit hazard management. The five-stage model clarifies stage roles (IF, ID, EX, MEM, WB), while speedup equations show why ideal gains are reduced by latch overhead and stalls. CPI is modeled as $1 + \text{stalls}_{avg}$, with stalls explained by the interaction of program behavior and hazard structure. Structural hazards are reduced through resource provisioning; control hazards are addressed through delay/squash semantics and prediction structures. BTBs predict branch targets, RAS handles returns efficiently, and direction prediction determines whether control transfer occurs, with software heuristics like backward taken/forward not taken and simple baselines like always taken providing reference points. Simple dynamic predictors such as one-bit prediction and Smith two-bit counters add robustness with very little hardware, while history-based predictors such as GShare, GSelect, and Yeh/Patt, neural-style schemes such as perceptrons, and hybrid tournament predictors refine that prediction further by learning correlations across prior branch outcomes. Operationally, those predictors are best understood as table- or vector-update procedures: identify the prediction-time index or history, read the predictor state, make the prediction, and only later train using the correct saved context. Because prediction and training happen at different times in the pipeline, practical predictors must also preserve enough prediction-time context to update correctly later. Timing diagrams then tie the whole chapter together by exposing which hazards, forwarding paths, and same-cycle register-file behaviors a concrete pipeline must implement. Together, these ideas define practical pipeline performance analysis and design.

4.17 Practice Questions

The following questions reinforce the main ideas of Chapter 4 and are intended for self-checking after the summary.

1. An unpipelined design has $CPI = 5$. A new implementation uses five pipeline stages. What is the maximum possible speedup if the pipelined clock period is unchanged? Then compute the new CPI if 25% of instructions suffer a single-cycle stall.
2. A scalar pipeline has base $CPI = 1$. Structural hazards add an average of 0.05×1 cycles per instruction, load-use hazards add 0.12×1, and branch mispredictions add 0.15×2. Compute the final CPI.
3. For the sequence below, list the RAW, WAR, and WAW dependences. Then identify which forwarding path is most important for reducing stalls.

```
LW R1, 0(R2)
ADD R3, R1, R4
SUB R5, R3, R6
SW R5, 0(R7)
```

4. A timing diagram shows that instruction fetch continues in the same cycle that another instruction accesses data memory, that a dependent ALU consumer reaches EX before the producer reaches WB, and that a decode-stage consumer can read a value in the same cycle an older instruction writes it back. What does each of these observations imply about the hardware organization?
5. Explain the distinct roles of the branch target buffer (BTB), return-address stack (RAS), and direction predictor. Why does a BTB remain useful even when the branch direction is trivial?
6. A BTB is indexed by `PC[7:4]`. The return-address stack is initially empty. Simulate this short instruction stream: `0x4000: CALL 0x4A00`, `0x4A00: BREQZ R1,0x4AF0` (taken), `0x4AF8: RET`, `0x4004: ADD`. State the BTB effect of each control instruction and the final contents of the top of the RAS.
7. A GSelect predictor uses a 2-bit global history register and indexes the predictor table with `PC[2]` and the current GHR. The row for `PC[2]=0` is `[01,10,11,00]` for GHR `00,01,10,11`, the row for `PC[2]=1` is `[10,01,00,11]`, and the initial GHR is `01`. Simulate the following branch sequence in order: `(0x2004,T)`, `(0x2008,N)`, `(0x2004,T)`, `(0x200C,T)`. For each access, give the selected table entry before update, the prediction, the updated counter, and the new GHR.
8. A Yeh/Patt predictor uses `PC[3:2]` to index a history table, and each history register is 2 bits wide. Explain how the history-table lookup and the pattern-table lookup differ from the indexing rule used in GSelect. Then state one situation in which Yeh/Patt can distinguish branches that simple global-history indexing tends to alias together.
9. A student claims that if every Yeh/Patt local-history register is initialized to `00`, then the first prediction for every branch must be not taken. Is that claim correct? Explain which structure determines the actual first prediction and why the history table alone is not enough.
10. A perceptron predictor uses $G = 4$ bits of global history. The active perceptron has weights $[1, 2, -1, 0, 1]$, the current GHR is `1010`, and the next actual branch outcome is not taken. Compute the output y, state the prediction, determine whether training occurs, and give the

new GHR after update. You may assume the threshold rule $\theta = \lfloor 1.93G + 14 \rfloor$ used in this text.

Chapter 5

Integrated Synthesis and Cache Simulation Concepts

A simulator reports fewer cache misses after a change, yet the modeled program runs more slowly. That result is possible: the larger cache may take longer to access, or the new policy may generate expensive traffic elsewhere. To explain it, we need to follow an event through the whole machine instead of treating each counter as a separate verdict.

5.1 From an Event to an Explanation

Begin with one demand access. Ask which level receives it, whether a miss causes a fill or write-back, and which counters record each event. Then ask whether the resulting delay actually stalls the processor or overlaps other work. This sequence connects the cache model to runtime and prevents a common mistake: interpreting more activity as more useful work.

5.2 Integrated Themes from Chapters 1 and 2

The first two chapters establish the measurement framework used throughout the rest of the book. An architectural change should first be classified by what it is trying to improve: parallelism, speculation, or some combination of the two. Its performance impact must then be evaluated with the correct metric, usually runtime for one job or throughput for a stream of jobs. The central CPU-time equation,

$$\text{CPU time} = \text{IC} \times \text{CPI} \times \text{CT},$$

turns that reasoning into a concrete tradeoff among instruction count, pipeline efficiency, and clock period. Within that framework, Amdahl's law bounds fixed-size speedup, Gustafson's law explains scaled-size behavior, and the choice between arithmetic and harmonic means determines whether a summary is mathematically meaningful.

Worked check Consider an optimization that accelerates 30% of runtime by 2×. The total speedup is

$$S = \frac{1}{(1 - 0.3) + \frac{0.3}{2}} = \frac{1}{0.85} \approx 1.176. \tag{5.1}$$

The unaccelerated fraction remains in the denominator and bounds overall speedup.

Worked check: maximum possible speedup Consider a mechanism that accelerates 25% of the original benchmark runtime and makes that portion arbitrarily fast. The maximum total speedup is

$$S_{\max} = \frac{1}{1 - 0.25} = \frac{4}{3} \approx 1.33. \tag{5.2}$$

This is the standard way to interpret the *maximum* speedup of a partial optimization.

5.3 Integrated Themes from Chapter 3

Chapter 3 adds the memory-hierarchy side of the performance model. That means translating a cache specification such as (C, B, S) into sets, ways, and tag/index/offset fields; distinguishing compulsory, conflict, and capacity misses; and understanding how replacement and write policies alter observed traffic. It also means using

$$AAT = HT + MR \cdot MP$$

as a compact summary of hierarchy behavior while remembering that cache size, block size, and associativity can influence all three terms in different ways.

Worked check Consider a cache with $HT = 1.8$ ns, $MR = 0.06$, and $MP = 28$ ns (Eq. (5.3)):

$$AAT = 1.8 + 0.06 \cdot 28 = 3.48 \text{ ns}. \tag{5.3}$$

Worked check: repeated trace reasoning Consider a direct-mapped L1 with $(C, B, S) = (8, 4, 0)$, write-back/write-allocate, and the repeated access sequence

$$R\ 0x41F1,\ R\ 0xB54A,\ W\ 0x4204,\ W\ 0xCA3B,\ R\ 0x420E,\ W\ 0xB537. \tag{5.4}$$

With $B = 4$, the six accesses touch the five blocks $\{0x41F, 0xB54, 0x420, 0xCA3, 0xB53\}$ because $0x4204$ and $0x420E$ lie in the same block. The first iteration therefore has 5 compulsory misses. The key structural fact is that blocks $0xCA3$ and $0xB53$ map to the same direct-mapped set, so after warm-up the sequence continues to suffer two conflict misses per iteration. Over iterations 2–10, that produces 18 additional misses. Those 18 misses are the non-compulsory misses for the full run, and because the conflicting writes repeatedly evict dirty lines, the full sequence produces 19 writebacks if dirty lines are *not* flushed after the final access.

Victim cache and +1 prefetch contrast That same trace separates two common remedies cleanly. A small victim cache can eliminate the recurring two-block ping-pong at one set; after warm-up, the combined L1+victim-cache miss count can drop to zero because each conflicting block is recovered from the victim structure instead of DRAM. By contrast, a +1 prefetcher is not automatically a cure. In this trace it can prefetch block $0x420$ early enough to remove one first-iteration demand miss, but it does not remove the recurring conflict between $0xCA3$ and $0xB53$. This is the key design lesson of the trace: next-block prefetching and victim caching address different miss mechanisms.

5.4 Integrated Themes from Chapter 4

Chapter 4 connects pipeline structure to observed performance. Reading a timing diagram is therefore an exercise in inference: the reader is reconstructing hazards, bypass paths, memory behavior, and branch-handling policy from visible stage occupancy. The same chapter also introduces the progression from static branch handling to dynamic predictors such as always taken, one-bit and Smith-counter schemes, GShare, GSelect, Yeh/Patt, perceptrons, and hybrid tournament predictors.

Pipeline Speedup

Assume a 5-stage design where each stage has 250 ps logic delay and pipeline-register overhead is 30 ps. Unpipelined instruction time is $5 \times 250 = 1250$ ps. Pipelined clock is $250 + 30 = 280$ ps. Ideal long-run speedup is (Eq. (5.5)):

$$S \approx \frac{1250}{280} \approx 4.46. \tag{5.5}$$

This is below 5 because register overhead is non-zero.

CPI with Branch Stalls

If branch frequency is 18% and each branch contributes an average of 1.6 stall cycles (Eq. (5.6)):

$$CPI = 1 + 0.18 \times 1.6 = 1.288. \tag{5.6}$$

This illustrates how even moderate branch penalties raise CPI above the ideal value of 1.

Integrated CPI Accounting

Suppose a scalar core has base $CPI = 1$. Loads are 30% of all instructions, the L1 data-cache miss ratio on loads is 5%, and each such miss adds 20 cycles. Branches are 18% of all instructions, the misprediction rate is 10%, and each misprediction adds 2 cycles. Then the added CPI from load misses is

$$0.30 \times 0.05 \times 20 = 0.30, \tag{5.7}$$

while the added CPI from branch mispredictions is

$$0.18 \times 0.10 \times 2 = 0.036. \tag{5.8}$$

So the total CPI estimate is

$$CPI \approx 1 + 0.30 + 0.036 = 1.336. \tag{5.9}$$

This kind of combined accounting is the most useful way to compare bottlenecks: all stall mechanisms are translated into the common unit of cycles per retired instruction.

Software Branch Handling

When a branch has delay slots, every instruction moved into those slots must be safe regardless of whether the branch is taken. When a branch instead has squash slots under an “assume taken” policy, the compiler may place instructions from the predicted taken path there, because wrong-path entries will be canceled. The resulting placement rule is:

1. fill delay slots only with path-independent instructions,
2. fill squash slots from the predicted path when profitable, and
3. leave `NOP`s when correctness would otherwise be uncertain.

5.5 Cache Simulation Conceptual Model

The cache simulation framework models hierarchy behavior at the level of addresses, tags, and policy events rather than payload data values. A common parameterization is (Eq. (5.10)):

$$(C_1, B, S_1, C_2, S_2, \{\text{MIP,LIP}\}, \{\text{None,+1,Markov,Hybrid}\}, r). \tag{5.10}$$

In this notation, C controls capacity (2^C bytes), B controls block size (2^B bytes), and S controls associativity (2^S ways). Here *most recently used insertion policy (MIP)* and *least recently used insertion policy (LIP)* are common insertion-policy options.

5.6 Hierarchy Semantics in the Model

The hierarchy model treats L1 accesses separately from L2 internals, while L2 requests are viewed as arriving from an upper-level requester. This separation makes it easier to reason about statistics at each level without conflating local policy with downstream timing.

Write strategy also changes the meaning of those statistics. *Write-back, write-allocate (WBWA)* behavior in L1 emphasizes dirty-state tracking and deferred propagation, whereas *write-through, write-no-allocate (WTWNA)* behavior in L2 emphasizes immediate lower-level visibility and no allocation on write misses.

The ordering of read-miss repair and writeback events can also matter. Different servicing orders change replacement-state evolution and therefore alter downstream hit, miss, and writeback counts. If a lower level is disabled in the model, it is often best understood as an always-miss transit layer whose contribution to AAT is only the cost of passing requests to the next active level.

5.7 Prefetching Concepts

+1 *prefetching* predicts the next sequential block ($X + 1$) after a demand miss on block X. *Markov prefetching* instead models miss-stream transitions as a finite-state table, associates each source miss block with a bounded set of likely successors and their frequencies, and selects a prediction from the highest-frequency successor under a deterministic tie-break rule. *Hybrid prefetching* combines correlation-based prediction with sequential fallback when correlation information is absent.

To interpret prefetch behavior correctly, the simulator must also distinguish demand-filled lines from prefetched lines. A prefetch metadata bit provides that distinction. The first demand use of a prefetched line contributes to prefetch-hit interpretation, while eviction before first demand use contributes to prefetch-miss interpretation.

Worked prefetch-counter interpretation Suppose a simulator reports 120 prefetched lines, 70 prefetch hits, and 25 prefetched lines evicted before first use. The 70 prefetch hits show that many speculative fills were in fact useful: later demand accesses avoided ordinary misses because of earlier prefetching. The 25 unused evictions show that some prefetched lines consumed bandwidth and space without helping demand accesses. The remaining prefetched lines are ambiguous without more context, because some may still be resident and not yet demanded. This is why prefetch usefulness is not judged by one counter alone; both benefit and waste must be read together.

5.8 Statistics and AAT Interpretation

Typical hierarchy statistics include access counts, hit/miss counts, hit/miss ratios, writeback counts, and prefetch effectiveness counters. In two-level analysis (Eq. (5.11)):

$$AAT_{L1} = HT_{L1} + MR_{L1} \cdot MP_{L1}, \qquad MP_{L1} = AAT_{L2}. \tag{5.11}$$

$$AAT_{L2} = HT_{L2} + MR_{L2,read} \cdot DRAM_TIME. \tag{5.12}$$

$$DRAM_TIME = DRAM_AT + DRAM_AT_PER_WORD \cdot WORDS_PER_BLOCK. \tag{5.13}$$

This form emphasizes that higher-level penalty is the lower-level average access behavior, not a fixed constant.

5.9 Conceptual Tradeoffs Across Parameters

Increasing C generally reduces capacity misses but tends to raise hit time. Increasing S reduces conflict misses but increases lookup complexity. Increasing B can exploit spatial locality but may also increase pollution and miss penalty. More aggressive prefetching can reduce compulsory misses yet increase bandwidth pressure and useless fills.

5.10 Chapter 5 Summary

Chapter 5 draws together the earlier performance, pipeline, and memory ideas into one simulation-oriented view. Pipeline timing and cache behavior are not separate topics; they are two views of the same broader question of where cycles are spent and why. The simulation concepts in this chapter provide a concrete way to observe those tradeoffs in controlled workloads. That same simulator-oriented reasoning extends naturally to branch-prediction accuracy, fetched-versus-retired instruction counts, queue pressure, and reorder-buffer pressure, which are developed further in Chapter 6. A grouped formula index for the full text is provided in Appendix B.

5.11 Practice Questions

The following questions reinforce the integrated ideas of Chapter 5 and are intended for self-checking after the summary.

1. A hierarchy reports the following parameters:

   $$HT_{L1} = 1, \quad MR_{L1} = 0.10, \quad HT_{L2} = 7, \quad MR_{L2,read} = 0.20, \quad \text{DRAM_TIME} = 80.$$

   Compute AAT_{L2} and AAT_{L1}. Then explain why the L1 miss penalty is modeled as the lower-level *average* access time rather than as a fixed constant.

2. A simulator reports 120 prefetched lines, 70 prefetch hits, and 25 prefetched lines evicted before first use. What do these three counters say about prefetch usefulness and prefetch waste? What additional information would you want before concluding that the prefetcher is a net win?

3. In a hierarchy that uses write-through, write-no-allocate behavior at the lower level, why is it often more meaningful to report the lower-level miss ratio over *reads only* rather than over all accesses?
4. Two cache configurations have the same capacity and associativity. Configuration A has lower hit time but higher miss ratio. Configuration B has higher hit time but lower miss ratio. Explain why the better design cannot be chosen correctly without computing or estimating average access time.
5. Explain why the servicing order of read-miss repair and writeback traffic can change later hierarchy statistics even when the high-level policy choices are unchanged.
6. A scalar core has base $CPI = 1$. Loads are 30% of all instructions, the L1 data-cache miss ratio on loads is 5%, and each such miss adds 20 stall cycles. Branches are 18% of all instructions, the branch misprediction rate is 10%, and each misprediction adds 2 cycles. Estimate the total CPI and identify which source of stalls contributes more.

Chapter 6

Dynamic ILP: Superscalar Scheduling and Out-of-Order Execution

A load misses in the cache. The next instruction needs its result, but a later instruction could add two values that are already available. Must the whole machine wait? Out-of-order execution begins with that question and earns its complexity by answering no, while still making the final result look as though the program ran in order.

6.1 Why Cycles per Instruction Can Go Below One

In basic single-issue pipelines, designers often treat cycles per instruction (CPI) near 1 as a practical lower bound. Superscalar execution challenges that assumption by allowing multiple instructions to make progress in the same cycle. This broader goal is usually described as instruction-level parallelism (ILP). In this context, instructions per cycle (IPC) is the natural companion metric, as shown in Eq. (6.1):

$$IPC = \frac{1}{CPI}. \tag{6.1}$$

So achieving $CPI < 1$ is exactly equivalent to achieving $IPC > 1$. A useful way to view design limits is that average retire throughput cannot exceed the average throughput of the stages before it (Eq. (6.2)):

$$IPC_{retire} \leq \min(IPC_{fetch},\ IPC_{dispatch},\ IPC_{execute},\ IPC_{state\ update})\,. \tag{6.2}$$

If any major stage can only handle one instruction per cycle on average, then the whole machine cannot retire two instructions per cycle on average, even if another stage is wide enough to do so. In other words, the minimum term in Eq. (6.2) is the throughput bottleneck.

Each term corresponds to a different possible limit. IPC_{fetch} asks how many useful instructions the front end can supply per cycle. $IPC_{dispatch}$ asks how many decoded instructions can be prepared and sent onward. $IPC_{execute}$ asks how many instructions the function units (FUs) and memory system can actually complete. $IPC_{state\ update}$ asks how quickly the machine can accept completed work into precise architectural state. The retire rate cannot exceed the weakest of these rates, because retirement is the final visible outcome of all the earlier pipeline activity.

6.2 Historical Motivation

The lecture introduces Tomasulo's algorithm through a constraint of the IBM System/360 Model 91: long floating-point operations had to share an architecture with only four floating-point registers.[1] More arithmetic hardware alone would leave two problems unsolved. A calculation could wait on

[1] `n5_ilp_part1.pdf`, ILP-14; R. M. Tomasulo's original paper is listed in Appendix F.

a genuinely unfinished input, or an independent calculation could be held back simply because the program reused a register name. Reservation stations let ready work wait near the execution units; tags distinguish which unfinished calculation will produce each value. That historical pressure makes the modern distinction memorable: a dependency on a value must survive, while a collision between names can be removed.

6.3 Superscalar Pipeline View

A practical superscalar model can be described with five main logical units plus a final retirement or commit action. Figure 6.1 provides a compact visual summary of how these units interact in the overall datapath. In that figure, the boxed labels denote the main structures or stages, while the smaller labels on the right identify the dominant action on the adjacent arrow bundle: fetch, dispatch, fire or issue, complete, and retire or commit. The front end is drawn as separate Branch Handling and Instruction Fetch boxes, but together they still represent the single logical instruction fetch (IF) unit described below.

1. Branch Handling and Instruction Fetch (IF) This unit performs instruction fetch (IF) while consulting branch-handling structures such as direction predictors, branch-target mechanisms, and return-address prediction. Its goal is to produce multiple candidate instructions per cycle.

2. Dispatch and Decode (DISP) This unit decodes incoming instructions in program order and dispatches them toward the scheduling structures. Dispatch is where program order is still explicit, which matters for precise recovery.

3. Scheduling Scheduling assigns ready instructions to available FUs in time. This is where execution can diverge from program order to exploit parallelism.

4. Execute (EX) FUs perform arithmetic, memory, and control operations with operation-specific latencies. Multiple FUs allow parallel progress.

5. State Update (SU) State update gathers completed results, handles squash or recovery, and prepares instructions for precise architectural completion. This unit supports squash/recovery when speculation is wrong. The final outgoing flow from this unit is the retirement or commit action, where results become architecturally visible in program order. In Fig. 6.1, retirement is represented by the final outgoing arrows below State Update rather than by a separate boxed stage.

6.4 Dependency Types and Why They Matter

Superscalar scheduling must respect three dependency classes.

Read-after-write (RAW) is a true flow dependency: a later instruction needs a value produced by an earlier one, so executing them in the wrong order would use stale data and produce the wrong result.

Write-after-read (WAR) is an anti-dependency: a later instruction must not overwrite a register before an earlier instruction has finished reading it, even though the two instructions do not share a true data value.

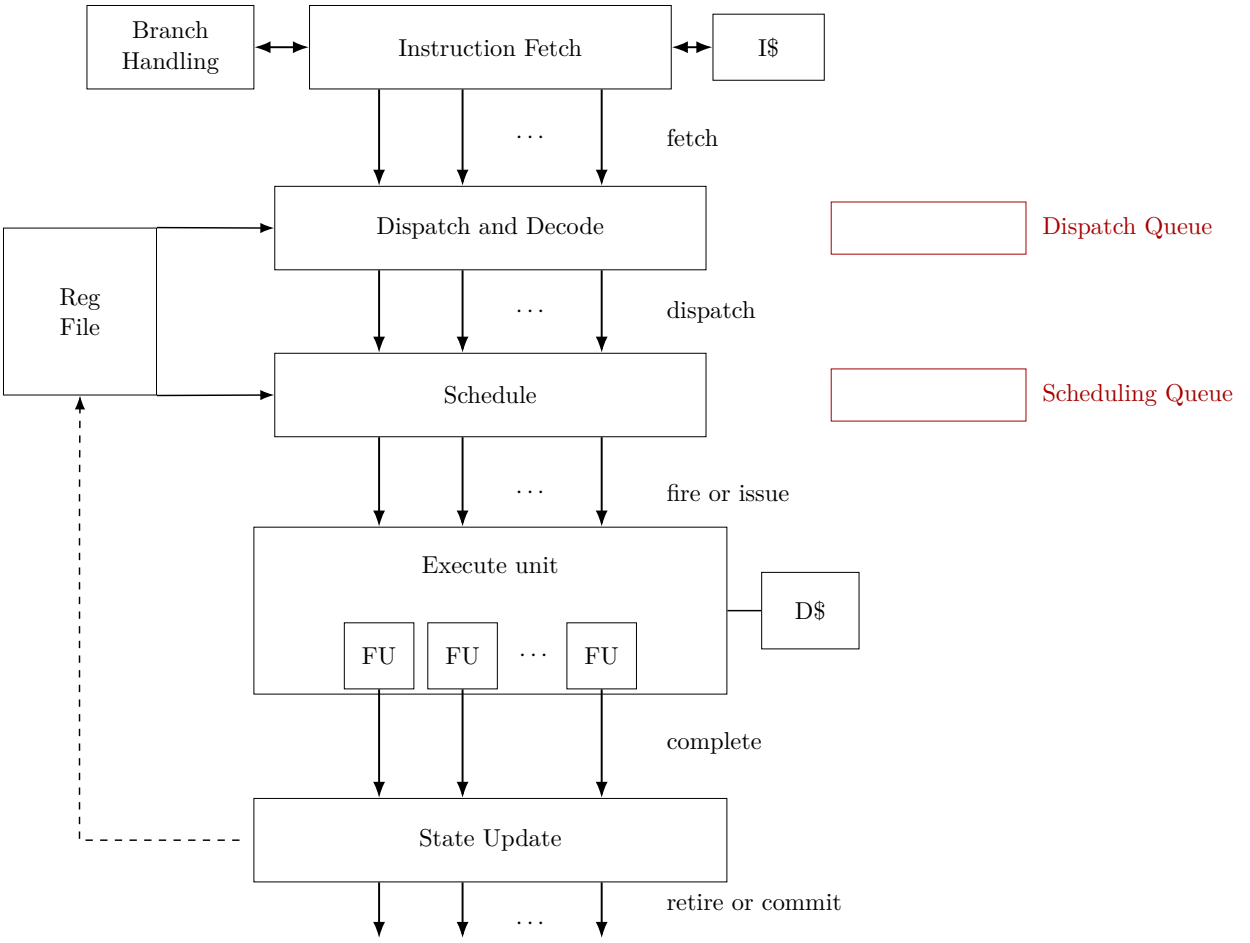


Figure 6.1: Logical superscalar pipeline view illustrating one representative organization. The figure is not specific to FICI, FICO, or FOCO; those classifications depend on the firing and completion policy. Boxed elements denote major structures or stages, while the right-side annotations denote the adjacent flow actions.

Write-after-write (WAW) is an output dependency: two instructions write the same destination, so their writes must appear in the correct order or the final architectural value will be wrong.

For arbitrary input values, a true RAW dependency must be respected even when register names change. WAR and WAW are name dependencies that arise from register reuse and can be reduced or removed by renaming the register.

6.5 Fire/Complete Taxonomy

A compact way to classify designs is by fire order and complete order.

Fire in order, complete in order (FICI) is the classic simple-pipeline behavior. Instructions begin execution in program order and also finish in program order, which simplifies control but limits how much parallelism the machine can exploit.

Fire in order, complete out of order (FICO) is a simpler superscalar step. Instructions still begin execution in order, but different latencies allow later instructions to finish first, so some parallel speedup is possible without fully out-of-order scheduling.

Fire out of order, complete out of order (FOCO) is more aggressive and offers higher potential

ILP. The scheduler may start and finish instructions whenever operands and hardware are ready, which exposes more ILP but requires more machinery to preserve correct architectural behavior.

6.6 Scoreboarding Intuition (FICO Style)

A scoreboard tracks whether source operands, destinations, and required function units are currently available. A simplified FICO-style fire condition is shown in Eq. (6.3).

$$\text{Fire if } \neg busy(src_1) \ \wedge \ \neg busy(src_2) \ \wedge \ \neg busy(dest) \ \wedge \ \neg busy(FU_{needed}). \tag{6.3}$$

When an instruction fires, destination and function-unit busy state are set. When the instruction completes, those busy states are cleared according to the implementation timing rules. The result is correct execution with moderate hardware complexity, but some parallel opportunities remain unused compared with more aggressive out-of-order firing.

6.7 Function Units and Latency

Superscalar benefit depends on the latency mix and on the number of available units. In many designs, arithmetic logic unit (ALU) operations have short latency, multiply operations take more cycles, and memory accesses vary widely depending on cache behavior. The key point is that latency and throughput are related but not identical. An operation may have a long latency and still support good throughput if the corresponding FU is pipelined and can accept a new instruction every cycle. By contrast, a long-latency operation on a scarce or non-pipelined FU can create backpressure quickly, because younger ready instructions may have no place to execute. Memory operations are especially important here. A cache hit may behave like a modest fixed-latency operation, while a cache miss can hold resources for many extra cycles and expose whether the machine can continue doing useful independent work during the delay.

Unit count matters for the same reason. If the machine has only one ALU, one multiplier, and one load/store path, the scheduler has few choices. Adding more execution resources, including additional FUs such as ALUs, multipliers, or more load/store capacity, gives the scheduler more flexibility to overlap different instruction classes and reduce structural conflicts. Designers therefore optimize both unit availability and scheduling policy, not just raw clock frequency. The practical goal is not merely to make individual operations fast, but to ensure that the mix of latencies, FU counts, and pipeline structure supports high average IPC on realistic workloads. A common simplified latency model is shown in Table 6.1. In this simplified performance model, the exact cycle count for divide is not emphasized because divide is used relatively rarely. In real designs, however, divide often has high latency and is less deeply pipelined than common ALU operations.

Type	Typical latency	Why
Floating-point (FP) multiply	3	Denormalize, multiply, renormalize.
FP add	2	Denormalize and add, then renormalize.
Load	2	Typical cache pipeline behavior.
Store	1	Write buffering, or a load/store buffer, can hide much of the visible latency.
Divide	Variable	Often implemented with unpipelined iterative methods such as Newton iteration, and many simplified models treat divide as too rare to emphasize.
ALU	1	Most common operation, so short latency is especially valuable.

Table 6.1: Typical simplified function-unit latencies used in superscalar examples.

6.8 Worked FICO Timing Example

The following seven-instruction sequence illustrates the scoreboard idea in a concrete schedule:

$$A: \texttt{ADD}\ R1, R2, R3$$
$$B: \texttt{MUL}\ R5, R1, R4$$
$$C: \texttt{ADD}\ R3, R1, R2$$
$$D: \texttt{ADD}\ R1, R5, R4$$
$$E: \texttt{MUL}\ R6, R7, R8$$
$$F: \texttt{ADD}\ R1, R3, R3$$
$$G: \texttt{ADD}\ R4, R3, R5$$

Assume a fetch rate of two instructions per cycle, one ALU with latency one cycle, one multiplier with latency three cycles, and a simplified *state update (SU)* stage used to make completion visible. Table 6.2 shows one compact timing schedule for this example. The stage labels are *instruction fetch (IF)*, *dispatch (DISP)*, *execute (EX)*, and *state update (SU)*. Repeated DISP entries indicate that the instruction is blocked in the in-order fire logic, not that it is being redispatched from scratch.

Walkthrough Cycles 0–3 establish the basic pattern. A and B are fetched together, but B cannot leave DISP immediately because it needs the value of $R1$ from A. Once A executes, B can move forward, and C can also reach EX shortly afterward because it depends on the same produced value. This is why B spends an extra cycle in DISP in Table 6.2: the machine is still firing instructions in program order, so the scoreboard must wait until A has released the needed state.

Cycles 4–6 expose the main weakness of FICO. D is genuinely blocked because it needs $R5$ from B, so its repeated DISP entries reflect a real RAW dependence. E, however, is independent of D and could use the multiplier, yet it still cannot fire early because the machine will not let younger instructions pass an older blocked instruction in the dispatch path. That is the key limitation of fire-in-order scheduling: some stalls come from true dependencies, but others come from the policy itself.

Cycles 7–9 show both overlap and continued restrictions. E eventually occupies the multiplier while F and G reach the ALU later, so the machine does achieve some parallel execution and

Instr.	Operation	0	1	2	3	4	5	6	7	8	9
A	ADD $R1, R2, R3$	IF	DISP	EX	SU						
B	MUL $R5, R1, R4$	IF	DISP	DISP	EX	EX	EX	SU			
C	ADD $R3, R1, R2$		IF	DISP	EX	SU					
D	ADD $R1, R5, R4$		IF	DISP	DISP	DISP	DISP	EX	SU		
E	MUL $R6, R7, R8$			IF	DISP	DISP	DISP	EX	EX	EX	SU
F	ADD $R1, R3, R3$			IF	DISP	DISP	DISP	DISP	EX	SU	
G	ADD $R4, R3, R5$				IF	DISP	DISP	DISP	DISP	EX	SU

Table 6.2: Timing table for a simplified FICO schedule. Repeated DISP entries indicate that an instruction remains blocked in the in-order dispatch-and-fire path while waiting for data or hardware to become available.

some out-of-order completion. At the same time, the single ALU and single multiplier keep the scheduler's choices narrow, and the in-order fire rule prevents the machine from exploiting all available independence. The example therefore illustrates why FICO is attractive as a first superscalar step—it is much simpler than full FOCO—but also why more aggressive out-of-order firing and renaming are needed to capture more ILP.

6.9 Register Renaming as the Next Step

Register renaming is the introduction of additional physical register identities so later writes do not falsely block earlier reads or writes. Conceptually, renaming transforms many WAR/WAW hazards into non-hazards while preserving program meaning. The dependency pressure that remains is primarily RAW, which reflects true data flow. In a typical implementation, the registers named by the instruction set architecture (ISA) are the *architected registers*, while the larger hidden pool used internally by the machine is the *physical register file*. The architected registers are the names the programmer sees in assembly language and that the ISA defines as part of the visible machine state. The physical register file contains additional hidden storage used internally by the processor to hold renamed versions of those values.

This distinction matters because several in-flight instructions may all refer to the same architected destination at different moments in time. Rather than repeatedly reusing one architectural name and creating false WAR/WAW conflicts, the machine can assign a different physical register to each new write while still preserving the illusion of a small fixed architected register set. Older instructions can continue reading the older physical version, while younger instructions are redirected to the newer one. A *register alias table (RAT)* maps each architected register name to the physical register that currently holds its most recent value. When dispatch renames a destination, it allocates a fresh physical register and updates the RAT so younger instructions will refer to the new location rather than the older one. Ready bits in the physical register file then indicate whether each renamed value has actually been produced yet. Some architectures describe the same underlying status with *busy bits* instead. The terminology varies, but the idea is the same: one convention marks a value as ready to use, while the other marks it as still busy or unavailable.

6.10 Tomasulo's Algorithm

Tomasulo's algorithm is a classic out-of-order execution method that implements *FOCO* by allowing instructions to fire when their operands are ready rather than strictly in program order. Its central idea is to combine renaming with *reservation stations*, which are scheduling entries that hold an instruction until the required operands and execution resources are available. Historically, this approach is associated with Bob Tomasulo's work on the IBM System/360 Model 91, and it remains the conceptual foundation behind many modern out-of-order schedulers.

Main structures A simplified Tomasulo-style machine contains a dispatch unit, a set of reservation stations, one or more FUs, a result-broadcast network, and a renamed register file. Dispatch still runs in program order, but once instructions occupy reservation stations they may wake up and fire out of order. Each reservation-station entry records the required FU type, the destination tag or physical register, and for each source either a ready value or a tag identifying which older instruction will eventually produce that value.

Dispatch, fire, and complete At dispatch time, each source operand is examined. If the needed value is already ready, the value is copied into the reservation station. If it is not ready, the reservation station records the producing tag instead. The destination is renamed so younger instructions refer to the new producer rather than to the older architectural name. Later, when both sources are ready and a matching FU is available, the instruction can fire even if an older unrelated instruction is still waiting somewhere else in the machine. Completion is a separate event from firing: when an instruction completes, it broadcasts its tag and value on the common data bus (CDB) so dependent instructions can wake up and the renamed register file can be updated.

Illustrative four-instruction trace The following short trace captures the key behavior. Instruction A is a load that misses in the cache. Instruction B is an add that depends on A. Instruction C is an independent load that hits in the cache. Instruction D is an add that depends on C. Because the machine is out of order and the cache is non-blocking, C and then D can make progress while A is still waiting on the miss. Figure 6.2 gives a compact cycle-by-cycle reference for the same sequence before the text walks through it in words. Each cell shows the dominant structure label first—dispatch (DISP), reservation station (RS), execute (EX), or common data bus (CDB)—followed by the local action.

Cycle 0. A, B, C, and D dispatch into reservation stations. Sources that are already available are copied in as values; missing sources are recorded as tags. Destinations are renamed, and the corresponding destination entries are marked not ready.

Cycle 1. A fires, and independent instruction C can also fire because it does not depend on A. This is the key gain from out-of-order scheduling: an older miss does not automatically freeze younger independent work.

Cycles 2–3. Nothing new completes. B is still waiting for A, and D is still waiting for C.

Cycle 4. C completes first and broadcasts its tag and value on the common data bus (CDB). Reservation stations compare that tag against all waiting operands, so D wakes up and becomes ready to fire.

Cycle 5. D fires and begins its add using the value from C.

Instruction / event	0	1	2	3	4	5	6
A	**DISP** Dispatch	**EX** Fire	**EX** Miss wait	**EX** Miss wait	**EX** Miss wait	**EX** Miss wait	**CDB** Complete
B	**DISP** Dispatch	**RS** Wait on A	**RS** Wait on A	**RS** Wait on A	**RS** Wait on A	**RS** Wait on A	**RS** Ready after A
C	**DISP** Dispatch	**EX** Fire	**EX** Hit wait	**EX** Hit wait	**CDB** Complete		
D	**DISP** Dispatch	**RS** Wait on C	**RS** Wait on C	**RS** Wait on C	**RS** Wake on C	**EX** Fire	**CDB** Complete after A
CDB					**CDB** C		**CDB** A then D

Figure 6.2: Compact cycle table for the four-instruction Tomasulo trace. The table emphasizes logical ordering rather than exact micro-timing details.

Cycle 6. A finally completes its miss and D also wants to complete. With only one CDB, both cannot broadcast at once, so the usual rule is to let the older instruction win. A therefore broadcasts first, and B immediately becomes ready. D then broadcasts its own result, and B can fire as soon as the machine allows post-broadcast wakeup.

This trace shows why it is important to distinguish *fire* from *complete*. Firing sends operands into an FU and begins execution. Completing publishes the result so the rest of the machine can observe it.

Non-blocking caches This Tomasulo-style example also motivates a *non-blocking cache*. Such a cache is often described as supporting *hit under miss*. If A misses in the cache, the machine does not necessarily want every younger memory access to stop. Instead, independent cache accesses may continue to probe the cache pipeline. That allows C to hit and complete while A is still waiting on a lower-level response. That improves throughput, but it also makes control more complex because the cache and scheduler must now track multiple outstanding requests correctly.

6.11 Result Buses and Physical Register Files

Basic Tomasulo has an obvious bottleneck: if all completed instructions must use one common data bus (CDB), then only one completion can be broadcast in a cycle. That limitation can be summarized as Eq. (6.4).

$$IPC_{complete} \leq N_{result\ buses}. \tag{6.4}$$

Adding more result buses raises the completion ceiling, but it also increases wakeup complexity because each reservation station must compare against more simultaneous broadcasts.

One historical way to manage this cost is to redesign Tomasulo around an explicit physical register file and a RAT rather than storing all transient values only in reservation-station entries. With physical registers, the scheduler can often replace some associative tag searches with direct lookup by physical register number. That is faster and usually lower power than asking every waiting instruction to compare against every possible producer all the time.

The tradeoff is a heavier register file. If many reservation-station entries are checked each cycle, the physical register file may need substantial read bandwidth. A common first-order estimate is shown in Eq. (6.5).

$$N_{read\ ports} \approx 2N_{sched}, \qquad N_{write\ ports} \approx N_{result\ buses}. \tag{6.5}$$

The factor of two on the read side comes from the common case of two source operands per instruction. Real designs use banking, clustering, bypass paths, operand caching, and other tricks to reduce this raw multi-port cost, but the estimate is a useful baseline for understanding why renamed out-of-order cores are expensive.

6.12 RATs, Ready Bits, and Dispatch in Renamed Machines

Once the machine uses an explicit physical register file, dispatch must translate instruction set architecture (ISA) register names into physical register numbers. That basic renaming step is summarized by Eq. (6.6).

$$P_{src1} = RAT[A_{src1}], \qquad P_{src2} = RAT[A_{src2}], \qquad RAT[A_{dest}] \leftarrow P_{free}. \tag{6.6}$$

The first two mappings identify the current physical locations of the source values. The third mapping allocates a fresh physical destination so younger instructions depend on the new producer rather than on an older architectural name.

In this style of design, the fire rule is usually expressed with *ready bits* on physical registers rather than with *busy bits* on architected registers. The same logical condition is shown in Eq. (6.7).

$$fire_i \Leftarrow ready(P_{src1}) \wedge ready(P_{src2}) \wedge free(FU_i). \tag{6.7}$$

Each scheduling-queue entry is a reservation station that records the FU type, the destination physical register or tag, and either ready source values or the physical-register identities that will eventually supply them. When a producing instruction completes, the result bus updates the physical register file and wakes any waiting entries that need that value.

The speculative physical-register state may be called the *future file* or the *messy register file*. In either case, the term refers to a speculative state that may be ahead of the last coherent architectural state. The key idea is the same: the speculative state can move ahead aggressively, while the committed state must remain precise.

6.13 Precise State and Recovery

Aggressive out-of-order execution creates a second problem beyond scheduling: the machine may complete instructions in an order that does not match the sequential state the programmer expects. If an interrupt, page fault, or other exception occurs at the wrong moment, software still needs a coherent restart point. That requirement is usually stated as the *precise exception* property in Eq. (6.8).

$$\begin{aligned} &\text{precise exception at } i \Rightarrow \text{all instructions older than } i \text{ are committed} \\ &\qquad\qquad \wedge\ \text{no instruction younger than } i \text{ is architecturally visible} \end{aligned} \tag{6.8}$$

This requirement was not always imposed in the same way. In Tomasulo's original floating-point setting on the IBM System/360 Model 91, the out-of-order machinery covered a narrower part of the processor, and the machine could tolerate *imprecise exceptions* in situations that modern general-purpose cores cannot. Once out-of-order execution extended to ordinary integer, load/store, and control-flow instructions under virtual memory, recovery to an exact sequential state became mandatory rather than optional.

6.14 Reorder Buffers and Retirement

The standard answer to precise-state recovery is the *reorder buffer (ROB)*. Dispatch allocates ROB entries in program order. Execution may finish out of order, but retirement occurs in order from the ROB head, which creates a clean boundary between speculative work and architecturally committed state.

A typical ROB entry stores destination information, a result field, a ready bit, an exception flag, and enough control information to recover the correct program counter (PC). In simple form, retirement follows Eq. (6.9).

$$retire \Leftarrow ready(ROB_{head}) \wedge \neg exception(ROB_{head}). \tag{6.9}$$

If the machine can retire at most W_{retire} instructions per cycle, then one cycle may commit up to the oldest W_{retire} consecutive ready non-excepting entries from the ROB head, but it may not skip over an older unready or excepting entry. Retire width therefore limits architecturally visible IPC even when execution hardware has already completed more work internally. If the instruction at the ROB head is marked as an exception, the machine instead flushes younger speculative work and transfers control to the retry or handler PC, as shown in Eq. (6.10).

$$\begin{aligned} exception(ROB_{head}) \Rightarrow\ & \text{flush younger instructions} \\ & \wedge\ \text{transfer control to the retry or handler PC} \end{aligned} \tag{6.10}$$

For example, a page-faulting load is usually retried after the operating system resolves the fault, whereas a trap-like event may resume at the following instruction.

ROB with bypass or forwarding Some designs let dispatch search the ROB from youngest to oldest for the most recent in-flight producer of a source register. If the matching entry is ready, dispatch can copy the value directly. If it exists but is not ready, dispatch copies the tag or ROB identity and waits. If no matching entry exists, the value must already have retired, so dispatch reads it from the architected register file. This is often described as a ROB with bypass or forwarding. It is effective, but the search itself is associative and can become part of the critical timing path.

Worked tag-based ROB-bypass trace The following tag-based example makes the lookup rule explicit. Assume the architected register file initially holds committed ready values for $R1$, $R2$, and $R3$, and that dispatch allocates ROB tags in program order. Consider:

A: LW $R1, 0(R3)$
B: ADD $R3, R2, R1$
C: ADD $R1, R3, R2$

At dispatch, the source resolution is:

Instr.	Dest reg/tag	Src #1	Src #2	Queue status
A	$R1$/A	$R3$ value ready	n/a	ready to fire
B	$R3$/B	$R2$ value ready	tag A not ready	waiting on A
C	$R1$/C	tag B not ready	$R2$ value ready	waiting on B

ROB entry	Dest reg	Ready	Value source
A	$R1$	0	pending load result
B	$R3$	0	pending add result
C	$R1$	0	pending add result

Table 6.3: Tag-based Tomasulo plus ROB bypass after dispatch of A, B, and C.

Instr.	Dest reg/tag	Src #1 at dispatch	Src #2 at dispatch
A	$R1$/A	$R3$ value from architected RF	n/a
B	$R3$/B	$R2$ value from architected RF	waits on tag A from ROB entry A
C	$R1$/C	waits on tag B from ROB entry B	$R2$ value from architected RF

The key rule is that dispatch always takes the newest in-flight producer. So B does not read the old committed $R1$ value; it waits on tag A. Likewise, C does not read the old committed $R3$ value; it waits on tag B. After all three instructions have dispatched, the in-flight structures look like Table 6.3.

From this point onward, execution follows the dependence chain directly. A completes first and marks ROB entry A ready, which wakes B. B then completes and marks ROB entry B ready, which wakes C. C completes last. Retirement still occurs strictly in order A, then B, then C, so after the last retirement both the scheduling queue and the ROB are empty. After retirement, the architected register file holds the retired results, with $R3$ containing the value produced by B and $R1$ containing the younger value produced by C. For any such trace, once the total elapsed cycles are known, fragment IPC is retired instructions divided by elapsed cycles over the chosen measurement interval.

Freeing physical registers Renamed machines also need a rule for reclaiming old physical registers. The exact policy differs across implementations, but the underlying principle is simple: an old physical register cannot return to the free list until no older in-flight instruction can still need it. Common implementations describe this with use counters, pending-free bits, or retirement-based reclamation. For example, if an architected register was formerly mapped to physical register $P23$ and rename later redirects that architected name to $P27$, the old $P23$ mapping cannot be recycled immediately. It becomes reclaimable only after no older instruction can still read it and the newer mapping has safely become the committed architectural version. The details vary, but all correct schemes delay reuse until the older mapping is no longer architecturally or speculatively required.

When architected registers are actually written Completion and retirement are not the same event. Completion usually writes the speculative destination into the physical register file and marks the matching ROB entry ready. The architected register file is updated only when that ROB entry reaches the head and retires in order. This separation is what allows the machine to execute aggressively while still presenting precise sequential state to software.

Worked PRegs/RAT/ROB end-state trace The following short trace makes the bookkeeping easier to follow. Assume the initial mappings are $R0 \to P0$, $R1 \to P1$, $R2 \to P2$, and $R3 \to P3$, with $P4$ through $P7$ free. Assume two-wide fetch and dispatch, two-wide retirement, two 1-cycle ALUs, one 1-cycle LSU, and a back-to-front cycle evaluation so that Execute can mark a destination physical register ready before Schedule runs later in the same cycle. Consider the following sequence:

A: LW $R3, 0(R1)$
B: ADD $R1, R3, R2$
C: ADD $R2, R2, R1$

One legal cycle-by-cycle interpretation is:

Cycle 1. Fetch A and B.

Cycle 2. Dispatch A and B. A allocates $P4$ as the new destination for $R3$ and records old mapping $P3$ in its ROB entry. B therefore reads $P4$ as its first source, allocates $P5$ as the new destination for $R1$, and records old mapping $P1$ in its ROB entry. Fetch C.

Cycle 3. A fires on the LSU. Dispatch C, which now reads $P2$ and $P5$, allocates $P6$ as the new destination for $R2$, and records old mapping $P2$ in its ROB entry.

Cycle 4. A completes, marking $P4$ ready and its ROB entry ready. Because that completion is visible before scheduling in the same cycle, B can now fire.

Cycle 5. A retires, so the architected copy of $R3$ now comes from $P4$ and old mapping $P3$ can be freed. B completes, marking $P5$ ready, and C can then fire.

Cycle 6. B retires, so the architected copy of $R1$ now comes from $P5$ and old mapping $P1$ can be freed. C completes, marking $P6$ ready.

Cycle 7. C retires, so the architected copy of $R2$ now comes from $P6$ and old mapping $P2$ can be freed.

The exact cycle numbers can shift by one under a different but internally consistent simulator ordering, but the final renamed state does not change.

Instr.	FU	Dest physical reg.	Src #1	Src #2	Final status
A	LSU	$P4$	$P1$	n/a	completed and retired
B	ALU	$P5$	$P4$	$P2$	completed and retired
C	ALU	$P6$	$P2$	$P5$	completed and retired

ROB entry	Dest architected reg.	Prev physical reg.	Final status
A	$R3$	$P3$	retired
B	$R1$	$P1$	retired
C	$R2$	$P2$	retired

Architected reg.	Final physical reg.
$R0$	$P0$
$R1$	$P5$
$R2$	$P6$
$R3$	$P4$

Physical reg.	Ready	Free
$P0$	1	0
$P1$	1	1
$P2$	1	1
$P3$	1	1
$P4$	1	0
$P5$	1	0
$P6$	1	0
$P7$	0	1

Three rules determine the outcome. Dispatch allocates the next free physical destination and records the previous architectural mapping in the ROB. Completion makes the speculative destination ready for younger consumers. Retirement is the event that both updates the architected view and frees the previous physical mapping.

6.15 Future Files and Checkpoint/Repair

One useful way to describe renamed out-of-order state is to separate it into two layers. The *future file* holds the most recent speculative values that execution is building toward. The *architected register file* holds only committed state. Execution and wakeup consult the speculative state, while retirement copies correct in-order results into the architected state. This explains how the machine can behave aggressively internally while still presenting a sequential interface to software. In a physical-register implementation, this often means retirement copies the committed value from the current physical register into the architected register file, while exception recovery can restore a precise view by redirecting the register alias table (RAT) back to the committed architectural mappings.

Comparison with tag-only Tomasulo A physical-register plus RAT design does not automatically increase the amount of ILP that the scheduler can discover. If a tag-based Tomasulo machine and a physical-register machine have the same function units, the same wakeup/select policy, the same retire width, and enough result buses that completion bandwidth is not the bottleneck, then they can expose the same execution schedule and the same IPC on a given trace. The main differences are in bookkeeping cost, lookup style, recovery organization, and physical-register reuse rather than in the dependence constraints themselves. Performance diverges only when one design changes a practical limit such as result-bus count, port bandwidth, wakeup timing, or retirement bandwidth.

The ROB is not the only recovery method. *Checkpoint/repair* schemes periodically capture a coherent snapshot of register state and use that snapshot as the recovery point. One common description uses three conceptual views of the state: a messy current copy, a working copy being built toward coherence, and a golden copy representing the last known coherent sequential state.

The key control concept is the *instruction boundary*. Dispatch chooses a dynamic boundary, often denoted by markers such as IB1 and IB2. Instructions that complete before the current boundary update both the messy state and the working copy. Instructions that complete after the boundary update only the messy state. Once every instruction older than the boundary has completed, the working copy has become coherent and may be promoted to the new golden copy. The machine then chooses a new boundary and repeats the process.

If an exception occurs, the machine restores the golden copy and continues from that last coherent point. It may then re-execute younger instructions sequentially or under tighter control until it reaches the faulting instruction and handles the event precisely. This can reduce some of the timing pressure associated with aggressive ROB-bypass logic, though it introduces its own bookkeeping and copying costs.

6.16 Speculation Without and With Prediction

Superscalar pipelines use speculation to keep FUs busy. Some designs can speculate without high-quality prediction (for example, eager execution ideas), but the useful work fraction is then limited. High-accuracy branch handling usually gives better effective IPC because fewer speculative instructions are squashed.

6.17 Security Implications of Speculative Execution

Precise exceptions solve the architectural recovery problem, but they do not automatically solve the microarchitectural side-effect problem. An instruction that is later squashed may still have changed caches, predictors, or other timing-visible structures before the machine discovered that the instruction should not retire. That is the core issue behind speculative side channels.

The best-known examples are Spectre- and Meltdown-style attacks. At a high level, the attacker causes the processor to execute work that will later be squashed or faulted. Even though the architectural state rolls back correctly, the speculative work may still have fetched a cache line or trained a predictor. The attacker then measures timing differences to infer information that should never have become visible through the ISA.

Shared-core execution can make this worse. With *hyperthreading*, or more generally simultaneous multithreading, two threads share parts of one physical core. That increases the opportunity for one thread to observe side effects caused by another thread's speculative activity. The security lesson is broader than any single named vulnerability: microarchitectural cleanup matters, not just architectural rollback.

6.18 Implementation Pressure

Once out-of-order execution moved from isolated examples into full commercial processors, these mechanisms had to satisfy real timing, area, power, and verification constraints. That practical pressure is one reason the field explored multiple recovery styles rather than converging immediately on a single universal design.

The architectural lesson is that elegant scheduling ideas are not sufficient by themselves. ROB bypass logic, associative searches, speculative-state tracking, and recovery control all stress timing closure and verification. As out-of-order windows grow wider and deeper, state management and recovery logic become first-order design problems rather than secondary implementation details.

6.19 Interpreting Out-of-Order Processor Statistics

Trace-driven superscalar simulators often report more than cycles and IPC. Those counters are useful only when their meanings and denominators are clear.

Fetched versus retired instructions The architecturally meaningful IPC uses retired instructions, not fetched instructions. *Trace instructions* count the dynamic instructions in the workload itself. *Fetched instructions* count every instruction the front end brought in, including wrong-path work fetched before a branch misprediction was repaired. *Retired instructions* count only the instructions that reached precise architectural state. In a correct full run, retired instructions usually match the trace instruction count, while fetched instructions may be much larger when prediction accuracy is poor.

Miss and misprediction counters Branch-misprediction counts measure control-flow repair events. Instruction-cache misses reduce front-end supply, while data-cache misses usually delay load or store completion and can starve dependent instructions. These counters therefore explain why IPC may fall even when the machine nominally has ample fetch width and many function units.

Queue-pressure counters Maximum and average dispatch-queue, scheduling-queue, and ROB occupancy show where pressure accumulates. If ROB usage remains near its limit and dispatch is frequently blocked because the ROB is full, then commit bandwidth, long-latency instructions, or recovery delays are throttling forward progress. If scheduling queues remain lightly used but *no-fire cycles* are high, then the machine is more likely waiting on ready operands or memory completion than on queue capacity.

Isolating control and memory effects Many simulators also offer a no-miss or no-misprediction mode. Such a configuration does not model real execution, but it is analytically useful because it isolates pure width, scheduling, and dependence limits from penalties caused by cache misses and poor control-flow prediction.

6.20 Chapter 6 Summary

Chapter 6 develops dynamic instruction-level parallelism as a concrete microarchitectural path to $CPI < 1$. Superscalar design achieves this by widening the pipeline's effective throughput and managing dependencies, hazards, and speculative state carefully. FICO-style scoreboarding illustrates the basic mechanism, while renaming, Tomasulo-style wakeup and result broadcast, and stronger out-of-order policies push performance further. Once the machine executes and completes work out of order, however, scheduling is no longer enough by itself. The core must also maintain precise architectural state through mechanisms such as physical-register renaming, reorder buffers, future files, or checkpoint/repair schemes. Modern processors therefore combine ILP techniques with careful recovery, retirement, and microarchitectural side-effect control. Operationally, this chapter is best checked through throughput-bound calculations, dependence-height reasoning on DDGs, cycle tables for FICO timing, reservation-station and RAT snapshots, and careful separation of speculative completion from architectural retirement. Simulator-visible counters such as fetched-versus-retired instructions, misprediction counts, miss counts, queue occupancy, and ROB pressure then provide a practical way to diagnose which limits are dominating a given design point. These ideas together form the bridge from classic five-stage thinking to modern high-throughput scalar cores.

6.21 Practice Questions

The following questions reinforce the main ideas of Chapter 6 and are intended for self-checking after the summary.

1. A FOCO processor fetches at most 6 instructions per cycle, dispatches at most 4 per cycle, has three 1-cycle function units that can each accept one instruction per cycle, and retires at most 5 per cycle. Assuming the window always contains enough independent ready work of the right types, what is the maximum IPC? Which resource creates the bound?
2. A dependence graph has nodes A through G and edges $A \to C$, $B \to C$, $B \to D$, $C \to E$, $D \to F$, $E \to G$, and $F \to G$. If the dependence height of a node is the length of the longest path from that node to an exit, compute the height of every node.
3. In a FICO-style machine with one LSU, one ADD unit, two-cycle loads, and one-cycle adds, trace the sequence `A: LW R1,0(R2)`, `B: ADD R3,R1,R4`, `C: ADD R5,R6,R7`, `D: ADD R8,R3,R9`. Assume dispatch is two-wide in program order starting in cycle 1; a dispatched instruction can fire no earlier than the next cycle. Fire is at most two-wide in program order, the ADD unit accepts one instruction per cycle, and results become usable in the cycle after completion. A two-cycle load firing in cycle 2 completes at the end of cycle 3. Ignore fetch, queue-capacity limits, and completion-bandwidth limits. Fill in a cycle table for dispatch, fire, and complete. Then state whether adding a second ADD unit removes all stalls, and explain which stalls remain due to the in-order fire rule.
4. Consider Tomasulo with tags and a ROB with bypass for the sequence `LW R1,0(R3)`; `ADD R3,R2,R1`; `ADD R1,R3,R2`. When the second and third instructions dispatch, which of their source operands come from the register file and which come from the youngest older ROB producer?
5. Using the same three-instruction sequence, write the contents of the scheduling queue or reservation stations immediately after dispatch of all three instructions. For each entry, identify the destination tag or physical register, which source operands are already ready, and which operands are waiting on older tags.
6. Suppose the machine from the previous question has two ADD units, one LSU, and enough result buses that completion bandwidth is not the bottleneck. What is the earliest completion order of the three instructions, and in what order do their results become architecturally visible?
7. In a renamed machine, the current mapping of architected register `R1` points to physical register `P10`. A new instruction `ADD R1,R2,R3` dispatches and receives destination physical register `P24`. What mapping change occurs immediately at dispatch, and why can `P10` not be freed at that same moment?
8. Distinguish the event of broadcasting a result on the common data bus from the event of updating precise architected state at retirement. Why is this distinction essential once execution and completion occur out of order?
9. For each statement below, answer true or false and justify briefly: (a) with enough physical registers, WAR and WAW dependences can be removed, but RAW dependences remain; (b) if a value is present in the future file or physical register file, then precise architected state has already been updated; (c) using fetched instructions instead of retired instructions can

overstate IPC on a run with many squashes.

10. A simulator reports 1,600 fetched instructions, 1,200 retired instructions, 1,000 cycles, 220 no-fire cycles, and 90 ROB-blocked dispatch cycles. Compute IPC using the architecturally meaningful instruction count. Then explain what the gap between fetched and retired instructions suggests and what the ROB-blocked count suggests.

Chapter 7

Compiler-Scheduled ILP: VLIW and Software Pipelining

The previous chapter asked the processor to find ready work while the program ran. For a regular loop, much of that work can be planned earlier. The compiler already knows which operations use each result and which functional units the target machine provides. It can arrange a schedule in advance, including an overlap in which one iteration loads data while another computes and a third stores its result. This chapter follows that alternative, along with the uncertainty that a static plan cannot remove.

7.1 VLIW as a Compiler-Scheduled Alternative

One response to dynamic-scheduling complexity is to shift more of the scheduling work into the compiler. As superscalar windows grow, the hardware would like a larger scheduling queue so it can search farther ahead for independent work, but larger queues also make wakeup, select, and operand-readiness checks harder to close at a short cycle time. This tradeoff motivates the *very long instruction word (VLIW)* approach, historically associated with Josh Fisher. In a VLIW machine, one instruction word encodes several operations intended to execute in parallel, so the compiler determines the schedule at compile time rather than leaving that work to large runtime scheduling hardware.

In this context, an individual encoded action is an *op*. A set of ops intended to execute in the same cycle is a *multi-op*. VLIW reduces runtime hardware complexity because the machine no longer needs a large associative scheduling queue, aggressive wakeup logic, or much of the dynamic issue machinery used in FOCO-style processors. The tradeoff is that the compiler must do much more work up front, and the resulting code is less adaptable to runtime variation such as cache misses or changing branch behavior. This tradeoff is attractive in domains such as *digital signal processors (DSPs)*, where workloads are regular and low-power execution is important.

Compiler design must balance three competing goals: code quality, compilation time, and maintainability. In practice, that usually rules out algorithms much worse than quadratic for large compiler passes, and even cubic methods are used cautiously. Optimal scheduling with general dependence and resource constraints is computationally difficult; exhaustive search grows rapidly with block size. Practical VLIW compilers therefore rely on heuristics rather than exact search for every block. List scheduling is one of the standard heuristics for this job. The broader performance point is that widening fetch and scheduling machinery yields diminishing returns: useful IPC usually grows sublinearly with front-end width, while the cost of dynamic scheduling hardware grows quickly.

Instr.	Operation	True dependences	Latency
A	LW $R2, X$	None	2
B	LW $R3, Y$	None	2
C	LW $R4, Z$	None	2
D	ADD $R5, R1, R2$	A	1
E	MUL $R6, R2, R3$	A, B	3
F	MUL $R7, R1, R3$	B	3
G	ADD $R8, R7, \#1$	F	1
H	SW $R8, X$	G	1
I	ADD $R9, R2, R4$	A, C	1

Table 7.1: Illustrative basic block for VLIW list scheduling.

7.2 Dependence Graphs and List Scheduling

The compiler begins with a *data dependence graph*. True flow edges capture read-after-write dependencies, while anti-dependence and output-dependence edges capture false constraints caused by register reuse. Pure flow dependence always takes priority over false dependence because it represents real data movement. If the code contains loops, this graph can contain cycles, so it is not necessarily a directed acyclic graph. Constructing the graph by hand is mechanical. Draw one node per op, then add all true flow edges from each definition to the later uses that require that value. Next add anti-dependence edges from a read to a later write of the same architectural name, and output-dependence edges between two writes of the same name. If two operations are already ordered by a true flow dependence, that true edge is the one that matters for scheduling because it already enforces the required order.

Table 7.1 gives a compact block that will be used for the scheduling example. Assume one pipelined load/store unit, one pipelined arithmetic logic unit (ALU), and one pipelined multiplier. Each unit can accept a new op every cycle; the listed latencies describe when results become available, not how long the unit remains occupied. Also assume the compiler has already assigned registers so that the remaining constraints shown in the table are dominated by true dependences rather than by avoidable name conflicts. Figure 7.1 shows the same block in a more visual form: the original operations, the resulting data dependence graph, and one priority list ordered by dependence height.

One common priority heuristic is *dependence height*, which estimates how much latency remains below a node on the longest dependent path. If the heuristic includes the latency of the current instruction itself, one convenient definition is Eq. (7.1).

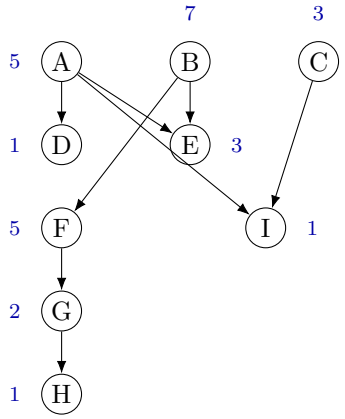
$$height(i) = \begin{cases} lat(i), & succ(i) = \emptyset \\ lat(i) + \max\limits_{j \in succ(i)} height(j), & \text{otherwise} \end{cases} \tag{7.1}$$

Higher-height instructions receive higher priority because delaying them is more likely to lengthen the critical path. To annotate a graph by hand, start from the sinks and assign each sink its own latency. Then sweep backward, adding the current node latency to the maximum height among its successors. The ready list is obtained by sorting nodes from highest height to lowest and breaking

Original operations

A: LW $R2, X$
B: LW $R3, Y$
C: LW $R4, Z$
D: ADD $R5, R1, R2$
E: MUL $R6, R2, R3$
F: MUL $R7, R1, R3$
G: ADD $R8, R7, \#1$
H: SW $R8, X$
I: ADD $R9, R2, R4$

Data Dependence Graph



List ordered by dependence height

B (7)
A (5)
F (5)
C (3)
E (3)
G (2)
D (1)
I (1)
H (1)

Figure 7.1: Original operations, dependence graph, and one priority ordering for the VLIW list-scheduling example. The small numeric annotations next to nodes indicate dependence height. Ties may be broken arbitrarily.

ties either arbitrarily or by an explicit secondary rule such as program order. For the block in Table 7.1, the longest true-dependence chain is $B \to F \to G \to H$, so the heuristic tends to prioritize F ahead of E once both become candidates for the single multiplier.

List scheduling then places the highest-priority ready instructions into each cycle, subject to both dependence timing and resource availability. A simple legality condition is shown in Eq. (7.2).

$$cycle(i) = \min\{c \mid \forall j \in pred(i),\ c \geq cycle(j) + lat(j),\ \text{and a compatible FU is free in } c\}\,. \quad (7.2)$$

This is not an optimal algorithm, but it is practical and captures the central VLIW idea: schedule as much parallel work as possible before runtime while still respecting dependencies and resource limits.

Table 7.2 shows one possible compiler-produced schedule for the block running on a machine with one arithmetic logic unit (ALU), one multiplier, and one load/store unit. The point is not that this schedule is unique, but that the compiler can choose the cycle packing in advance and encode it directly. For illustration, this schedule keeps the three initial loads in source order even though a greedier dependence-height policy could start B before A and shorten the block by one cycle. That choice leaves one explicit empty cycle later, which makes the pause-field encoding example concrete. The schedule highlights two points. First, the compiler only packs operations together when both the dependence constraints and the functional-unit constraints allow it. Second, the absence of an operation in a slot does not require the hardware to infer what to do dynamically; the compiler has already encoded the intended schedule. In particular, cycle 6 is intentionally empty because G cannot start until the three-cycle multiply F has finished producing $R7$. Table 7.3 shows the same schedule in a field-level packing format closer to an actual multi-op layout. This table defines which operands occupy each unit's fields in each cycle; it does not, by itself, define the serialized op order in memory.

7.3 Multi-Op Encoding, Tail Bits, and Pause Fields

VLIW raises an encoding question: how should a machine represent the boundaries of a multi-op when not every cycle uses every functional unit? One simple approach is to fill every unused slot

with an explicit no-op, but that wastes code space. An alternative is to encode only real ops and add a small amount of grouping information. Each encoded op still carries its ordinary fields such as opcode, destination register, and source registers; the additional VLIW-specific fields only describe grouping and delay.

One useful scheme adds a *tail bit* to each encoded op. If the tail bit is 0, the current op is not the end of the current multi-op. If the tail bit is 1, the current op ends the current multi-op. The following op then begins a new multi-op. This lets the instruction stream represent a variable number of ops per cycle without requiring every cycle to contain the same number of explicit no-op fillers.

A related field is a *pause field*, which tells the front-end control logic to wait a specified number of cycles before beginning the next multi-op. In the simple scheme used here, the pause value is interpreted only on the op whose tail bit ends the current multi-op; pause values on earlier ops in the same multi-op are ignored. This is a compact way to represent silence between multi-ops while keeping each op's encoding regular. Table 7.4 illustrates the idea using the schedule from Table 7.2. It shows one legal serialized op order for the same multi-ops that appear in Table 7.3. Some implementations instead use simple interlocking at the multi-op level, which can remove the need for an explicit pause field. That saves encoding space, but it also gives up some of the compiler's direct control over exactly when the following multi-op may begin.

7.4 Software Pipelining and Cyclic Scheduling

Loops create another opportunity for compiler-scheduled parallelism. Instead of scheduling only one copy of a loop body, the compiler can overlap operations from different iterations so that a later iteration begins before an earlier one has fully finished. The older name for this idea is *polycyclic scheduling*; the more common modern name is *software pipelining*.

Conceptually, the compiler behaves as if it had unrolled the loop many times, compacted the overlapping schedule, and then retained only the repeating pattern. That repeating pattern is the *kernel*. The code that fills the pipeline before the kernel reaches steady state is the *prologue*. The code that drains the remaining work after loop exit is the *epilogue*. The distance in cycles between the start of successive iterations is the *initiation interval (II)*. Once the schedule reaches steady state, the loop begins a new iteration every II cycles, so the throughput is $1/II$ iterations per cycle.

Cycle	Load/Store Unit	ALU	Multiplier
1	A		
2	B		
3	C	D	
4			F
5		I	E
6			
7		G	
8	H		

Table 7.2: One legal VLIW-style schedule for the block under fixed functional-unit limits.

Cycle	IALU (1 cycle)			MUL (3 cycles)			Load (2 cycles)		Store (1 cycle)	
	d	s_1	s_2	d	s_1	s_2	d	addr	addr	s
1							$R2$	X		
2							$R3$	Y		
3	$R5$	$R1$	$R2$				$R4$	Z		
4				$R7$	$R1$	$R3$				
5	$R9$	$R2$	$R4$	$R6$	$R2$	$R3$				
6										
7	$R8$	$R7$	#1							
8									X	$R8$

Table 7.3: Field-level packing of the schedule from Table 7.2. Rows 3 and 5 are multi-ops because they contain more than one op in the same cycle.

Encoded op	Tail	Pause	Meaning
A	1	0	Single-op multi-op for cycle 1.
B	1	0	Single-op multi-op for cycle 2.
C	0	0	First op of the cycle-3 multi-op.
D	1	0	Ends the cycle-3 multi-op.
F	1	0	Single-op multi-op for cycle 4.
I	0	0	First op of the cycle-5 multi-op.
E	1	1	Ends the cycle-5 multi-op and inserts one pause cycle before the next multi-op.
G	1	0	Single-op multi-op for cycle 7.
H	1	0	Single-op multi-op for cycle 8.

Table 7.4: Illustrative VLIW encoding using tail bits and a pause field rather than explicit no-op padding.

Unrolling view before compaction One useful way to understand software pipelining is to look first at a small explicit unrolling. Suppose a loop body consists of five ops A, B, C, D, E, where E is the loop-closing branch. A two-times unrolled form can be sketched conceptually as

$$A,\ B,\ C,\ D,\ E^*,\ A',\ B',\ C',\ D',\ E',$$

where E^* is the inverted early-exit branch that leaves the loop if the second copy should not begin. This makes the scheduling opportunity visible: once the loop is unrolled, the compiler can interleave A' with still-pending work from the first copy instead of waiting for the whole first iteration to drain first. Software pipelining then takes the next step and *compacts* that overlapping pattern into a finite prologue, a repeating kernel, and a finite epilogue rather than materializing a large explicitly unrolled body.

Which dependences matter most in the first sketch In early cyclic-scheduling sketches, it is common to draw only true dependences and the loop-carried recurrences that constrain them. Anti-dependence and output-dependence edges are still real constraints in the full compiler problem, but they are often omitted from the first visualization because software renaming can frequently remove

them. The scheduling bottlenecks that determine $RecII$ come from the true cyclic constraints, so those are the ones worth emphasizing first.

7.5 Iterative Modulo Scheduling

A practical way to construct these schedules is *Iterative Modulo Scheduling (IMS)*. The compiler first estimates a lower bound on II, then attempts to build a modulo schedule at that value. If the attempt fails, it increments II and tries again.

The first bound comes from resource pressure. If a loop contains N_t operations of type t and the machine provides U_t compatible units of that type, then

$$ResII = \max_t \left\lceil \frac{N_t}{U_t} \right\rceil \tag{7.3}$$

This is the resource-constrained initiation interval.

The second bound comes from loop-carried recurrences. If a dependence cycle c has total latency $Lat(c)$ and spans $Dist(c)$ loop iterations, then

$$RecII = \max_c \left\lceil \frac{Lat(c)}{Dist(c)} \right\rceil \tag{7.4}$$

This is the recurrence-constrained initiation interval. If the dependence graph has no loop-carried cycles, then the practical lower bound is $RecII = 1$.

The schedule must satisfy both limits, so the compiler begins from

$$MII = \max(ResII, RecII), \qquad II \geq MII. \tag{7.5}$$

Many texts call this lower bound the *minimum initiation interval (MII)*. IMS follows a simple pattern:

1. Start with $II = MII$.
2. Attempt a legal modulo schedule at that value.
3. If the attempt fails, increment II and retry.

A modulo reservation table for a chosen II has exactly II rows. An operation that would occupy cycle x in a longer unrolled schedule is placed into row $x \bmod II$, provided the dependence and resource constraints remain legal.

Review Emphasis: Why IMS Starts at a Lower Bound The naive version of "try every initiation interval" can become too expensive because each scheduling attempt is itself a scheduling problem. List scheduling is often treated as roughly $O(n^2)$ in the number of operations, so repeatedly trying many impossible values of II can push the compile-time cost much higher. The point of $ResII$ and $RecII$ is to skip initiation intervals that cannot possibly work. In practice, a conservative lower bound usually lets IMS succeed after only a small number of attempts, while still guaranteeing eventual success once II becomes large enough.

In most textbook treatments, the underlying machine is modeled with a point-latency abstraction: an op occupies one issue slot, and its result becomes ready after a fixed latency. This keeps the scheduling problem manageable. More detailed models that explicitly schedule register-file ports, result buses, and intermediate pipeline stages are possible, but they are much more expensive to optimize and are usually avoided in introductory treatments.

Instr.	Operation	Key dependences	Latency
A	ADD $R2, R2, \#1$	loop-carried on $R2$	1
B	LW $R3, 0(R2)$	A	2
C	SW $R0, 0(R3)$	B	1
D	ADD $R10, R10, \#1$	loop-carried on $R10$	1
E	BRGTZ $R10$, LOOP	D; branch must close the loop body	1

Table 7.5: Loop used to illustrate software pipelining and modulo scheduling.

Modulo slot	ALU 0	ALU 1	MEM 0	MEM 1
0	A_i	D_i		
1	E_i		B_i	C_{i-1}

Table 7.6: One modulo reservation table for the loop in Table 7.5 with $II = 2$.

7.6 Worked Software-Pipelining Example

The following loop illustrates the idea. Assume two arithmetic logic units, two memory units, one-cycle add/branch latency, and two-cycle load latency. The branch still logically closes the loop body even though later iterations will overlap older ones.

The cyclic dependence view of this loop contains the within-iteration edges $A_i \to B_i$, $B_i \to C_i$, and $D_i \to E_i$, plus the loop-carried edges $A_i \to A_{i+1}$ and $D_i \to D_{i+1}$. Those loop-carried edges are exactly the recurrences used when computing $RecII$.

For this loop, the resource bound is

$$ResII = \max\left(\left\lceil \frac{3 \text{ ALU/branch ops}}{2 \text{ ALUs}} \right\rceil, \left\lceil \frac{2 \text{ memory ops}}{2 \text{ memory units}} \right\rceil\right) = \max(2, 1) = 2.$$

The loop-carried recurrences on $R2$ and $R10$ each have latency 1 and distance 1, so $RecII = 1$. Therefore the first useful attempt is $II = 2$.

One legal modulo reservation table appears in Table 7.6. The first row begins iteration i. The second row finishes the current iteration's control work while also performing the store from iteration $i - 1$. That overlap is the essence of software pipelining.

The emitted code must still handle loop entry and loop exit correctly. Table 7.7 separates the schedule into prologue, kernel, and epilogue form. The early-exit branch E^* is the inverted form of the loop branch; it lets the machine leave the prologue cleanly if the loop should terminate before the steady-state kernel is entered.

This example also shows why software pipelining is attractive for VLIW machines. The compiler exposes the repeating parallel structure explicitly, so the hardware can execute the steady-state kernel with very little dynamic scheduling logic. At the same time, general-purpose superscalar machines still benefit from dynamic scheduling because real loads do not always return in a fixed number of cycles and because binaries may need to run across several microarchitectural variants.

Phase	Operations
Prologue	A_1, D_1 then B_1, E_1^*
Kernel	repeat every $II = 2$ cycles: A_i, D_i then B_i, C_{i-1}, E_i
Epilogue	C_{last}

Table 7.7: Prologue, kernel, and epilogue for the software-pipelined loop.

Review Emphasis: Prologue, Kernel, and Epilogue The prologue is the startup code needed before the repeating pattern is fully populated. The kernel is the compact repeated pattern that behaves like many overlapped loop iterations. The epilogue drains work that was started by the final kernel iterations but has not yet completed. If nothing wraps across the compact kernel boundary, the prologue or epilogue may become trivial, but the conceptual split is still useful because it explains how a rerolled schedule preserves the behavior of the unrolled loop.

7.7 Memory Disambiguation

Loads and stores create a special scheduling problem. If a compiler moves a load ahead of an older store to the same address, the load may read the wrong value. The compiler therefore needs *memory disambiguation*: it must prove that two memory references do not overlap, or else preserve the original order or insert explicit synchronization.

This is one of the limits of VLIW compared with dynamic scheduling hardware. An out-of-order processor may speculate and recover if it guesses wrong. A VLIW compiler must usually be more conservative unless it has strong alias information. That is one reason VLIW works best when memory behavior is regular enough for compile-time reasoning to be reliable.

7.8 Chapter 7 Summary

Chapter 7 presents compiler-scheduled ILP as an alternative to increasingly aggressive dynamic superscalar scheduling. Instead of building increasingly complex runtime issue logic, a VLIW machine relies on the compiler to expose parallel ops ahead of time and encode them explicitly. Data dependence graphs, dependence-height ordering, and list scheduling provide the basic machinery for building legal multi-ops. Tail-bit and pause-field encoding then translate that schedule into a compact instruction stream without filling unused slots with explicit no-ops. Software pipelining extends the same idea across loop iterations by constructing a prologue, steady-state kernel, and epilogue around a chosen initiation interval. Memory disambiguation remains the key correctness limit: compile-time scheduling can move loads and stores only when aliasing is understood well enough to preserve program behavior. Together, these techniques show how compiler scheduling can approach the performance of aggressive ILP hardware while trading hardware complexity for compile-time analysis.

7.9 Practice Questions

The following questions reinforce the main ideas of Chapter 7 and are intended for self-checking after the summary.

1. For the sequence `LW R1,X`; `ADD R2,R1,R3`; `ADD R1,R4,R5`; `MUL R6,R2,R1`, draw the data dependence graph and label the pure, anti-, and output dependences.
2. Using appropriate latencies for load, add, and multiply, compute the dependence height of each instruction in the previous question. Then list the instructions in descending order of dependence height.
3. Assume a VLIW machine with at most one ALU op, one MUL op, and one memory op per multi-op. Produce one legal list schedule for the instruction sequence in Question 1.
4. A VLIW schedule contains the following multi-ops: row 1 has a single load; row 2 has an add and a multiply; row 3 is empty because of a dependence delay; row 4 has a store. Explain how a tail-bit plus pause-field encoding can represent this schedule without inserting explicit no-op instructions into every unused slot.
5. A loop contains 3 ALU/branch operations, 1 multiply, and 3 memory operations per iteration. The machine has 1 ALU, 1 MUL, and 1 memory unit. Compute $ResII$. Then explain how a loop-carried recurrence could still force MII above that resource bound.
6. For the loop in Question 5, suppose the strongest loop-carried recurrence has total latency 4 and distance 1. Compute $RecII$ and MII.
7. Explain why a modulo reservation table for initiation interval II has exactly II rows, and explain the role of the mapping from cycle x in the unrolled schedule to row $x \bmod II$.
8. Why does IMS begin at MII instead of always starting at $II = 1$? Explain the compile-time cost argument and the correctness argument.
9. In a software-pipelined loop, what work belongs in the prologue, what work belongs in the kernel, and what work belongs in the epilogue?
10. Under what conditions may a compiler legally move a load ahead of an older store? Explain why this question is fundamentally a memory-disambiguation problem rather than only a functional-unit scheduling problem.

Chapter 8

Multiprocessors and Cache Coherence

Private caches solved a single core's distance problem by keeping useful data nearby. Put two such cores beside one shared memory and the solution creates a new problem: both cores can keep a copy, but one may change it. Which copy should the next reader trust? Coherence is the machinery that makes those private copies behave like one shared location.

8.1 From Instruction-Level to Thread-Level Parallelism

Once a machine has already exploited as much instruction-level parallelism as is practical, the next major source of speedup is to execute more than one thread of control at the same time. This moves the discussion from single-core scheduling into *multiprocessor* design. At this point the most visible form of parallelism is *thread-level parallelism (TLP)*: the programmer or runtime system must decide how to divide work across multiple processing elements. Unlike instruction-level parallelism, thread-level parallelism is visible to software and therefore directly affects correctness, synchronization, and programmability.

Historically, architecture texts often used the term *processing element (PE)* for one independently executing processor in a larger machine. Modern commercial systems more often use *core*, but the older PE terminology remains useful because it applies equally well to CPUs, accelerator lanes, and other specialized execution agents. Table 8.1 places multiprocessors in the broader grain-size view of parallelism.

At the architecture-classification level, two familiar Flynn terms continue to matter: *Single instruction, multiple data (SIMD)* machines apply the same operation to multiple data elements at once. *Multiple instruction, multiple data (MIMD)* machines allow different PEs to execute different instructions on different data at the same time. Modern shared-memory multiprocessors are usually discussed as MIMD machines. That does not exclude SIMD units inside each core; it only means that the overall machine permits several independently executing instruction streams.

Parallelism level	Typical mechanism	Programmer visibility
Instruction-level	superscalar issue, out-of-order execution, VLIW	usually hidden
Data-level	SIMD vectors, packed operations, vector pipelines	partly visible through data layout or intrinsics
Thread-level	multiple threads or PEs executing concurrently	directly visible

Table 8.1: Common grain sizes of parallelism. Multiprocessor programming is primarily concerned with the thread level.

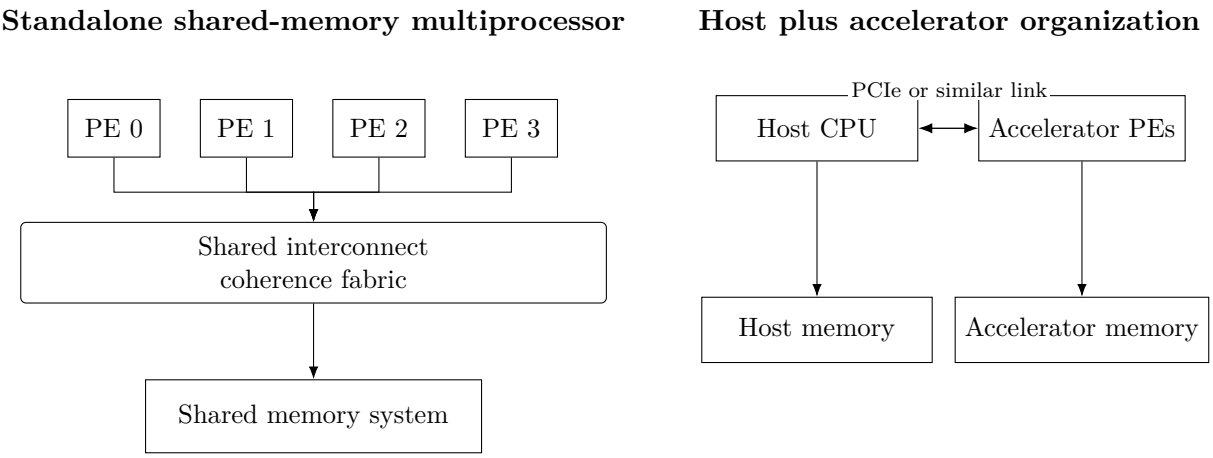


Figure 8.1: Two common multiprocessor organizations. Left: one shared-memory system. Right: a host plus accelerator with distinct memory, so software must move data explicitly.

8.2 Memory Organization and System Structure

A multiprocessor can be classified along another axis besides SIMD versus MIMD: how the memory system is organized. The two clean endpoints are *shared memory* and *message passing*.

In a *shared-memory* machine, all PEs conceptually operate in one common address space. A pointer value can identify the same logical object for every PE, and communication occurs when one PE writes data that another PE later reads. This model is attractive because it is easy to think about, but it immediately raises the cache-coherence problem once private caches are introduced.

In a *message-passing* machine, each PE has its own private memory, and data movement occurs through explicit communication operations such as `send()` and `receive()`. This model avoids coherence traffic because data is copied intentionally from one memory to another, but it forces the program to expose that communication directly. Between these extremes lies the *partitioned global address space (PGAS)* idea, where a single address space exists but some portions are substantially more expensive to access than others.

Figure 8.1 contrasts two common organizations. A standalone shared-memory multiprocessor integrates several PEs around a common memory system and runs the operating system and application code directly. An accelerator organization instead separates a host processor from a specialized parallel engine; the host launches work on the accelerator and explicitly copies data to and from accelerator memory.

Accelerators include graphics processors, machine-learning accelerators, and many other specialized engines. Their memories are often kept separate for protection, compatibility, and implementation reasons. The cost of that separation is explicit data motion: the host must copy inputs to the accelerator, launch the parallel work, and copy results back.

8.3 Interconnects: Shared Buses and Point-to-Point Links

The interconnect between PEs matters just as much as the PE count. In this text, a *bus* means a *shared observable broadcast medium*: all agents attached to the bus can observe the transactions sent across it. That observability is the key property used later by snooping coherence.

A *point-to-point interconnect* behaves differently. A packet is sent from one endpoint to another over a dedicated link, and unrelated agents do not automatically see that traffic. Modern peripheral links such as *PCI Express (PCIe)* fit this model even though informal conversation sometimes calls

Step	Event	Locally visible value of x
0	Initial memory value	memory holds 1
1	P_0, P_1, and P_2 all cache x	each cache now holds 1
2	P_0 writes $x \leftarrow 22$ in its cache	P_0 sees 22
3	P_1 writes $x \leftarrow 44$ in its cache	P_1 sees 44
4	P_2 executes `x++` using its stale cache copy	P_2 sees 2

Table 8.2: An incoherent-cache scenario. Each PE can observe a different value of the same logical variable.

them buses. PCIe uses differential pairs rather than one shared bundle of broadcast wires, so it scales better than a traditional shared bus but does not naturally support global observation of every memory transaction.

This distinction matters because some coherence mechanisms require every cache to observe every relevant request. Snooping works naturally on a shared observable medium. Directory-based coherence does not require broadcast visibility because a separate directory structure explicitly tracks who has each block.

8.4 The Cache-Coherence Problem

Shared memory becomes difficult as soon as each PE receives a private cache. Without additional coordination, different caches can hold different copies of the same memory location and update them independently. That breaks the programmer's illusion of one shared value.

Consider a memory location x whose initial value is 1. Assume three PEs each read x and therefore bring a copy into their private caches. Now suppose synchronization orders these operations across the three PEs:

1. P_0 writes $x \leftarrow 22$.
2. P_1 writes $x \leftarrow 44$.
3. P_2 executes `x++`.

If the caches do not coordinate, then P_2 may still read its stale local copy of 1, increment it, and write back 2. With that ordering, the increment should read 44 and produce 45, but an incoherent cache system can instead use the stale value and produce 2. Program order alone orders instructions within one PE; it does not establish this sequence across different PEs. Table 8.2 summarizes the failure.

Simply using write-through caches does not solve the problem. Write-through ensures that memory is updated promptly, but it does not inform the other caches that their local copies have become stale. The real issue is not whether memory eventually receives the write; it is whether all cached copies remain mutually consistent.

A coherent cache system is normally expected to satisfy three properties:

1. **Read-after-write semantics for one processor.** If a PE writes a value and no one else writes that location in the meantime, a later read by the same PE should return the value it wrote.
2. **Eventual visibility to other processors.** If one PE writes a location and another PE later

State	My copy	Other caches	Memory copy
M	dirty and writable	no one else has a valid copy	stale
S	clean	others may also hold clean copies	correct
I	not valid in this cache	unspecified	unspecified

Table 8.3: Meaning of the three MSI states for one cache block.

reads it, then after sufficient time has passed and assuming no intervening writes, the reader must eventually observe the value that was written.

3. **Consistent ordering of writes.** All PEs must agree on one order of writes to the same location.

The second property is intentionally phrased in terms of *sufficient time* rather than instant propagation. Physical systems cannot propagate signals across large chips or boards instantaneously. Coherence therefore promises eventual agreement and a consistent write story, not instantaneous zero-time updates.

8.5 Coherence Solution Families

Architecturally, there are three standard ways to solve the coherence problem.

The first is a *shared-cache* design in which all PEs communicate through one common cache structure. This avoids inter-cache inconsistency because there is only one place that holds the shared block state. The weakness is scalability: once the shared cache becomes the bottleneck, adding more PEs does not produce linear speedup even for embarrassingly parallel workloads.

The second is *directory-based coherence*. A directory tracks which caches currently hold each block and coordinates invalidations, forwarded data, or ownership transfers. This can scale better than a pure broadcast bus because the system does not require every cache to observe every request.

The third is *snooping coherence*. Here each cache has a *snooper*: logic that knows both what is currently in that cache and what transactions are appearing on the shared interconnect. If the interconnect is a shared observable medium, the snooper can react immediately when another PE requests a block that this cache currently owns or shares. The remainder of this chapter develops the classic snooping approach because it exposes the central state-machine ideas clearly.

8.6 Snooping Coherence and the MSI Protocol

A classic starting point for snooping coherence is the *MSI* protocol. Each cached block is marked as *modified (M)*, *shared (S)*, or *invalid (I)*. The meaning of those states is summarized in Table 8.3.

Two kinds of bus requests are especially important:

- `GetS`: request shared read permission for a block.
- `GetM`: request exclusive access with intent to write.

A dirty eviction produces a `WriteBack` transaction. Snoopers may also perform two high-level actions:

Transaction or action	Meaning
`GetS`	Read miss requesting a clean shared copy.
`GetM`	Read-for-ownership requesting exclusive writable access.
`WriteBack`	Dirty block is written back to memory on eviction or ownership handoff.
abort	Requester must wait or retry because another cache is still handling the block.
intervene	A snooping cache supplies the current data value directly.

Table 8.4: Common bus transactions and snooper responses in a snooping MSI system.

Current state	Local event	Bus action	Next state
I	read miss	`GetS`	S
I	write miss	`GetM`	M
S	read hit	none	S
S	write hit	`GetM`	M
M	read hit	none	M
M	write hit	none	M
S	replacement	none	I
M	replacement	`WriteBack`	I

Table 8.5: Stable-state MSI behavior from the requesting cache's perspective.

- *abort*: tell the requester to wait or retry because another cache still owns the necessary data or permission.
- *intervene*: supply the data directly rather than letting memory respond.

Table 8.4 summarizes the basic bus-level vocabulary.

From the requesting PE's perspective, the stable-state behavior is simple. If a block is invalid and the PE reads it, the cache issues `GetS` and moves to `S` when the data arrives. If a block is invalid and the PE writes it, the cache issues `GetM` and moves to `M` when ownership arrives. If a block is already shared and the PE wants to write, the cache must first upgrade through `GetM` so that all other copies are invalidated. Table 8.5 gives one compact summary.

From the snooper's perspective, the same bus requests mean something different. If a cache in state `S` sees another PE request `GetM`, it must invalidate its own copy and move to `I`. If a cache in state `M` sees `GetS`, it must ensure that the most recent data becomes visible and then downgrade to `S`. If it sees `GetM` while in `M`, it must give up ownership entirely and move to `I`. Table 8.6 summarizes the stable-state snooping behavior.

8.7 Transient States and a Worked Two-Cache Trace

Real buses and coherence fabrics are not instantaneous, so caches often need *transient states* while waiting for a request to complete. A common notation concatenates the path of the transition. For example, `IM` means that the cache started in `I`, issued a request whose destination is `M`, and is still

Current state	Snooped event	Action	Next state
I	sees `GetS` or `GetM`	none	I
S	sees `GetS`	none	S
S	sees `GetM`	invalidate local copy	I
M	sees `GetS`	provide or write back data, downgrade	S
M	sees `GetM`	provide or write back data, invalidate	I

Table 8.6: Stable-state MSI behavior from the snooper's perspective.

Step	Event	Bus msg.	P_1 state or action	P_2 state or action
1	P_1 read miss	`GetS`	$I \to S$	$I \to I$
2	P_2 read miss	`GetS`	$S \to S$	$I \to S$
3	P_1 write hit on shared block	`GetM`	$S \to M$	snoops `GetM`: $S \to I$
4	P_2 write miss while P_1 owns block	`GetM`, abort	snoops `GetM`: $M \to I$ with `WriteBack` or intervention	$I \to IM$
5	P_2 retries after ownership transfer	retry `GetM`	$I \to I$	$IM \to M$

Table 8.7: One worked MSI trace showing stable states, a transient `IM` state, and a write-ownership transfer between two caches.

waiting for that request to complete. Likewise, `SM` means a shared block has requested an upgrade to modified state but has not yet received final ownership.

Table 8.7 walks through a short two-cache scenario that mirrors the classic classroom introduction to MSI:

1. P_1 has a read miss and obtains a shared copy.
2. P_2 then has a read miss and also obtains a shared copy.
3. P_1 writes the block and upgrades from `S` to `M`.
4. P_2 then tries to write the same block, initially receives an abort, enters transient state `IM`, and retries after P_1 writes back or hands off the dirty value.

The important insight is that the coherence protocol is maintaining one simple invariant: a writable dirty copy must be unique. Whenever one cache acquires `M`, all competing copies must either become invalid or be updated into a clean shared form before execution continues.

8.8 MESI, MOESI, and MOESIF Refinements

MSI captures the essential coherence idea, but practical protocols often add extra stable states that reduce unnecessary traffic. The most common additions are *exclusive (E)*, *owned (O)*, and *forward (F)*. These states do not change the basic correctness goal. They refine who may write, whether memory already has the current value, and which cache should respond to later requests.

State	Dirty?	Shared?	Memory correct?	Responds to later `GetS`?	Write locally?
M	yes	no	no	yes	yes
O	yes	yes	no	yes	no
E	no	no	yes	yes, if another cache asks	yes
F	no	yes	yes	yes	no
S	no	yes	yes	usually no; memory or a designated responder answers	no
I	–	–	–	no	no

Table 8.8: Meaning of the commonly discussed MESI/MOESI/MOESIF states. The key distinctions are whether the copy is dirty, whether memory is already correct, and whether the cache is expected to respond to later shared requests.

Exclusive State and the Hit Line

The `E` state means that one cache has the block, the block is clean, memory is correct, and no other cache currently holds a copy. Its main advantage is that a later local write can move silently from `E` to `M` without first broadcasting `GetM`. That saves an upgrade transaction when a PE reads a block and then soon writes it.

The system needs some way to determine whether the requester is truly the only cache holding the block. In a snooping bus, a common mechanism is a *hit line*: when a cache issues `GetS`, any other cache that already has the block asserts one shared indication line. If no cache asserts the line, the requester may enter `E`; otherwise it must enter `S`. Without that shared-presence indication, the protocol cannot safely distinguish "only clean copy" from "one of several clean copies."

Owned and Forward States

The `O` and `F` states refine what happens after a cache-to-cache transfer. Suppose a cache holds a dirty block in `M` and another PE requests `GetS`. In MSI, the owner typically writes the block back or otherwise makes memory current before both copies become shared. The `O` state avoids that immediate writeback. Instead, the original owner keeps a dirty shared copy, memory remains stale, and that owner becomes the designated responder for later requests. The cost is that `O` does not permit silent local writes: because other sharers exist, a write still requires `GetM`.

The `F` state plays an analogous role for clean data. Suppose a cache has a block in `E` and another PE issues `GetS`. The original cache can provide the data directly and become the clean designated responder while memory remains correct. That cache is placed in `F`, and the requester enters `S`. On a later shared read miss, the `F` holder can respond instead of memory, avoiding ambiguity among multiple clean sharers.

Table 8.8 summarizes the practical meaning of the six commonly discussed states.

When solving coherence traces, it is useful to track three kinds of information separately after each access:

1. the stable or transient state of the block in each relevant cache,
2. whether main memory was read or written,
3. and whether the data arrived from memory or from another cache through intervention.

That bookkeeping separates a *state-machine* question from a *traffic* question. Two protocols can end in similar visible states yet generate different memory actions and cache-to-cache transfers along

the way.

One useful way to think about these states is as a permission hierarchy. `M` grants read and write permission and carries responsibility for the current dirty value. `E` grants read and write permission but for a clean exclusive copy. `O`, `F`, and `S` are read-only states from the local PE's perspective; they differ mainly in whether that cache is responsible for serving future requests and whether memory is already correct. `I` grants no rights at all.

Protocol Families

Different stable-state sets lead to familiar protocol families:

- *MOSI*: `M`, `O`, `S`, `I`. This family can capture dirty sharing without requiring an exclusive-state detector.
- *MESI*: `M`, `E`, `S`, `I`. This family adds silent clean-exclusive upgrades.
- *MOESI*: `M`, `O`, `E`, `S`, `I`. This combines dirty-sharing optimization with silent clean-exclusive upgrades.
- *MOESIF*: `M`, `O`, `E`, `S`, `I`, `F`. This further designates one clean shared responder.

The presence of these states does not fully determine a protocol. The precise transition rules are an implementation choice. The important architectural question is what optimizations the state set makes possible.

Worked State-Refinement Examples

Three short examples capture the intent of the extra states:

1. **MSI versus MESI on a private read followed by a write.** A read miss issues `GetS`. If no other cache indicates possession, MESI places the block in `E`. A later local write becomes `E`→`M` with no bus upgrade. MSI would have placed the block in `S`, so the write would require `GetM`.
2. **MOESI optimization for a dirty owner receiving `GetS`.** Suppose P_1 holds a block in `M`, and P_2 issues `GetS`. Instead of writing the block back immediately and making both copies clean, P_1 may supply the data directly and become `O` while P_2 becomes `S`. Memory is still stale, so P_1 remains responsible for that value until eviction or another ownership change.
3. **MOESIF optimization for a clean exclusive owner receiving `GetS`.** Suppose P_1 holds a block in `E`, and P_2 issues `GetS`. P_1 can supply the clean value directly, become `F`, and let P_2 enter `S`. Memory remains correct, but P_1 is now the designated clean responder for later shared requests.

8.9 Directory-Based Coherence

Snooping assumes that every interested cache can observe every relevant transaction. That assumption is natural on a shared broadcast bus but becomes difficult to maintain as systems grow larger or move to point-to-point interconnects. Directory-based coherence removes the need for one globally visible snooping medium. Instead, a separate directory structure records who has each block and coordinates the messages required to preserve coherence.

Field	Purpose
Presence bits	One bit per cache or PE indicating which agents currently hold the block.
Dirty bit	Indicates whether one cache currently holds the dirty up-to-date value.
Optional owner/forward ID	Identifies the cache responsible for responding to requests in protocols with `O` or `F`.

Table 8.9: Conceptual fields in a directory entry for one memory block. Real systems may compress or distribute this information, but these are the core ideas.

Why a Directory Exists

Two jobs must still be done even without a snooping bus:

1. The system must know which caches currently hold a copy of the block.
2. The system must serialize ownership-changing writes so that all PEs observe one consistent write order.

In a directory protocol, the directory performs both roles. It tracks sharers and also acts as the central ordering point for coherence-changing requests. That is why directory protocols preserve the causality requirement: two processors may request writes close together in time, but the directory selects one ordering and makes all invalidations and acknowledgments complete in that order.

Directory Entry Structure

At a high level, the directory is organized per memory block. Each entry must at least record which caches currently hold the block. For an MSI-style protocol, a useful minimal conceptual format is:

- a *presence bit vector* with one bit per cache or PE,
- a *dirty bit* indicating whether some cache has the only current dirty copy.

The presence vector distinguishes one sharer from many sharers. The dirty bit distinguishes a clean shared situation from a case in which one cache has the current modified value. Protocols with `O` or `F` often also record which cache is the current designated responder.

Table 8.9 shows a conceptual directory entry layout.

Directory Messages

The vocabulary shifts slightly in a directory protocol because requests no longer appear as one shared visible bus transaction. Instead, caches and the directory exchange explicit messages. Typical messages include:

- `GetS`: request a shared readable copy.
- `GetM`: request exclusive writable ownership.
- `Ack`: acknowledgment that a requested action completed.
- `Nack`: negative acknowledgment, typically meaning the requester must retry later.
- `Invalidate`: command to discard a clean shared copy.

- `Recall`: command to return a dirty value and surrender ownership.

The distinction between `Invalidate` and `Recall` is important. An invalidated clean sharer can simply discard the block and acknowledge. A recalled dirty owner must return data because memory does not yet have the current value.

Agent View versus Directory-Controller View

In a controller-oriented implementation, each cache contains an *agent* that handles processor-facing requests and local state transitions, while memory contains the *directory controller* that serializes global permission changes. The agent is responsible for actions such as servicing a local `LOAD` hit, turning a `STORE` miss into `GetM`, and holding transient states while replies are still in flight. The directory controller is responsible for sharer tracking, issuing invalidations or recalls, deciding when memory must be accessed, and determining when a requester may finally receive data or permission.

This viewpoint often expands the message vocabulary beyond the abstract `GetS`/`GetM` pair alone. It is common to distinguish:

- processor-side requests such as `LOAD` and `STORE`,
- network-visible permission requests such as `GETS`, `GETM`, and sometimes `GETX`,
- and controller responses such as `DATA`, `DATA E`, `REQ INVALID`, `INVACK`, `RECALL GOTO I`, and `RECALL GOTO S`.

`GETX` is especially useful in simulator-oriented MESI-like protocols. Architecturally, a clean exclusive copy could often upgrade silently from `E` to `M`. However, if the directory is the authoritative sharer record, the controller may still need to observe that upgrade explicitly. The agent then issues `GETX` and may wait in a transient such as `EM`, meaning "I believe I am the exclusive clean holder, and I am waiting for the directory to confirm my writable upgrade." This is not the only implementation choice, but it is a practical one because it keeps the directory's view synchronized with the cache's permissions.

Simulation-Oriented Coherence Accounting

Many coherence simulators do not model the payload data values themselves. They model only state, message flow, and where the up-to-date value logically comes from. That is sufficient for the main architectural questions: was memory read, was dirty data written back, did another cache provide the value, and did the requester already have the only clean copy?

For the same reason, some pedagogical directory simulators route a cache-to-cache transfer through the directory controller even though the value logically originated in another cache. Architecturally, that still counts as a cache-to-cache transfer because memory was not the source of truth for the requester. The most useful counters in that style of analysis are therefore memory reads, memory writes, cache-to-cache transfers, and silent-upgrade-style events. In a controller-oriented MESI implementation, an `E`→`M` case may still be counted as a silent upgrade even if the simulator models it with an explicit `GETX`/`ACK` exchange, because no peer invalidation was required.

Worked MSI-Style Directory Transactions

Consider a block whose directory entry currently shows either a clean shared state or one dirty owner.

Shared read request (`GetS`). Suppose P_2 issues `GetS`. If the directory sees no dirty owner, then memory already has the correct value. The directory sets the presence bit for P_2 and returns the block so that P_2 enters `S`. If the directory sees a dirty owner, then it sends `Recall` to that owner, waits for the data and acknowledgment, updates memory or otherwise records the returned value, sets the requester's presence bit, and then sends the data to P_2. The result is that the requester receives a correct shared copy and the directory's state matches reality.

Exclusive write request (`GetM`). Now suppose P_2 instead issues `GetM`. The directory must ensure that all competing copies disappear before granting writable ownership. If another cache has the block dirty, the directory sends `Recall` to that owner and collects the returned data. If other caches merely hold clean shared copies, the directory sends `Invalidate` to each one and waits for all acknowledgments. Only after every required acknowledgment has arrived does the directory send the data and permission to P_2, clear the other presence bits, set the presence bit for P_2, and mark the entry dirty. That ordering is what enforces the single-writer invariant and a consistent global order of writes.

Why Directories Scale Better

Directories remove the requirement that every coherence event be globally broadcast. Only the requester, the directory, and the currently relevant sharers need to participate in a transaction. That scales more naturally to larger systems and to topologies in which point-to-point hops replace one visible shared bus. The cost is storage and bookkeeping overhead. Directory state can be large, which is why real designs use compressed, sparse, hierarchical, or distributed variants. The conceptual model, however, remains the same: a directory tracks who has the block and coordinates legal ownership changes.

8.10 Directory Scaling, Limited Directories, and Distributed Organizations

The conceptual directory in the previous section is easy to explain because it behaves like one central hardware record of all sharers. The immediate drawback is size. If memory contains 2^m bytes and the coherence block size is 2^v bytes, then the number of directory entries is

$$\frac{2^m}{2^v} = 2^{m-v}.$$

Each entry then needs enough information to identify the sharers and encode control state. For a full bit-vector directory with N PEs, that means roughly N sharer bits plus a small number of control bits. As N grows, the directory becomes both large and increasingly central to system throughput. One useful way to reason about review-style metadata questions is therefore:

1. compute the number of directory entries from memory size and block size,
2. estimate the number of bits per entry from the sharer encoding plus control bits,
3. multiply the two.

For a full MSI-style bit-vector directory, the per-entry cost is roughly "one sharer bit per PE plus a few control bits." For a bounded-pointer directory, the sharer field becomes "K IDs of width $\lceil \log_2 N \rceil$ plus an overflow bit," which reduces metadata only as long as K stays small enough.

Limited Directory Schemes

One way to reduce directory size is to track only a bounded number of sharers explicitly. Instead of keeping one presence bit for every PE, the directory may keep K *presence IDs*, each wide enough to name one PE. Each ID therefore needs about $\lceil \log_2 N \rceil$ bits, so the sharing record becomes proportional to $K\lceil \log_2 N \rceil$ rather than N. This saves space when $K \ll N$, but it introduces an overflow problem: what happens when more than K sharers want the block at once?

Two broad responses are common:

1. **Broadcast on overflow.** The directory sets an overflow bit and stops tracking every sharer exactly. A later invalidation must then be sent to every PE, or at least to every PE in the relevant coherence domain, and the requester must wait for all necessary acknowledgments. This preserves correctness but increases latency and traffic.
2. **Limit sharing aggressively.** The directory evicts or invalidates one existing sharer when the pointer list is full. This preserves a bounded metadata size but can be harmful for read-mostly or read-only data, where many readers would ideally coexist.

The basic tradeoff is unavoidable: either metadata grows with the possible sharer count, or the protocol pays additional traffic or invalidation cost once that count is exceeded.

Hierarchical and Distributed Directories

The next step is to remove the single centralized directory bottleneck. One option is a hierarchy of coherence domains. For example, a machine may group several L1 caches under a private or local lower-level cache and maintain one local directory for that group. A higher-level directory then tracks which lower-level domains hold the block. This works most cleanly when the lower-level cache is *inclusive*: if an L1 holds a block, the local L2 is guaranteed to hold it as well. Then the higher directory can reason in terms of the lower-level caches rather than every individual L1.

This style is sometimes described as a *last-level private* organization, because each cluster keeps its own lower-level cache and directory knowledge. The cost is duplication: if several domains need the same block, copies may appear in several lower-level caches. That is a reasonable exchange when it simplifies lookup and scales more cleanly than one fully shared cache.

Shared Last-Level Caches and Sliced Organizations

A single shared L2 or last-level cache is not a complete solution by itself. If the L1 caches are still private, coherence information is still needed for those L1s. Worse, one shared lower-level cache quickly becomes a bandwidth bottleneck. Even if two PEs want unrelated blocks, they still contend for the same shared cache ports and internal bandwidth.

Modern designs therefore tend to partition the last-level cache into *slices*. A hash of the physical address determines which slice owns the block and which directory slice manages coherence for that address. This distributes both data and directory traffic across the on-chip interconnect. Two unrelated blocks can then proceed through different slices in parallel. Table 8.10 summarizes the major tradeoffs.

The inclusive / exclusive / non-exclusive distinction also matters here. An *inclusive* hierarchy guarantees that upper-level data is also present below, which simplifies tracking. An *exclusive* hierarchy forbids duplication between adjacent levels and maximizes effective capacity. A *non-exclusive* hierarchy allows some overlap without requiring it in all cases. Different choices change

Organization	Main advantage	Main drawback	Typical coherence implication
One shared lower-level cache	Simple to explain and easy to share data	Becomes a bandwidth bottleneck quickly	L1 coherence still required; shared cache alone does not remove the need for directory or snooping logic
Private lower-level caches with higher directory	Scales better and supports hierarchical domains	Duplicates some lower-level data across domains	Higher directory tracks which domains, not just which individual L1s, hold a block
Sliced distributed lower-level cache and sliced directory	Balances traffic and avoids one lower-level bottleneck	Requires an address-to-slice hash and a more complex on-chip interconnect	Different addresses can be processed by different directory slices in parallel

Table 8.10: Common lower-level cache and directory organizations for scalable shared-memory multiprocessors.

Sharing pattern	Typical behavior	Coherence consequence
Private	One PE accesses the data	No true sharing traffic is required
Read-only	Many PEs read, but no PE writes	Sharing is cheap once the copies are installed
Producer-consumer	One PE writes a value that another later reads	Ownership moves in an ordered handoff pattern
Migratory	A writable variable moves from one PE to another over time, often under lock protection	The writable copy repeatedly changes owners
Read-write shared	Multiple PEs repeatedly read and write the same location	Coherence traffic can become intense, especially for synchronization variables

Table 8.11: Common sharing patterns and their coherence implications.

how much coherence metadata must be stored at each level and how much traffic is required when ownership changes.

8.11 Sharing Patterns and False Sharing

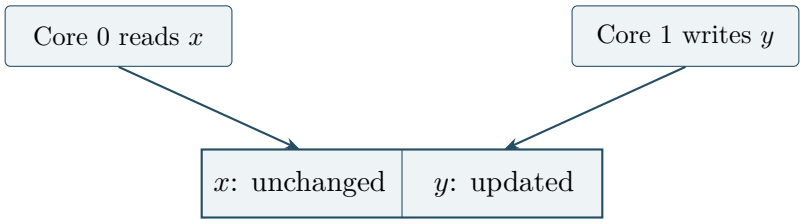
Not all shared data behaves the same way. Architecturally, it is useful to classify a block by its *sharing pattern* because that pattern strongly influences coherence traffic. Table 8.11 summarizes several common cases.

The cache operates on blocks, not on individual source-language variables. That fact creates the classic *false-sharing* problem. Suppose one block holds two variables:

- x, which is effectively read-only, and
- y, which is migratory because different PEs write it under lock protection.

Each time y changes owners, the coherence protocol invalidates or transfers the whole block. That also invalidates x, even though no PE wrote x at all. The miss on the next access to x is therefore

caused by block placement, not by true semantic sharing of x itself. Figure 8.2 isolates the placement problem.



One cache block: coherence permissions cover both variables.

Writing y invalidates Core 0's copy of the whole block.
Its next read of x can miss even though x did not change.

Figure 8.2: False sharing comes from the unit of coherence. Placing x and y in different blocks can remove this interference, at the cost of extra space and possibly a larger working set.

This is why false sharing is often described as a *coherence miss*, or informally as a "fourth C" beyond compulsory, capacity, and conflict misses. The cure is usually structural rather than algorithmic: avoid placing variables with very different sharing patterns into the same cache block. Read-only data, migratory lock-protected data, and heavily read-write shared synchronization variables are best separated when layout control is available. In trace-based reasoning, the symptom is usually easy to spot: when two variables share one block, writes to one variable increase invalidations, transfers, and misses for the other even though the source program did not truly share both variables semantically.

8.12 Programming Models: Shared Memory and Message Passing

The architecture choices above are reflected directly in parallel programming models. A shared-memory program typically divides the data among threads, lets each thread compute a local partial result, and then uses explicit synchronization to merge those results safely. The following example uses a lock to protect a critical section that updates a global sum:

```
float X[64];
float globalSum = 0;

Thread 0: sum X[0:15] locally, then lock(L); globalSum += local0; unlock(L);
Thread 1: sum X[16:31] locally, then lock(L); globalSum += local1; unlock(L);
Thread 2: sum X[32:47] locally, then lock(L); globalSum += local2; unlock(L);
Thread 3: sum X[48:63] locally, then lock(L); globalSum += local3; unlock(L);
```

The lock prevents a lost update in the critical section. Without it, two threads might read the same old value of `globalSum`, add their local sums independently, and then overwrite one another. That kind of latent synchronization bug is one reason shared memory is powerful but dangerous.

A message-passing version of the same idea avoids shared writable state:

```
Thread 0: compute local0; send(local0, 4);
Thread 1: compute local1; send(local1, 4);
```

```
Thread 2: compute local2; send(local2, 4);
Thread 3: compute local3; send(local3, 4);

Thread 4:
  globalSum = 0;
  for (i = 0; i < 4; i++)
    globalSum += receive(i);
```

In this formulation, synchronization is implicit in the blocking `receive()` operation. The collector thread waits until the required messages arrive. The advantage is portability: this model works across independent machines with separate memories. The cost is explicit communication overhead and a more demanding programming model.

8.13 Synchronization Primitives and Coherence Traffic

Coherence preserves the illusion of one shared memory, but it does not remove the need for synchronization. If several threads can update the same variable concurrently, the program still needs explicit coordination such as locks or barriers. A *mutex* or lock grants one thread exclusive access to a critical section. A *barrier* forces a set of threads to wait until the last participant arrives.

At the architectural level, the key question is how the hardware supports these operations efficiently. A lock can be represented by a single shared variable, for example:

- `lock = 0`: free
- `lock = 1`: held

The challenge is acquiring that lock atomically so that two PEs cannot both believe they succeeded.

Test-and-Set

A classic atomic primitive is *test-and-set*: set a location to 1 and simultaneously return its old value. In pseudocode, a simple spin lock looks like:

```
while (test_and_set(lock) == 1) { }
```

Unlocking then writes 0 back to the lock variable. The operation is *atomic*: no competing PE may observe or interleave a partial update. Architecturally, that atomicity may be provided by a cache-coherent read-modify-write sequence or by another equivalent mechanism.

The weakness of this simple loop is coherence traffic. Every failed `test_and_set()` still tries to acquire writable ownership of the lock variable. If many PEs spin on the same lock, they repeatedly invalidate one another's copies and keep forcing the block into a writable-owner state. The protocol spends a large fraction of its effort moving the lock variable rather than doing useful work.

Test-and-Test-and-Set

A better spin strategy is *test-and-test-and-set*:

```
while (1) {
  while (lock == 1) { }
  if (test_and_set(lock) == 0) break;
}
```

The inner read-only loop lets waiting PEs spin on a shared cached copy of the lock. Only when the lock appears free do they attempt the atomic acquisition. This changes the coherence pattern substantially:

- while the lock is held, most waiting PEs perform only repeated local reads,
- when the owner releases the lock, the write invalidates the shared copies,
- only then do contenders attempt the more expensive atomic ownership request.

The result is still a spin lock, but one that generates far less coherence traffic than naive repeated test-and-set.

Barriers and Busy Waiting

Barriers are another important synchronization primitive. They ensure that all participating threads reach a common point before any are allowed to continue. They are often necessary, but they are not free: a barrier can force faster threads to idle while waiting for the slowest one. As with locks, the architectural cost appears as traffic, serialization, and delay rather than a violation of coherence correctness.

More generally, synchronization exposes the interaction between the programming model and the coherence system. A workload with frequent lock acquisitions, spinning, or barriers may obey all correctness rules and still perform poorly because shared synchronization variables become hot spots in the coherence fabric.

8.14 Scaling and Architectural Limits

Amdahl's Law still governs the big picture. If the parallelizable fraction is 1, then the application is often called *embarrassingly parallel*, and the ideal speedup curve is linear in the PE count. Real machines fall short of that ideal once a shared resource saturates. A single shared cache saturates quickly. A shared interconnect can also become a bottleneck. Even a well-designed coherence protocol can lose efficiency if the workload repeatedly bounces writable blocks between PEs. On a bus, the shared medium itself eventually limits scaling. In larger systems, the directory's storage cost and message traffic become the next design pressure point. Synchronization hot spots and false sharing are additional practical limits because they inject communication even when the algorithm's useful work is highly parallel.

Another difficulty is verification. Coherence protocols contain stable states, transient states, race windows, and multiple simultaneously active agents. The full system state is the cross-product of one block's state in every relevant cache and directory. That makes formal verification difficult but essential: the protocol must be correct not only in clean textbook traces but also under overlapping requests and rare timing corners.

This is why the interconnect, the memory model, and the programming style cannot be separated cleanly. A coherence protocol is not only a correctness mechanism; it also determines how much communication traffic the machine must generate in order to preserve the illusion of one shared memory.

8.15 Chapter 8 Summary

Chapter 8 moves from instruction-level techniques to multiprocessor architecture. The central classifications are SIMD versus MIMD, shared memory versus message passing, and shared observable buses versus point-to-point links. Accelerator systems introduce another common organization by separating host and accelerator memory and therefore requiring explicit data movement. Private caches make shared memory fast enough to be attractive, but they also create the cache-coherence problem. Coherence requires local read-after-write correctness, eventual visibility to other processors, and a consistent ordering of writes. Shared caches, directory protocols, and snooping protocols are the three standard solution families. The MSI snooping protocol demonstrates the core state-machine ideas: blocks move among modified, shared, and invalid states in response to local requests and snooped bus traffic, with transient states used while ownership transfers are still in flight. MESI, MOESI, and MOESIF refine that baseline by distinguishing clean exclusivity, dirty shared ownership, and designated clean responders. Directory protocols generalize coherence beyond one visible bus by recording sharers explicitly and ordering ownership changes through acknowledgments, recalls, invalidations, and, in controller-oriented implementations, richer agent-to-directory messages such as `GETX`, `DATA E`, and recall commands. Because directories grow with memory size and sharer count, practical systems use limited pointers, hierarchical domains, and distributed sliced lower-level caches and directories. Sharing patterns matter just as much as protocol states: private, read-only, producer-consumer, migratory, and read-write shared data generate very different traffic patterns, and false sharing can create coherence misses even when variables are not truly shared semantically. Finally, the programming model seen by software reflects these architectural choices directly: shared memory relies on synchronization such as locks, barriers, and atomic operations, while message passing relies on explicit communication and blocking receives. In simulator-style reasoning, the practical outputs are often message traces and traffic counts rather than data values: which controller sent what, whether the latest value came from memory or another cache, and how many reads, writebacks, or silent upgrades the protocol generated.

8.16 Practice Questions

The following questions reinforce the main ideas of Chapter 8 and are intended for self-checking after the summary.

Concept Checks

1. Explain the difference between SIMD and MIMD. Then explain why a machine with multiple independent cores and a SIMD unit inside each core is still classified overall as MIMD.
2. Compare a shared-memory multiprocessor with a message-passing multiprocessor. What does the programmer gain and lose in each model?
3. Why is a traditional shared bus a natural fit for snooping coherence, while a point-to-point link such as PCIe is not naturally a broadcast snooping medium?
4. In the incoherent-cache example with initial value $x = 1$, explain why the sequence "P_0 writes 22; P_1 writes 44; P_2 executes `x++`" can produce 2 instead of 45.
5. State the three standard coherence properties in words and explain why one of them must be phrased in terms of "sufficient time" rather than instant visibility.

6. For one cache block in an MSI protocol, explain what it means for that block to be in states `M`, `S`, and `I`. In particular, explain what memory does or does not know in each case.

Trace and Protocol Exercises

7. Starting from all caches invalid for one block, use MSI to trace the access sequence `(P1,R)`, `(P1,W)`, `(P2,R)`, `(P3,R)`, `(P2,W)`, `(P1,R)`, `(P3,W)`. For each step, record the state of the block in each cache and whether the access causes a main-memory read or a main-memory write.
8. Repeat the previous exercise for MESI, but focus only on the steps whose outcomes differ from MSI. Which steps use `E`, and how does that change later behavior?
9. Starting from both caches in state `S` for the same block, show the state changes when one cache performs a write hit. Which bus transaction occurs, and what must the other cache do?
10. What does the transient state `IM` mean? Give a short scenario in which a cache would enter `IM` rather than moving immediately from `I` to `M`.
11. Explain the practical value of the `E` state in MESI. Why does the protocol need some way to detect whether another cache already has the block?
12. A cache in `M` snoops `GetS`. Compare the MSI outcome with the MOESI outcome. What extra benefit does the `O` state provide, and when must memory be updated in each case?

Directory and Simulator Exercises

13. In a directory-based MSI protocol, distinguish the responsibilities of the cache agent from those of the directory controller. Which side handles processor-facing `LOAD/STORE` requests, and which side is responsible for global ordering and invalidation fan-out?
14. In a directory-based MSI protocol, compare what the directory must do for a `GetM` when the current other copies are all shared versus when one other cache has the dirty copy.
15. Why does a directory serve as both a sharer tracker and a write-ordering point in a scalable coherence system?
16. In a controller-oriented MESI implementation, why might a cache in `E` send `GETX` and enter a transient `EM` state rather than simply changing locally to `M`? What information is the directory learning through that exchange?
17. A system has 16 PEs, supports 128 GiB of memory, and uses 64-byte coherence blocks. Estimate the number of directory entries. Then estimate the metadata cost of a full MSI-style bit-vector directory if each entry stores one sharer bit per PE plus two control bits.
18. Using the same system, suppose a limited directory stores at most K sharer IDs, each $\lceil \log_2 16 \rceil$ bits wide, plus one overflow bit. What is the largest K for which this scheme still uses fewer sharer-tracking bits than the full 16-bit sharer vector? In what kinds of sharing patterns does this scheme perform poorly?
19. Consider the trace

    ```
    (P0,W,Y), (P1,W,Y), (P0,R,X), (P1,R,Y),
    (P0,W,Y), (P1,R,X), (P0,W,Y)
    ```

    in a write-back, write-invalidate system that intervenes where possible. Compare two cases:

(a) X and Y are in the same cache block, and (b) X and Y are in different blocks. Which case causes more cache-to-cache transfers and why? Which case better avoids false sharing?

20. A controller-oriented simulator tracks memory reads, memory writes, cache-to-cache transfers, and silent-upgrade-style events. For the sequence

    `(P0,R), (P0,W), (P1,R), (P0,evict)`

    on one block under a MOESI-like protocol with intervention, which of those counters must increase, and at which steps?

21. Why is it often sufficient for a coherence simulator to model only states and messages rather than the actual payload data values?

Scaling and Synchronization

22. Why does one shared lower-level cache fail to scale cleanly even if it is coherent? Explain the role of bandwidth and port contention.

23. Explain why a sliced lower-level cache plus sliced directory reduces bottlenecks compared with one centralized directory and one shared lower-level cache.

24. In the array-sum examples, explain why the shared-memory version needs a lock around the update of `globalSum`, while the message-passing version does not require the same kind of critical section.

25. Compare repeated test-and-set with test-and-test-and-set for a spin lock. Which one causes more coherence traffic, and why?

26. Why do barriers and locks remain necessary even in a perfectly coherent shared-memory system?

Chapter 9

Synchronization and Memory Consistency

One thread prepares a result and sets a flag to announce that it is ready. Another sees the flag and reads the result. The story sounds complete, yet a coherent machine can still expose the flag before the data unless the program requests the required ordering. Shared-memory correctness therefore needs three separate ideas: coherent copies, indivisible updates, and rules for ordering communication.

9.1 Protocol States Versus Protocol Behavior

Chapter 8 introduced stable coherence states such as `M`, `S`, `I`, and later `O`. The next important refinement is that a list of stable states does *not* completely define the protocol. Two systems may expose the same end states yet differ in how they move data, when they write back to memory, and how much traffic they generate while ownership changes are in flight.

Suppose one PE holds a block in `M` and another PE requests that block. Table 9.1 summarizes several common responses.

The key semantic point is this: if the final state is `S`, then the block is supposed to be *clean*. That means memory must be made correct by the end of the transaction. If the protocol instead supports `O`, then memory may remain stale temporarily because one cache is explicitly responsible for the dirty shared copy. Thus, the presence or absence of `O` changes *when* write-back is required. The distinction matters most when memory is far away and cache-to-cache transfer is much cheaper than forcing every ownership change to go through DRAM.

This is why it is safer to think of protocol states as *permissions plus responsibilities* rather than as the full protocol. The state says what a cache is allowed to do and what cleanup it owes the rest of the system. The implementation still decides whether ownership changes happen through retry, direct intervention, deferred write-back, or some combination of those techniques.

9.2 Atomic Primitives for Shared-Memory Synchronization

Once multiple threads share writable state, the machine needs atomic operations that let software coordinate safely. Chapter 8 already introduced simple spin locks based on *test-and-set* and *test-and-test-and-set*. Those remain important because they reveal a core architectural truth: a synchronization primitive is not just a correctness device, but also a traffic pattern imposed on the coherence system.

A naive repeated `test_and_set()` forces the lock variable into a writable-owner state over and over again. That makes the modified copy bounce from PE to PE and fills the interconnect with invalidation traffic. *Test-and-test-and-set* reduces that cost by letting waiters spin mostly on shared cached copies and attempt the expensive atomic step only when the lock appears free. The lock is still correct either way; the difference is how much communication the machine must generate just to keep score.

Mechanism	Behavior when another PE requests a block currently held in `M`
Abort and retry	The current owner stalls the requester, completes the required ownership change or write-back, and then forces the requester to try again later.
Intervene	The current owner supplies the data directly, effectively pretending to be memory for that transfer and avoiding an unnecessary trip all the way to main memory.
Owned-state optimization	If the protocol supports `O`, the old owner may keep dirty responsibility while another cache receives a readable copy, so the memory write-back can be delayed until a later eviction or ownership loss.

Table 9.1: The stable state set does not by itself determine the transfer behavior. Different protocol implementations can move the same block in different ways and incur different write-back costs.

Load-Link / Store-Conditional

Another classic primitive pair is *load-link / store-conditional* (LL/SC). A load-link reads a memory value and simultaneously records a reservation on that address. A later store-conditional to the same address succeeds only if no conflicting coherence event invalidated that reservation in the meantime. If another PE obtains ownership of the block or invalidates the local copy, the reservation is cleared and the store-conditional fails.

One way to express a lock acquisition using LL/SC is:

```
do {
  old = load_link(lock);
  if (old == 0)
    success = store_conditional(lock, 1);
  else
    success = 0;
} while (!success);
```

If `store_conditional()` succeeds, the thread acquired the lock. If it fails, the loop retries. A conflicting write can clear the reservation, but failure need not prove that another thread changed the value: an ISA can permit reservation loss or spurious failure for other implementation reasons.[1]

Architecturally, LL/SC works well when the machine can cheaply observe invalidation events affecting the reservation. That is natural in a broadcast snooping setting. In a larger directory-based system, however, the reservation must survive longer-latency ownership transfers and more complex message paths, so the mechanism becomes harder to scale cleanly. The primitive remains elegant, but its implementation cost depends strongly on the interconnect and coherence organization.

[1] For a concrete example, see the RISC-V atomic-extension specification in Appendix F.

Fetch-and-Op and Fetch-and-Add

A more general atomic primitive is *fetch-and-op*: return the old value of a location while atomically applying some operation to it. In abstract form,

$$\texttt{fetch_and_op}(x, c): \qquad old = x, \qquad x \leftarrow x \ op \ c, \qquad \text{return } old. \tag{9.1}$$

The most common special case is *fetch-and-add*, where the operation is integer addition. Implementations may provide that primitive directly in hardware, build it from LL/SC, or realize it through another atomic read-modify-write path.

Fetch-and-add is extremely useful for dynamic work distribution. Instead of statically assigning iterations to threads, the runtime can let each worker claim the next available iteration number as it becomes free:

```
while ((i = fetch_and_add(next, 1)) < N) {
  body(i);
}
```

This is *dynamic work claiming* with one iteration per claim. Guided self-scheduling is a related policy that starts with larger chunks and reduces their size as less work remains; it is not the same as repeatedly claiming one iteration.[2] It handles uneven iteration costs naturally: a faster core simply claims more iterations over time, while a slower core claims fewer.

High-level interfaces often hide these mechanics. *OpenMP* is the standard directive-based shared-memory API. A compiler may translate a parallel loop into a runtime structure that internally resembles repeated work claiming. *MPI*, by contrast, is the standard message-passing interface; it exposes explicit communication rather than a shared atomic counter. The architectural lesson is that even high-level APIs eventually bottom out in a small set of atomic ownership and ordering operations.

9.3 Reducing Synchronization Contention

Correct synchronization is necessary, but naive synchronization can turn one hot variable into the dominant source of coherence traffic. Two common responses are to reduce the number of contenders hitting the same lock at once and to structure barrier release so consecutive barrier instances do not interfere with one another.

Tree and Tournament Locks

A *tree lock* or *tournament lock* replaces one heavily contended global lock with a hierarchy of smaller locks. For example, with 16 PEs one design might let: P_0 through P_3 compete first on one local lock, P_4 through P_7 compete on another, P_8 through P_{11} and P_{12} through P_{15} do the same, and then only the local winners advance to higher-level locks. That structure reduces the invalidation flood at the top of the tree because not all 16 contenders compete for the same variable at the same instant. Instead, most of the traffic stays localized within smaller competing groups.

The price is latency. Acquiring the final global lock now requires winning several stages in sequence. Tree locks therefore trade reduced contention for longer acquisition paths. They are attractive when raw contention is the dominant problem and the extra hierarchy depth is cheaper than one enormous burst of invalidations on a single lock variable.

[2] See the OpenMP worksharing-loop specification listed in Appendix F.

Barrier Construction with Fetch-and-Add

Fetch-and-add also provides a simple software barrier. The main idea is to count arrivals and release all threads when the last one arrives. The subtle part is making one barrier instance distinct from the next, especially if the barrier sits inside a loop. A common solution is a *sense-reversing barrier*, in which each thread keeps its own local phase bit.

A compact form is:

```
Shared: count = 0, phase = 0
Per thread: localPhase = 0

barrier(N):
  localPhase = 1 - localPhase
  if (fetch_and_add(count, 1) == N-1) {
    count = 0
    phase = localPhase
  } else {
    while (phase != localPhase) { }
  }
```

The last arriving thread is the one for which

$$\texttt{fetch_and_add}(count, 1) = N - 1. \tag{9.2}$$

That thread resets the counter and flips the shared phase. All other threads spin until they observe the new phase value. The phase bit plays the role of the "toggle" idea often used in informal explanations: it prevents one completed barrier instance from being confused with the next one.

Hardware can implement a similar idea more directly. On a bus-based machine, one simple design gives each PE a dedicated arrival line. A PE asserts its line when it reaches the barrier, and everyone leaves once all lines are asserted. This is conceptually simple and resembles the one-hot signaling style used for other shared bus signals, but it does not scale well because it consumes dedicated hardware resources and assumes a shared observable medium.

Review Emphasis: Barrier Versus Lock A lock protects a critical section: at most one thread owns the protected region at a time. A barrier is a phase boundary: all participating threads wait until the last one arrives, then all proceed. The two are easy to confuse because barriers can be implemented using atomic operations and locks can be used inside synchronization libraries, but their purpose is different. Locks provide mutual exclusion. Barriers regroup threads that are allowed to run independently during the phase before the barrier. This is why barriers are common in bulk-synchronous programming models, including GPU-style kernel phases, but become expensive if inserted after every small step.

9.4 Transactional Memory

Locks are not the only way to protect a critical section. Another idea is *transactional memory*: execute the critical section speculatively, then commit the result only if no conflicting access occurred. The programmer's conceptual model is close to a database transaction:

1. begin a transaction,
2. read and write the shared data optimistically,
3. commit if no conflict occurred,
4. otherwise abort and retry.

In hardware transactional memory, the processor typically tracks the transaction's read and write sets using the cache hierarchy. The accessed lines must remain resident until commit, and if some other PE invalidates one of those lines before the transaction ends, the transaction aborts. A pseudocode sketch looks like:

```
retry:
  xbegin
    critical section body
  xend
```

If the transaction aborts, control returns to a retry path or to a lock-based fallback.

The attraction is clear. Transactional memory can avoid some of the awkwardness of nested locks and can eliminate explicit lock variables in lightly contended critical sections. If conflicts are rare, many threads may complete without ever spinning on a shared lock.

The limitations are equally important. Transactions can fail because of true data conflicts, because the read or write set is too large for the available cache tracking, or because interrupts, exceptions, or implementation-specific hazards force an abort. Security and speculation concerns also make this design space delicate: mechanisms that delay architectural visibility or suppress exceptional behavior must be engineered very carefully to avoid side channels. As a result, practical support is vendor-specific and often paired with conservative fallbacks.

A slower but conceptually similar alternative is *software transactional memory*. Here the runtime records metadata in software, checks for conflicts explicitly, and retries when necessary. That approach sacrifices performance but keeps the optimistic-concurrency programming model.

9.5 Memory Consistency Beyond Coherence

Coherence and consistency are related, but they are not the same guarantee. *Cache coherence* constrains the order in which all processors observe writes to *one memory location*. *Memory consistency* constrains the relative order of operations to *different locations*. A machine can therefore be perfectly coherent and still violate a programmer's intended synchronization if it allows aggressive reordering across different addresses.

Data Races

A *data race* occurs when two or more accesses touch the same location, at least one of those accesses is a write, and the ordering among them is not properly constrained. That is the real danger behind relaxed memory ordering. The machine is free to overlap or reorder operations for performance, but if the program does not establish the intended order, the result can become nondeterministic or simply wrong.

The subtle point is that the race may involve more than one variable. An incorrectly constructed barrier, for example, can race on both the arrival counter and the phase or completion flag. Even if each individual variable is coherent, the combined protocol can still fail if the updates are observed

in the wrong cross-variable order. Coherence therefore removes only one class of failure; it does not automatically make parallel code race-free.

Strict Consistency

The strongest imaginable model is *strict consistency*. In a strict-consistency machine, all processors observe memory operations in one global order that matches actual time. One way to imagine such a system is a central arbiter that repeatedly asks each processor, one at a time, to perform its next memory access. If one processor's write physically happened first, every later read must see that fact.

This model is deterministic and easy to reason about. With the same timed inputs, the machine produces the same outputs. The cost is extreme. Strict consistency is effectively saying that the machine must behave as though there were only one instantly authoritative memory, no hidden reorderings, and no cache behavior that could let processors observe temporarily different views of the world. That makes it useful as a thought experiment and almost useless as a high-performance implementation target. In worked problems, a supplied actual-time order or global-clock order must therefore be respected exactly under strict consistency.

Sequential Consistency

The best-known practical teaching model is *sequential consistency*. It weakens strict consistency slightly by discarding the requirement that the global order match physical time exactly. Instead, it requires only two things:

1. each processor's memory operations appear in the global order in the same order as that processor's program,
2. all processors observe one common total interleaving of those per-processor streams.

This is equivalent to imagining that the program ran on a uniprocessor that interleaved whole-thread memory operations in some unpredictable but legal order. Thus, in an interleaving exercise, any total order is legal as long as each thread's local program order is preserved, even if the chosen interleaving does not match the originally supplied real-time order.

A classic example uses two shared variables initially set to 0:

```
Processor 1:            Processor 2:
  A = 1                   B = 1
  BB = B                  AA = A
```

Coherence alone only says that all processors will agree on the order of writes to `A`, and separately on the order of writes to `B`. It says nothing about the cross-address ordering between `A = 1` and `B = 1`. If the machine allows the later loads to become visible before the earlier stores, then outcomes such as `AA = 0` and `BB = 0` become possible even though each individual location remains coherent.

Under sequential consistency, however, the result `AA = 0` and `BB = 0` is impossible. To get `AA = 0`, Processor 2's load of `A` would have to appear before Processor 1's earlier store of `A`. To get `BB = 0`, Processor 1's load of `B` would have to appear before Processor 2's earlier store of `B`. Those two requirements create a cyclic ordering contradiction once both processors' program orders must be preserved. Sequential consistency is therefore much stronger than plain coherence.

It is still expensive to implement. Modern processors want to issue loads and stores out of order, buffer writes, speculate past unresolved misses, and overlap communication aggressively. A fully

sequentially consistent implementation must either forbid much of that behavior or detect when speculation violated the model and then roll the machine back. That is possible, but not cheap.

Weak and Relaxed Consistency

Real multiprocessors therefore use weaker or more selective models. There is no single weak-consistency design; there are many variants. The common idea is that not every memory access deserves the same ordering guarantee.

One useful mental model is to divide shared accesses into two broad categories:

- *ordinary* or uncontended accesses, where the hardware is free to reorder aggressively if no synchronization rule is violated,
- *synchronization* or contended accesses, such as lock variables, atomic counters, and flags, which must carry stronger ordering guarantees.

Atomic operations such as `test_and_set()`, `fetch_and_add()`, and LL/SC-based updates can support synchronization, but atomicity alone does not imply acquire or release ordering. The instruction or API must also request the ordering needed by the protocol. For example, an atomic counter used only to allocate distinct work indices can need weaker ordering than an atomic flag that publishes data.

Memory Fences and Acquire/Release Ordering

The hardware mechanism that exposes this stronger ordering is the *memory fence*. Common fence styles include:

- a *store-ordering fence*, which at least orders earlier stores before later stores in the model being discussed,
- a *load-ordering fence*, which at least orders earlier loads before later loads,
- and a *full memory fence*, which orders the specified earlier loads and stores before the specified later loads and stores.

Fence names are ISA-specific: do not infer load-to-store or store-to-load ordering from the word "fence" alone. Check which predecessor and successor access classes the actual instruction orders. Restricting that overlap can cost performance, especially when earlier memory operations are slow.

This is why locks are more than just atomic variables. An *acquire* operation must prevent later accesses inside the critical section from floating before the lock is obtained. A *release* operation must prevent earlier critical-section updates from remaining hidden after the lock is released. Put differently, a correct lock implementation behaves like an ordering device as well as a mutual-exclusion device.

The practical synchronization bug usually appears in a message-passing pattern such as:

```
Producer:                Consumer:
  x = 1                    while (flag == 0) { }
  flag = 1                 z = x
```

If the consumer observes `flag == 1`, the programmer expects `z` to become 1 as well. Coherence by itself does not guarantee that expectation, because the machine might make the write to `flag` visible before the earlier write to `x` becomes visible to the consumer. The two variables are each

coherent on their own, but the cross-variable ordering required by the synchronization protocol is missing.

One clear repair is to publish and consume the flag with release and acquire operations:

```
Producer:                  Consumer:
  x = 1                      while (load_acquire(flag) == 0) { }
  store_release(flag, 1)      z = x
```

Assume `flag` is atomic, initially 0, the consumer's successful load reads the producer's release store, and no other thread modifies `x`. Then the release/acquire handoff makes the earlier write of `x` available to the consumer's later read. Figure 9.1 traces that relationship. In C or C++, use the language's atomic operations or synchronization library so both the compiler and hardware honor it; ordinary shared variables plus invented fence function names are not a complete implementation.

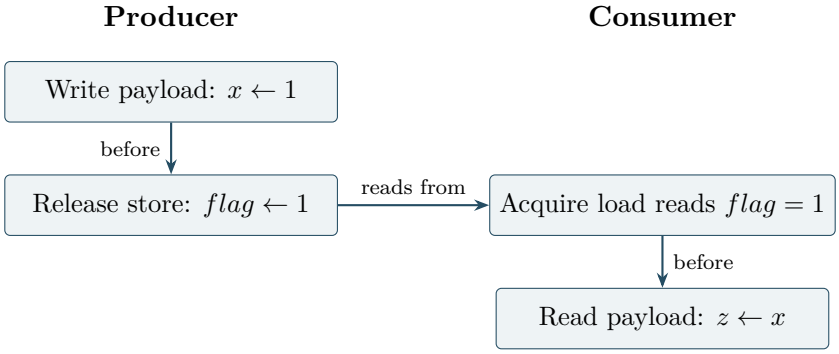


Figure 9.1: A release/acquire handoff orders the payload across two threads. The acquire must read the value published by the release. With the stated assumptions, the final read obtains $x = 1$. Coherence of each location alone does not establish this chain.

One brute-force solution would be to forbid essentially all load/store reordering and march memory operations through the system in a globally rigid order. That is simple to explain and expensive to implement. Real machines instead treat synchronization operations specially. An acquire prevents later memory actions from moving before the synchronization event, while a release prevents earlier actions from moving after it. Locks, unlocks, atomic synchronization primitives, and explicit memory fences rely on this idea. The result is a compromise: ordinary data accesses may still be reordered for performance, but synchronization points restore enough order for correctly written parallel programs to communicate.

Review Emphasis: The Consistency Failure Mode The recurring mistake is assuming that coherence makes the flag protocol correct. Coherence makes all processors agree on the order of writes to `x`, and separately on the order of writes to `flag`. It does not, by itself, force the write to `x` to become visible before the write to `flag`. That cross-address guarantee is exactly the job of consistency rules, fences, and acquire/release semantics.

This is the larger architectural lesson of the chapter pair. Chapter 8 showed how the machine preserves one shared-memory illusion for a single block at a time. This chapter adds the second half of the story: once software uses that shared memory for synchronization, the machine must also provide atomic updates and cross-address ordering guarantees. Without those additional guarantees, coherence alone is not enough.

9.6 Chapter 9 Summary

Chapter 9 extends the multiprocessor discussion from block ownership into synchronization and ordering. The first key idea is that stable coherence states do not by themselves define the whole protocol: abort/retry, direct intervention, and owned-state optimizations can all move the same block differently and change when write-back occurs. Atomic synchronization then builds on that coherence substrate. Repeated test-and-set is correct but traffic-heavy, while test-and-test-and-set reduces invalidation pressure by letting waiters spin on shared copies. Load-link / store-conditional offers another way to express atomic lock acquisition, and fetch-and-op primitives generalize atomic update so runtimes can implement work claiming, counters, and barriers. Tree locks reduce the invalidation storm at one global lock by spreading contention across a hierarchy, while sense-reversing barriers distinguish one barrier instance from the next with an explicit phase bit. Transactional memory replaces explicit lock ownership with optimistic execution and commit-or-retry semantics, but it depends on careful hardware or runtime support and may still abort under conflict or implementation pressure. Finally, coherence and consistency are distinct: coherence orders writes to one location, whereas memory consistency governs the visibility of operations to different locations. Strict consistency is conceptually simplest and essentially impractical. Sequential consistency preserves each processor's program order within one shared global interleaving, which makes it much easier to reason about but still costly to implement efficiently. Weaker models therefore rely on synchronization variables, atomic operations, and fences to provide strong ordering only where software actually needs it. Correct shared-memory synchronization thus requires both atomic primitives and explicit ordering rules such as acquire/release semantics or fences.

9.7 Practice Questions

The following questions reinforce the main ideas of Chapter 9.

1. Explain why "the states are not the protocol." Use a block currently held in `M` by one cache and requested by another cache to illustrate your answer.
2. Compare abort/retry, intervene, and the `O` state as ways of handling a request to a block currently held dirty in another cache. Which choice can delay write-back the longest, and why?
3. Why does repeated `test_and_set()` cause more coherence traffic than test-and-test-and-set?
4. Describe the roles of *load-link* and *store-conditional*. What coherence event causes the store-conditional to fail?
5. What is the difference between *fetch-and-op* and *fetch-and-add*? Why is fetch-and-add useful for dynamic loop scheduling?
6. A workload uses one global lock and shows a severe invalidation storm when many threads enter the critical section together. Why might a tree lock help, and what latency cost does it introduce?
7. In a fetch-and-add barrier, why is a phase or toggle bit needed in addition to the arrival counter?
8. Compare a software barrier based on an atomic counter with a hardware barrier built from one dedicated signal line per PE. What assumptions does the hardware version make about

the interconnect?

9. When should a programmer use a lock, and when should a programmer use a barrier? Give one example where each is the natural primitive.
10. What problem is transactional memory trying to avoid compared with conventional lock-based critical sections? Give two reasons a transaction may still abort.
11. Distinguish clearly between cache coherence and memory consistency.
12. In the producer/consumer example with `x` and `flag`, explain how a consumer can legally observe `flag == 1` yet still read an old value of `x` if the machine provides coherence but not sufficient consistency ordering.
13. Consider two threads:

```
Thread A:  A1: tA = M[0x3008]
           A2: tA = tA + 4
           A3: M[0x3004] = tA

Thread B:  B1: tB = M[0x3004]
           B2: tB = tB + 9
           B3: M[0x3008] = tB
```

    Assume the initial values are $M[0x3008] = 20$ and $M[0x3004] = 10$. If the actual-time order is $A1, B1, A2, B2, A3, B3$, give the final values under strict consistency. Then give two different legal interleavings under sequential consistency that do *not* follow that actual-time order, and state the final memory values in each case.
14. What does sequential consistency guarantee, and why is it usually more expensive than weaker models with acquire/release ordering?
15. Define a data race precisely. Why can a program still contain a data race even when every individual location is cache coherent?
16. Compare strict consistency with sequential consistency. Which one is tied directly to physical time, and which one only requires preservation of each processor's program order?
17. In the two-processor example with `A`, `B`, `AA`, and `BB`, explain why the outcome `AA = 0, BB = 0` is forbidden under sequential consistency.
18. What do store fences, load fences, and full memory fences each prevent? Why are they expensive?
19. Why is read-only shared data usually less problematic for weak consistency than data protected by locks or atomics?

Chapter 10

Parallel Interconnection Networks: Topology, Routing, and Flow Control

An ownership request, a cache block, and an acknowledgment all have to travel somewhere. On a small machine a shared bus may carry them; on a larger one, several messages can compete for links and buffers along different routes. A useful network must answer three questions: where can a message go, which path will it take, and what happens when the next step is busy? Topology, routing, and flow control answer those questions in turn.

10.1 Why Parallel Interconnects Are Their Own Topic

Shared-memory multiprocessors ultimately depend on an interconnection fabric that moves requests, data, invalidations, acknowledgments, and coherence messages among processors, caches, directories, and memories. That fabric is not a minor implementation detail. Its structure determines whether the machine can scale, how quickly requests reach their destinations, and how badly latency grows when many transfers contend for the same resources.

Parallel interconnection networks differ from wide-area communication networks in several ways. First, they usually move more than one bit per transfer step. A link therefore has a width measured in bits per cycle, and a common transfer unit is the *flit* (flow-control digit), which is the amount of data the network forwards as one flow-control object. Second, the machine is engineered as one tightly integrated system, so the design emphasis shifts away from end-to-end repair of arbitrary external faults and toward topology, routing, buffering, and router behavior. Table 10.1 summarizes the contrast.

10.2 Direct and Indirect Topologies

A *topology* is the graph describing which nodes are connected directly to which other nodes. Two broad categories are common.

Design aspect	Parallel interconnection network	Conventional communication network
Transfer granularity	Usually many bits in parallel; flit width matters directly	Bit-serial or packet-serial transmission over long links is common
Reliability focus	Built as one integrated machine; low-latency movement is the main concern	End-to-end recovery and fault tolerance dominate
Central design questions	Topology, routing, flow control, buffering, router design	Addressing, congestion management, reliability, heterogeneous paths

Table 10.1: Parallel interconnection networks optimize for tightly coupled communication inside one machine rather than for general-purpose wide-area connectivity.

Step	3-bit Gray code
0	000
1	001
2	011
3	010
4	110
5	111
6	101
7	100

Table 10.2: One Gray-code ordering for the eight nodes of a 3-dimensional cube. Consecutive entries differ in exactly one bit.

In a *direct topology*, each processor or node has explicit links to selected neighboring nodes. The processors themselves are vertices in the communication graph. Examples include meshes, tori, and the historically important *hypercube*.

In an *indirect topology*, processors connect to a separate network of switching elements. The communication graph is therefore built from routers or switches rather than from processor-to-processor links alone. A *crossbar* is the cleanest example: any input may be connected to any output, so the switch is functionally all-to-all. That flexibility is attractive, but full crossbars become expensive very quickly as the processor count grows.

Indirect organizations were sometimes called *dance hall* machines in older architecture discussions because the processors all had to meet in the central switching fabric rather than walking directly to one another. The metaphor is informal, but the architectural point is real: the switch fabric becomes the place where path selection, contention, and buffering are resolved.

10.3 Direct Networks: Hypercubes, Meshes, and Tori

10.3.1 Hypercubes and Gray-Code Intuition

A *d-dimensional hypercube* connects nodes so that each node has one link per dimension and neighboring nodes differ in exactly one coordinate. The standard counting facts are

$$N_{nodes} = 2^d, \qquad degree = d. \tag{10.1}$$

Thus a 3-dimensional hypercube has 8 nodes, each with degree 3. A 10-dimensional hypercube would have 1024 nodes, each with degree 10.

One useful numbering intuition is *Gray code*, a binary ordering in which consecutive labels differ in only one bit. Gray code is not the only possible node labeling, but it is a convenient way to trace a path through a cube because moving from one label to the next changes only one coordinate. Table 10.2 shows one 3-bit Gray-code sequence.

Hypercubes were historically attractive because their diameter grows only logarithmically with node count. The physical difficulty is packaging. As the conceptual dimension count grows, the machine still has to be laid out in ordinary physical space, and routing many long links cleanly becomes increasingly awkward. That is one reason hypercubes became less popular in commercial systems even though the graph-theoretic properties are elegant.

10.3.2 K-ary N-Dimensional Networks

A broader family of direct networks is the *k-ary n-dimensional* topology. It has n dimensions and k nodes in each dimension, so the total node count is

$$N_{nodes} = k^n. \tag{10.2}$$

A 4-ary 2-dimensional network therefore has $4^2 = 16$ nodes.

The two most common members of this family are the *mesh* and the *torus*. A mesh connects each node to its neighbors along each dimension but does not wrap at the edges. A torus adds wraparound links so that each dimension becomes a ring. Thus a *k-ary n-torus* is the direct generalization of a 1-dimensional ring to multiple dimensions. In a torus, each node typically has degree $2n$ because it has two neighbors per dimension.

A 4-ary 2-torus can be visualized as a 4-by-4 grid in which the left and right edges wrap together and the top and bottom edges wrap together. That wraparound is what gives the torus its name: the graph can be drawn on the surface of a donut without edge crossings. Modern many-core and server-class designs have often favored mesh- or torus-like organizations because they map naturally onto on-chip planar layouts.

10.3.3 Rings as a Pedagogical Baseline

Before moving to richer meshes and tori, many teaching simulators start with a simple deterministic ring. A ring is just the 1-dimensional torus case, so it is still a direct network. Its value is clarity: message latency is easy to interpret as hop count times link latency, and congestion is easy to see because all traffic shares one simple path structure.

For example, in a unidirectional ring with 1-cycle links, a message that must traverse 5 hops has a zero-load network latency of 5 cycles before router or queueing delays are added. If each node can consume at most one message per cycle, then additional delay appears as queueing rather than as route ambiguity. Many teaching models intentionally use this simplified structure, sometimes together with very deep input queues, so that the main architectural questions are message ordering, hop count, and contention rather than finite-buffer loss.

10.3.4 Physical Layout and Wire-Length Balance

Logical dimensionality is only part of the story. The network must still be fabricated in a planar physical layout. Longer wires have higher capacitance, and higher capacitance increases delay. An otherwise elegant logical network can therefore perform badly if its physical layout creates a few unusually long wires that become timing bottlenecks.

A simple ring illustrates the problem. If an 8-node 1-dimensional torus is drawn naively as a straight row with one long wraparound connection from the last node back to the first, that one link becomes much longer than the others. The result is an *unbalanced* layout: one path suffers distinctly higher delay than the rest.

A better approach is a *jump-length* style layout that distributes wraparound connectivity more evenly. The idea is not that all wires become identical; that is impossible in a strict planar embedding of a ring. The idea is that the layout avoids a single pathological wraparound and instead spreads wire lengths so the network is more balanced overall. That balance matters because the system clock must ultimately respect the longest critical wire delay.

10.3.5 Bisectional Bandwidth

A standard figure of merit for direct networks is *bisectional bandwidth*. Conceptually, split the network into two equal halves and sum the bandwidth of the links crossing that cut. The *bisection bandwidth* is the minimum such value over all equal-halves cuts. If every link has the same bandwidth W_{link}, then the value reduces to

$$B_{bisect} = N_{cut} \times W_{link}, \tag{10.3}$$

where N_{cut} is the number of links crossing the minimum bisection.

Bisection bandwidth is useful because it compresses a network's best-case communication potential into one number. It does not predict the exact runtime of a real workload, because real traffic creates conflicts and hot spots, but it does reveal a hard structural limit on how much data can move between large halves of the machine. The minimum is over *all* equal-halves partitions, not just visually straight cuts. For the regular square mesh and torus considered here, a dimension-aligned cut attains the minimum and makes the count easy to see. A diagonal or zig-zag partition is still a candidate if it divides the nodes equally; it does not change the definition, and counting extra crossings cannot increase the minimum.

A 4-ary 2-mesh with 1-unit links has a bisection bandwidth of 4 link-units because the natural equal cut crosses four links. The corresponding 4-ary 2-torus has a bisection bandwidth of 8 link-units because the wraparound links create a second independent set of crossings. For this reason, an equally sized torus has twice the bisection bandwidth of the corresponding mesh. That does not make a torus automatically superior in every design, but it does explain why wraparound links are architecturally attractive.

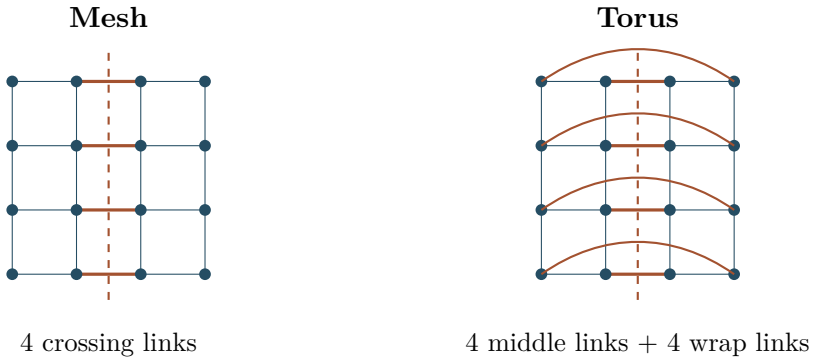


Figure 10.1: Equal halves in a 4×4 mesh and torus. Brown links cross the dashed partition. The torus's horizontal wraparound links add four crossings; vertical wraparound links are omitted because they stay within each half. Count each bidirectional link once using the same bandwidth convention in both panels.

10.4 Indirect Networks and Multi-Stage Fabrics

A full crossbar is not the only indirect topology. A cheaper alternative uses several stages of small switches. The standard building block is a 2-by-2 switch element. Each such switch can either pass its two inputs straight through or cross them. By composing many of these elements across multiple stages, the network can connect many inputs to many outputs without paying full crossbar cost.

One of the classic examples is the *omega network*. Between successive switch stages, the wires

Stage	Destination bit consumed	Example switch choice
0	1	choose the output corresponding to bit 1
1	0	choose the output corresponding to bit 0
2	1	choose the output corresponding to bit 1

Table 10.3: One bit of the destination is consumed at each stage in an omega network. Exact straight/cross labeling depends on the wiring convention, but the routing decision is driven directly by the destination bits.

are connected in a *perfect shuffle* pattern. For an N-input omega network, the number of stages is

$$stages = \log_2 N. \tag{10.4}$$

An 8-input omega network therefore has 3 stages, which fits naturally with a 3-bit destination address.

The attractive property is routing simplicity. At each stage, one bit of the destination address determines whether the local switch should send the flit along one output or the other. Table 10.3 shows a simple route example for destination `101` in an 8-endpoint omega network.

The weakness of a multi-stage network is *blocking*. Two transfers may need the same internal link or the same switch output at the same time. Unlike a nonblocking crossbar, the omega network does not generally offer an independent path for every pair of simultaneous requests. Thus the network may be fast when contention is low and much slower when several requests collide in the same internal region.

10.5 Routing Algorithms

Routing answers the question: given a source and a destination, which path should the message follow? The answer is not purely geometric. The chosen algorithm affects congestion balance, hardware complexity, deadlock risk, and average latency.

10.5.1 Deterministic Routing

A *deterministic* algorithm always chooses the same path for a given source-destination pair. The standard example is *dimension-order routing* (also called dimensional routing). In a 2-dimensional mesh or torus, the message may always traverse the x direction first and then the y direction. The method is simple, cheap, and easy to verify.

The weakness is load concentration. If many source-destination pairs are forced to use similar middle links, those links become hot spots even when alternate paths exist elsewhere in the network. Thus deterministic routing is attractive for simplicity but often poor at balancing congestion.

10.5.2 Adaptive Routing

An *adaptive* router uses local or global information about congestion to choose among multiple legal next hops. If one output queue is full or heavily loaded, the router may choose a different direction and still attempt to reach the same destination. This can reduce hot spots substantially.

The difficulty is correctness. If routers are allowed to keep redirecting traffic in response to temporary congestion, they can create cycles in which packets keep moving but never actually reach

the destination. That is *livelock*. Adaptive routing therefore needs carefully designed restrictions or escape paths to guarantee forward progress.

10.5.3 Oblivious Routing and Valiant's Idea

A third option is *oblivious* routing, which does not respond to current congestion but also does not force all traffic onto one canonical path. The classic example is *Valiant's algorithm*. To route from source X to destination Y:

1. choose an intermediate node Z such that $Z \neq X$ and $Z \neq Y$,
2. route from X to Z using a simple deterministic method such as dimension-order routing,
3. then route from Z to Y the same way.

The point is load balancing. Deterministic routing tends to punish the same central links repeatedly, while Valiant-style oblivious routing spreads traffic more evenly across the network. The cost is that many messages travel farther than the geometric shortest path. That is often a good trade: equalizing congestion can improve overall throughput even when some individual routes become longer. In practical implementations, obvious shortcuts are still worthwhile; if the destination already lies directly along the chosen legal path, there is no reason to force an unnecessary detour.

10.6 Flow Control and Congestion Management

Routing chooses a path. *Flow control* decides whether the next step along that path is currently safe. If the desired buffer or output link is full, the network must decide what to do with the blocked traffic.

10.6.1 Circuit Switching and Packet Switching

The oldest answer is *circuit switching*. A source first sends a setup request that reserves the entire route. Once the reservation succeeds, the source transmits along that dedicated path, and finally sends a teardown message to release the resources. Circuit switching can be very predictable and is attractive for real-time traffic, but it ties up resources even when the path is only intermittently active. Under load, this can create fairness problems and long waits.

The more common answer is *packet switching*. A message is divided into packets, each of which carries its own routing information. A packet typically contains a header, a payload, and a tail marker or trailer. Different packets may take different routes to the same destination, and no path must be reserved for the entire duration of a conversation. That flexibility is why packet switching dominates general-purpose interconnects.

10.6.2 Packets and Flits

A *packet* is the logical message unit that contains routing information. A *flit* is the flow-control unit used to move the packet through the network. The packet may therefore be broken into a header flit, one or more body flits, and a tail flit. The header carries the routing information; the body flits carry most of the data; the tail marks the end of the packet.

This distinction matters because flits do not behave like fully independent packets. A packet's flits must remain in order. The network therefore needs routing information only in the header, not repeated redundantly in every flit. That saves bandwidth, but it also means the network cannot freely reorder flits belonging to the same packet.

10.6.3 What Happens at Congestion?

Suppose a router wants to forward a packet but the desired downstream resource is full. Several broad responses are possible.

One option is a *negative acknowledgment* (NACK): tell the source the packet did not arrive so the source can retry later. That is workable in some best-effort systems but poor for real-time streams, where repeated drops create visible or audible artifacts.

A second option is *misrouting*: take some alternate path instead of waiting. This can preserve motion, but unless the routing rules are designed carefully it can also create livelock.

A third option is *wait and store-and-forward*. The router buffers the packet locally, retries later, and uses *back pressure* to prevent upstream senders from overflowing the waiting space. Back pressure can be implemented with a simple on-off scheme (stop when full, resume when space opens) or with a *credit-based* scheme in which the sender tracks how many downstream buffer slots are currently available.

10.7 From Store-and-Forward to Virtual Channels

10.7.1 Store-and-Forward

In pure *store-and-forward* switching, a router waits until it has received the entire packet before sending that packet to the next router. The method is simple but expensive in latency because each hop adds a full-packet reception delay. Large packets therefore move slowly across long paths.

10.7.2 Cut-Through Routing

Cut-through routing reduces that latency. As soon as the header arrives and the next hop is known to be available, the router begins forwarding the packet before the entire packet has arrived locally. This pipelines transmission across links and can reduce end-to-end delay substantially.

The tradeoff is resource reservation. If the packet is allowed to begin moving through the router, the downstream side must be able to absorb the entire packet without losing ordering. That often means buffering or reserving enough space for the whole packet, which can be wasteful.

10.7.3 Wormhole Flow Control

Wormhole flow control pushes the idea further by moving the packet as a chain of flits. Only a small amount of buffering is needed at each hop, often effectively one flit per virtualized buffer. This makes the network much more storage-efficient than packet-sized buffering.

The weakness is *head-of-line blocking*. If the header flit is blocked waiting for a resource, the rest of the packet is stretched across several links and routers behind it. Those occupied resources may then prevent unrelated traffic from advancing even though the conflict originated only at the blocked head. Wormhole is therefore fast and storage-efficient but vulnerable to interference between unrelated flows.

10.7.4 Virtual Channels

The standard modern response is the *virtual channel*. A physical link is divided logically into several independent channels, each with its own buffering and state. Each packet is assigned a virtual-channel identifier, and the identifier travels with the flits so the receiver knows which logical stream each flit belongs to.

Conceptually, each virtual channel has states such as *idle*, *waiting for resources*, and *active*. If one packet is blocked because its next-hop destination is unavailable, another packet using a different virtual channel on the same physical link can still move. This breaks much of the head-of-line blocking that would occur in a single-channel wormhole design.

Virtual channels therefore make two important improvements at once. First, they allow flits from different packets to interleave safely on one physical link. Second, they preserve forward progress for traffic that is unrelated to the current blockage. The price is complexity: the router must manage multiple buffers, virtual-channel allocation, and additional arbitration logic.

10.8 Switch Architecture

A modern switch or router is not just a crossbar. It is a small pipeline of control and data-movement stages. A typical design includes:

- input buffers, often organized per virtual channel,
- routing logic that determines the requested output direction,
- a virtual-channel allocator,
- a switch allocator or arbiter that decides which requests win a contested output,
- and the crossbar traversal itself.

This structure is analogous in spirit to a processor pipeline: several flits can be in different internal stages of the router at the same time. That pipelining is essential for throughput. At the same time, the arbitration policy matters. A useful intuition is the traffic-circle rule: when deciding whether to admit new traffic or let already-moving traffic continue, it is often wise to prioritize traffic that is already transiting through the switch over traffic that is merely trying to enter. That policy is not universal, but it captures a recurring design instinct in congestion management.

The large lesson is that topology alone does not determine performance. Real performance emerges from the interaction of topology, routing, buffering, virtual-channel structure, and router arbitration. A network with an impressive theoretical bisection bandwidth can still perform poorly if its switches create avoidable blocking or if its routing policy concentrates load onto a few hot links.

10.9 Chapter 10 Summary

Chapter 10 develops the communication fabric that ties a multiprocessor together. Parallel interconnection networks differ from general communication networks because they move wide transfers within one tightly engineered machine and therefore focus on topology, routing, flow control, and router behavior rather than on arbitrary end-to-end recovery. Direct topologies connect processors through explicit neighbor links, while indirect topologies place a switching fabric between processors. Among direct networks, the hypercube is the classic logarithmic-diameter example, while the broader

k-ary n-dimensional family captures rings, meshes, and tori; a torus gains wraparound links and therefore higher bisection bandwidth than the corresponding mesh. Physical realizations must respect planar wire layout, so logical beauty alone is not enough. Among indirect networks, the crossbar is flexible but expensive, while multi-stage designs such as the omega network reduce cost by composing many small switches with perfect-shuffle wiring and destination-bit routing, at the price of blocking. Routing policies may be deterministic, adaptive, or oblivious, each with different tradeoffs in simplicity, load balance, and correctness. Flow control then decides what happens when the desired next hop is unavailable. Store-and-forward, cut-through, wormhole, and virtual-channel designs form a progression from simplicity to high throughput, with virtual channels providing the modern answer to much of wormhole's head-of-line blocking. A practical network is therefore a combined design problem involving topology, routing, buffering, and switch microarchitecture.

10.10 Practice Questions

The following questions reinforce the main ideas of Chapter 10.

1. What distinguishes a parallel interconnection network from a general communication network? Why does flit width matter in the former?
2. Compare direct and indirect topologies. In which one are the processors themselves the vertices of the communication graph?
3. A hypercube has 64 nodes. What is its dimensionality, and what is the degree of each node?
4. What does the phrase *k-ary n-dimensional* mean? How many nodes are in a 4-ary 3-dimensional direct network?
5. What is the difference between a mesh and a torus? Why does the torus usually have higher bisection bandwidth?
6. An 8-node unidirectional ring has 1-cycle links and no contention. How many cycles does it take a message to travel from node 0 to node 5? If node 5 can consume at most one message per cycle and already has one older message queued on arrival, what extra service delay is introduced?
7. Why can a logically simple ring become physically problematic when laid out on a planar chip or board?
8. Define bisection bandwidth. A 4-ary 2-mesh and a 4-ary 2-torus use the same per-link bandwidth. Why does the torus have twice the bisection bandwidth?
9. For a regular square mesh or torus, why is a dimension-aligned equal cut a convenient way to find bisection bandwidth? Does the definition exclude other equal-halves partitions?
10. Compare a full crossbar with a multi-stage indirect network. Why might a designer choose the latter even though it can block?
11. What is a perfect shuffle, and why is it associated with omega networks?
12. An omega network has 16 endpoints. How many stages are required, and how many destination bits are consumed along one route?
13. Compare deterministic, adaptive, and oblivious routing. What problem is Valiant-style routing trying to solve?
14. What is the difference between a packet and a flit?

15. What are three possible responses when the desired downstream buffer or output is full?
16. Compare store-and-forward, cut-through, wormhole, and virtual-channel flow control. Which one most directly addresses head-of-line blocking?
17. Why does virtual-channel flow control make the router more complicated?

Appendix A

Pipeline Timing-Diagram Interpretation Checklist

A five-stage timing diagram is best read as a checklist rather than as raw symbols. Move cycle by cycle (column by column), then apply the prompts below.

A.1 How to Use This Checklist

Quick Reading Order

1. Mark repeated stage labels (stall behavior).
2. Mark killed instructions (squashes, often shown as dashes).
3. Identify the event that caused each stall/squash (dependence, branch, resource conflict, or memory latency).
4. Compute measured CPI from the traced window.

1. Structural: Resource Organization

1.1 IF and MEM overlap

- **Ask:** Can instruction fetch occur in the same cycle as a load/store data access?
- **Cue:** One cycle shows IF for one instruction while another instruction is in MEM with a real data access.
- **Meaning:** Sustained overlap indicates concurrent instruction/data paths. Repeated IF stalls when load/store reaches MEM suggests a shared bottleneck.

1.2 Single-issue vs superscalar

- **Ask:** Is the machine *single-issue* or *superscalar* (*multi-issue*)?
- **Cue:** In simple single-issue diagrams, each stage has at most one instruction per cycle.
- **Meaning:** Two instructions shown in one stage in one cycle (for example two EX uses) implies multi-issue and/or multiple functional units.

1.3 Multi-cycle execute units

- **Ask:** Do some operations occupy EX for multiple cycles?
- **Cue:** One instruction appears in EX across consecutive cycles (EX, EX, EX, . . .).
- **Meaning:** That operation is using a multi-cycle execute unit and can back up younger instructions.

2. Data: Dependence and Forwarding

2.1 RAW hazard location and cost

- **Ask:** Where are *RAW* hazards, and how many bubbles do they add?
- **Cue:** Consumer instruction repeats a stage (usually ID, sometimes IF) while waiting for producer timing.
- **Meaning:** Number of repeated cells is the local dependence penalty.

2.2 Forwarding strength

- **Ask:** Is forwarding present, and from where?
- **Cue:** Back-to-back ALU dependencies proceed without stalling even though producer WB occurs later.
- **Meaning:** This usually implies EX/MEM→EX and MEM/WB→EX forwarding. If simple ALU→ALU pairs stall, forwarding is limited.

2.3 Load-use behavior

- **Ask:** What is the load-use penalty?
- **Cue:** Load immediately followed by consumer, where consumer stalls one cycle.
- **Meaning:** Classic 5-stage with forwarding still needs a one-cycle load-use stall because load data appears at end of MEM.

2.4 WB/ID same-cycle timing

- **Ask:** Can ID read a register in the same cycle an older instruction writes it during the WB stage?
- **Cue:** Consumer ID aligns with producer WB, with no extra bubble.
- **Meaning:** Implies write-before-read register timing and/or WB→ID bypass behavior.

3. Control: Branch and Redirection Policy

3.1 Resolution stage and penalty

- **Ask:** Where is branch outcome resolved, and what is the visible penalty?
- **Cue:** Find where target/direction becomes known; then count wrong-path squashes or inserted bubbles before correct-path flow stabilizes.
- **Meaning:** Later resolution generally increases misfetch cost.

3.2 Delay slot vs squash slot

- **Ask:** Does the post-branch slot always execute, or can it be killed?
- **Cue:** Slot instruction either always completes (*delay slot*) or is conditionally killed (*squash slot*/branch-likely style).
- **Meaning:** Always-executed slot indicates delay-slot semantics; conditional kill indicates squash-style semantics.

3.3 Taken/not-taken inference (branch-likely style)

- **Ask:** Which branch outcome occurred?
- **Cue:** Whether the slot instruction executes or is squashed.
- **Meaning:** In this text's branch-likely examples, a squashed slot corresponds to not taken.

3.4 Prediction vs no prediction

- **Ask:** Does fetch speculate, and is speculation often correct?
- **Cue:** Continued fetching past unresolved branches followed by occasional squashes (speculation), versus front-end waiting for resolution (no speculation).
- **Meaning:** Wrong-path fetches later killed imply predictive/fixed speculation; repeated front-end pauses imply non-speculative handling.

4. Memory: Miss Effects in the Timeline

4.1 Miss penalty visibility

- **Ask:** Are long memory latencies visible?
- **Cue:** Load/store occupies MEM for multiple cycles (M, M, M, ...), with younger instructions backing up.
- **Meaning:** Indicates cache-miss (or similar long-latency memory) behavior.

4.2 Blocking vs non-blocking cache behavior

- **Ask:** Is cache behavior *blocking*, or can it do *hit-under-miss* (*non-blocking*)?
- **Cue:** Either all younger work freezes behind a miss, or independent work continues.
- **Meaning:** Full freeze implies blocking behavior (common in simple in-order models). Continued independent progress suggests non-blocking support and/or more advanced scheduling.

5. Performance from the Diagram

5.1 Measured CPI of a trace window If a window spans C cycles and retires N_{ret} non-squashed instructions (Eq. (A.1)):

$$\mathrm{CPI}_{measured} = \frac{C}{N_{ret}}. \tag{A.1}$$

Squashed instructions consume cycles but do not count as retired instructions.

5.2 Stall-cycle attribution by cause Assign repeated-stage cells and squashes to causes:

- dependence waiting (RAW/load-use),
- control redirection (branch squashes/bubbles),
- structural conflicts (resource contention),
- memory-latency extension (repeated MEM).

This gives a practical CPI breakdown from one timing chart.

6. Common Gotchas

- ALU instructions may still appear in MEM in pedagogical 5-stage diagrams; this alone does not prove a data-memory access.
- IF stalls can be secondary backpressure from a downstream stall (often ID), not a fetch-unit limitation by itself.
- Squash and stall are different: squashed instructions are killed; stalled instructions are preserved in place with repeated stage labels.

Appendix B

Formula Compendium (Grouped and Labeled)

This appendix gathers the core formulas used across Chapters 1–10.

Chapter 1 Formula Set

Chapter 1-F1 (Vector Serial vs Pipelined Cycles)

$$\text{serial cycles} = nk, \qquad \text{pipeline cycles} \approx k + n - 1. \tag{B.1}$$

Chapter 2 Formula Set

Chapter 2-F1 (CPU Time Decomposition)

$$\text{CPU time} = \text{IC} \times \text{CPI} \times \text{CT}. \tag{B.2}$$

Chapter 2-F2 (Frequency Form)

$$\text{CPU time} = \frac{\text{IC} \times \text{CPI}}{f}. \tag{B.3}$$

Chapter 2-F3 (Instruction-Mix Total Cycles)

$$\text{Total cycles} = \sum_i \left(\text{IC}_i \times \text{CPI}_i\right). \tag{B.4}$$

Chapter 2-F4 (Average CPI)

$$\text{CPI}_{avg} = \frac{\text{Total cycles}}{\text{Total instructions}}. \tag{B.5}$$

Chapter 2-F5 (Speedup Definition)

$$S = \frac{T_{old}}{T_{new}}. \tag{B.6}$$

Chapter 2-F6 (Amdahl's Law)

$$S_{Amdahl} = \frac{1}{(1-f) + \frac{f}{s}}. \tag{B.7}$$

Chapter 2-F7 (Amdahl Upper Bound)

$$S_{\text{upper}} = \frac{1}{1-f}. \tag{B.8}$$

Chapter 2-F8 (Gustafson's Law)

$$S_{Gustafson} = \alpha + (1 - \alpha)N. \tag{B.9}$$

Chapter 2-F9 (Arithmetic Mean for Time)

$$\bar{T}_{arith} = \frac{1}{n} \sum_{i=1}^{n} T_i. \tag{B.10}$$

Chapter 2-F10 (Harmonic Mean for Rate)

$$\bar{R}_{harm} = \frac{n}{\sum_{i=1}^{n} \frac{1}{R_i}}. \tag{B.11}$$

Chapter 3 Formula Set

Chapter 3-F1 ((C, B, S) Definitions)

$$\text{capacity} = 2^C \text{ bytes}, \qquad \text{block size} = 2^B \text{ bytes}, \qquad \text{associativity} = 2^S \text{ ways}. \tag{B.12}$$

Chapter 3-F2 (Total Blocks and Sets)

$$\text{blocks} = 2^{C-B}, \qquad \text{sets} = \frac{2^C}{2^B \cdot 2^S} = 2^{C-B-S}. \tag{B.13}$$

$$\text{lines} = \text{sets} \cdot \text{ways} = 2^{C-B-S} \cdot 2^S = 2^{C-B}. \tag{B.14}$$

Direct-mapped corollary: if $S = 0$, then ways $= 1$ and lines $=$ sets. Set-associative corollary: if $S > 0$, then lines $>$ sets.

Chapter 3-F3 (Address-Field Widths, General AW)

$$|\text{Offset}| = B, \qquad |\text{Index}| = C - B - S, \qquad |\text{Tag}| = AW - (C - S). \tag{B.15}$$

$$\text{Address} = [\text{Tag} \mid \text{Index} \mid \text{Offset}]. \tag{B.16}$$

Chapter 3-F4 (Address-Field Widths, 32-bit Address)

$$|\text{Offset}| = B, \qquad |\text{Index}| = C - B - S, \qquad |\text{Tag}| = 32 - (C - S) = 32 - C + S. \tag{B.17}$$

Chapter 3-F5 (Way Selection Relationship)

$$\text{ways} = 2^S, \qquad \log_2(\text{ways}) = S. \tag{B.18}$$

Chapter 3-F6 (Single-Level Average Access Time)

$$AAT = HT + MR \cdot MP. \tag{B.19}$$

Chapter 3-F7 (Bit Overhead from Valid/Dirty)

$$\text{overhead fraction} = \frac{n_v + n_d}{2^{B+3}}, \qquad n_v = 1, \quad n_d \in \{0, 1\}. \tag{B.20}$$

Chapter 3-F8 (Full-Block Miss Penalty Model)

$$MP_{full} = T_1 + (N_{sb} - 1)T_f, \qquad N_{sb} = 2^B/W. \tag{B.21}$$

Chapter 3-F9 (Early-Restart Approximation)

$$MP_{ER} \approx T_1 + \frac{N_{sb} - 1}{2} T_f. \tag{B.22}$$

Chapter 3-F10 (Recursive Two-Level Form)

$$AAT_{L1} = HT_{L1} + MR_{L1} \cdot AAT_{L2}. \tag{B.23}$$

Chapter 3-F11 (VIPT Constraint)

$$C - S \leq P. \tag{B.24}$$

Chapter 4 Formula Set

Chapter 4-F1 (Pipelined Clock Approximation)

$$T_{clk,pipe} \approx T_s + T_{reg}. \tag{B.25}$$

Chapter 4-F2 (Ideal Long-Run Pipeline Speedup)

$$S_{ideal} \approx \frac{NT_s}{T_s + T_{reg}}. \tag{B.26}$$

Chapter 4-F3 (Finite-Length Pipelined Time)

$$T_{pipe} \approx (N + M - 1)T_{clk,pipe}. \tag{B.27}$$

Chapter 4-F4 (Unpipelined Time for M Instructions)

$$T_{unpiped} \approx MNT_s. \tag{B.28}$$

Chapter 4-F5 (Practical CPI Model)

$$CPI = 1 + \text{stalls}_{avg}. \tag{B.29}$$

Chapter 4-F6 (Branch-Stall CPI Contribution)

$$CPI \approx 1 + f_{branch} \cdot \text{stall}_{branch,avg}. \tag{B.30}$$

Chapter 5 and Simulation Formula Set

Chapter 5-F1 (Two-Level AAT with Explicit L1 Penalty)

$$AAT_{L1} = HT_{L1} + MR_{L1} \cdot MP_{L1}, \qquad MP_{L1} = AAT_{L2}. \tag{B.31}$$

Chapter 5-F2 (L2 AAT Model)

$$AAT_{L2} = HT_{L2} + MR_{L2,read} \cdot DRAM_TIME. \tag{B.32}$$

Chapter 5-F3 (DRAM Block Service Model)

$$DRAM_TIME = DRAM_AT + DRAM_AT_PER_WORD \cdot WORDS_PER_BLOCK. \tag{B.33}$$

Chapter 6 Formula Set (Dynamic ILP, Superscalar, and Out-of-Order Execution)

Chapter 6-F1 (IPC and CPI Relationship)

$$IPC = \frac{1}{CPI}. \tag{B.34}$$

Chapter 6-F2 (CPI/IPC Threshold Equivalence)

$$CPI < 1 \iff IPC > 1. \tag{B.35}$$

Chapter 6-F3 (Retire-Throughput Bound)

$$IPC_{retire} \leq \min(IPC_{fetch},\ IPC_{dispatch},\ IPC_{execute},\ IPC_{state\ update})\,. \tag{B.36}$$

Chapter 6-F4 (Simplified Scoreboard Fire Condition)

$$\text{Fire if } \neg busy(src_1) \wedge \neg busy(src_2) \wedge \neg busy(dest) \wedge \neg busy(FU_{needed}). \tag{B.37}$$

Chapter 6-F5 (Result-Bus Completion Bound)

$$IPC_{complete} \leq N_{result\ buses}. \tag{B.38}$$

Chapter 6-F6 (Physical Register File Port Estimate)

$$N_{read\ ports} \approx 2N_{sched}, \qquad N_{write\ ports} \approx N_{result\ buses}. \tag{B.39}$$

Chapter 6-F7 (Precise Exception Property)

$$\begin{aligned} &\text{precise exception at } i \Rightarrow \text{all instructions older than } i \text{ are committed} \\ &\qquad\qquad \wedge\ \text{no instruction younger than } i \text{ is architecturally visible} \end{aligned} \tag{B.40}$$

Chapter 6-F8 (RAT Rename Mapping)

$$P_{src1} = RAT[A_{src1}], \qquad P_{src2} = RAT[A_{src2}], \qquad RAT[A_{dest}] \leftarrow P_{free}. \tag{B.41}$$

Chapter 6-F9 (Ready-Based Fire Condition)

$$fire_i \Leftarrow ready(P_{src1}) \wedge ready(P_{src2}) \wedge free(FU_i). \tag{B.42}$$

Chapter 6-F10 (ROB Retirement Condition)

$$retire \Leftarrow ready(ROB_{head}) \wedge \neg exception(ROB_{head}). \tag{B.43}$$

Chapter 6-F11 (ROB Exception Rule)

$$\begin{aligned} exception(ROB_{head}) \Rightarrow\ & \text{flush younger instructions} \\ & \wedge\ \text{transfer control to the retry or handler PC} \end{aligned} \tag{B.44}$$

Chapter 7 Formula Set (VLIW and Software Pipelining)

Chapter 7-F1 (Dependence Height)

$$height(i) = \begin{cases} lat(i), & succ(i) = \emptyset \\ lat(i) + \max\limits_{j \in succ(i)} height(j), & \text{otherwise} \end{cases} \tag{B.45}$$

Chapter 7-F2 (Earliest Legal List-Scheduling Cycle)

$$cycle(i) = \min\{c \mid \forall j \in pred(i),\ c \geq cycle(j) + lat(j),\ \text{and a compatible FU is free in } c\}. \tag{B.46}$$

Chapter 7-F3 (Resource-Constrained Initiation Interval)

$$ResII = \max_t \left\lceil \frac{N_t}{U_t} \right\rceil \tag{B.47}$$

Chapter 7-F4 (Recurrence-Constrained Initiation Interval)

$$RecII = \max_c \left\lceil \frac{Lat(c)}{Dist(c)} \right\rceil \tag{B.48}$$

Chapter 7-F5 (Minimum Initiation Interval Bound)

$$MII = \max(ResII, RecII), \qquad II \geq MII. \tag{B.49}$$

Chapter 8 Formula Set (Multiprocessors and Coherence)

Chapter 8-F1 (Ideal Embarrassingly Parallel Scaling)

$$S(N) = N \qquad \text{when } f = 1 \text{ and no shared resource becomes the bottleneck.} \tag{B.50}$$

Chapter 8-F2 (Directory Entry Count)

$$N_{entries} = \frac{2^m}{2^v} = 2^{m-v}, \tag{B.51}$$

where memory has 2^m bytes and the coherence block size is 2^v bytes.

Chapter 8-F3 (Full Bit-Vector Directory Storage Estimate)

$$bits_{dir} \approx 2^{m-v}\left(N + n_{ctrl}\right), \tag{B.52}$$

where N is the number of PEs and n_{ctrl} represents the small set of non-sharer control bits such as dirty, valid, or overflow indicators.

Chapter 8-F4 (Limited K-Pointer Directory Storage Estimate)

$$bits_{dir} \approx 2^{m-v}\left(K\left\lceil \log_2 N \right\rceil + n_{ctrl}\right). \tag{B.53}$$

Chapter 9 Formula Set (Synchronization and Memory Consistency)

Chapter 9-F1 (Fetch-and-Op Semantics)

$$\texttt{fetch_and_op}(x, c): \qquad old = x, \qquad x \leftarrow x \ op\ c, \qquad \text{return } old. \tag{B.54}$$

Chapter 9-F2 (Dynamic Work Claim via Fetch-and-Add)

$$i = \texttt{fetch_and_add}(next, 1), \qquad \text{execute iteration } i \text{ iff } i < N. \tag{B.55}$$

Chapter 9-F3 (Last Arrival in a Counter-Based Barrier)

$$\text{last arriver if } \texttt{fetch_and_add}(count, 1) = N - 1. \tag{B.56}$$

Chapter 10 Formula Set (Parallel Interconnection Networks)

Chapter 10-F1 (K-ary N-Dimensional Node Count)

$$N_{nodes} = k^n. \tag{B.57}$$

Chapter 10-F2 (Hypercube Size and Degree)

$$N_{nodes} = 2^d, \qquad degree = d. \tag{B.58}$$

Chapter 10-F3 (Bisection Bandwidth for Uniform Links)

$$B_{bisect} = N_{cut} \times W_{link}. \tag{B.59}$$

Chapter 10-F4 (Omega Network Stage Count)

$$stages = \log_2 N. \tag{B.60}$$

Appendix C

Glossary of Core Definitions

This appendix collects first-pass definitions used throughout Chapters 1–10 and the advanced-topics appendix.

Parallelism Doing multiple useful operations concurrently to increase throughput.

Speculation Executing work before all required conditions are confirmed, with recovery if assumptions are wrong.

Prediction Estimating uncertain future behavior to guide speculative execution.

Instruction-Level Parallelism (ILP) Overlap of independent instructions within one thread.

Very Long Instruction Word (VLIW) Instruction encoding style that packs several compiler-scheduled ops into one instruction word.

Polycyclic Scheduling Early name for overlapping multiple loop iterations in one compact repeating schedule.

Software Pipelining Loop-scheduling technique that overlaps operations from different iterations and emits a prologue, steady-state kernel, and epilogue.

Thread-Level Parallelism (TLP) Concurrent execution of multiple threads on one or more cores.

Processing Element (PE) One independently executing processor, core, or other execution agent in a multiprocessor.

Digital Signal Processor (DSP) Processor class optimized for regular numeric workloads such as filtering, radio, and image-processing kernels.

SIMD One instruction stream operating on multiple data elements in lockstep.

MIMD Multiple instruction streams operating on multiple data streams independently.

Shared Memory Multiprocessor organization in which all participating processors conceptually operate in one common address space.

Message Passing Multiprocessor organization in which processors communicate by explicitly sending data between private memories.

Partitioned Global Address Space (PGAS) Memory model with a global address space in which some regions are much more expensive to access than others.

Accelerator Specialized processing engine, often attached to a host CPU through an external link and using distinct memory.

Bus Shared observable broadcast medium across which all attached agents can see the same transactions.

Point-to-Point Interconnect Link organization in which packets move between specific endpoints rather than across one shared observable medium.

Cache Coherence Property that keeps multiple cached copies of the same location mutually consistent enough to preserve shared-memory semantics.

Snooping Coherence Coherence method in which each cache monitors shared interconnect traffic and reacts when another processor requests a block it holds.

Directory-Based Coherence Coherence method in which a directory tracks which caches currently hold a block and coordinates invalidations or forwarding.

MSI Coherence protocol whose stable states are modified, shared, and invalid.

MESI Coherence protocol whose stable states are modified, exclusive, shared, and invalid.

MOESI Coherence protocol whose stable states are modified, owned, exclusive, shared, and invalid.

MOESIF Coherence protocol whose stable states are modified, owned, exclusive, shared, invalid, and forward.

GetS Coherence request for a shared readable copy of a block.

GetM Coherence request for exclusive writable ownership of a block.

WriteBack Propagation of a dirty cache block to lower memory.

Exclusive (E) Clean state in which exactly one cache holds the block and may later write it without first broadcasting an upgrade request.

Owned (O) Dirty shared state in which one cache remains responsible for the current value and may respond to later requests even though other caches hold clean shared copies.

Forward (F) Clean shared state designating one cache as the responder for later shared requests.

Hit Line Shared indication line used to tell a requester that some other cache already holds the block, thereby distinguishing `E` from `S` in a snooping protocol.

Intervene Snooper action in which a cache supplies data directly in response to another processor's request.

Abort Snooper response instructing a requester to wait or retry because ownership transfer is not yet complete.

Transient State Intermediate coherence state used while a block is waiting for a request or ownership transfer to complete.

Presence Bit Vector Directory field that records which caches or processors currently hold a copy of one block.

Limited Directory Directory organization that records only a bounded number of sharers explicitly and uses an overflow policy when that bound is exceeded.

Recall Directory command requiring a cache to return the current block value and surrender ownership.

Invalidate Directory command requiring a cache to discard its current copy of a block.

Ack Directory-style acknowledgment indicating that a required coherence action completed.

Nack Directory-style negative acknowledgment indicating that a request cannot yet be completed and should be retried later.

Sharing Pattern Characteristic way in which a block is accessed across processors, such as private, read-only, producer-consumer, migratory, or read-write shared.

False Sharing Performance problem in which unrelated variables placed in the same cache block

force unnecessary invalidations or transfers because one of them is actively shared.

Coherence Miss Miss caused by coherence actions such as invalidation or ownership transfer rather than by compulsory, capacity, or conflict effects alone.

Inclusive Cache Hierarchy Hierarchy in which data present in an upper level is guaranteed to also be present in the next lower level.

Exclusive Cache Hierarchy Hierarchy in which adjacent levels are intended not to hold duplicate copies of the same block.

Non-Exclusive Cache Hierarchy Hierarchy that allows overlap between adjacent levels without requiring it for every block.

Critical Section Region of code that must not be executed concurrently by multiple threads if shared data is to remain correct.

Lock Synchronization primitive that grants exclusive access to a critical section.

Mutex Mutual-exclusion lock allowing at most one thread into a protected critical section at a time.

Barrier Synchronization point that forces all participating threads to wait until the last one arrives.

Spin Lock Lock implementation in which a waiting thread repeatedly polls a shared variable instead of blocking.

Atomic Read-Modify-Write Indivisible memory operation that reads a location, transforms it, and writes the new value with no visible intermediate state.

Test-and-Set Atomic read-modify-write primitive that writes a 1 and returns the old value.

Test-and-Test-and-Set Spin-lock strategy that first polls with ordinary reads and attempts an atomic acquisition only when the lock appears free.

Load-Link / Store-Conditional (LL/SC) Atomic primitive pair in which a load establishes a reservation and a later store succeeds only if no conflicting coherence event invalidated that reservation.

Fetch-and-Op Atomic primitive that returns the old value of a location while applying some operation to it.

Fetch-and-Add Fetch-and-op specialization in which the atomic update is addition.

Tree Lock Lock organization that reduces contention by arranging locks hierarchically so contenders meet in stages.

Tournament Lock Another name for a hierarchical tree lock in which contenders advance through multiple levels.

Sense-Reversing Barrier Barrier that uses a phase bit so one barrier instance is not confused with the next.

OpenMP Standard shared-memory parallel-programming interface based largely on compiler directives and runtime support.

MPI Message Passing Interface; standard API for distributed-memory or message-passing parallel programming.

Transactional Memory Synchronization model in which a critical section executes optimistically

and commits only if no conflicting access occurred.

Hardware Transactional Memory Transactional-memory implementation that uses hardware structures such as caches to track speculative read and write sets.

Software Transactional Memory Transactional-memory implementation that tracks conflicts and retries through software metadata rather than specialized hardware.

Memory Consistency Rules governing the visibility and ordering of memory operations to different addresses across multiple processors.

Strict Consistency Memory model in which all processors observe memory operations in one global order that matches actual time.

Sequential Consistency Strong memory model in which execution appears as one total interleaving that preserves each processor's program order.

Weak Consistency Family of relaxed models that provide strong ordering mainly at explicit synchronization operations rather than at every ordinary load and store.

Acquire Synchronization action that prevents later memory accesses from moving before it.

Release Synchronization action that prevents earlier memory accesses from moving after it.

Store Fence Ordering operation that ensures earlier stores become visible before later memory actions proceed.

Load Fence Ordering operation that prevents later loads from bypassing earlier outstanding memory actions.

Memory Fence Explicit ordering operation that constrains permitted memory reordering around a program point.

Flit Flow-control digit; the unit of transfer controlled by the network's flow-control mechanism.

K-ary N-Dimensional Topology Direct-network family with n dimensions and k nodes in each dimension, for a total of k^n nodes.

Direct Topology Network topology in which processors or nodes are connected directly to selected neighboring nodes.

Indirect Topology Network topology in which processors communicate through a separate switching fabric.

Hypercube Direct topology with 2^d nodes in which each node has degree d and neighbors differ in one coordinate.

Mesh Direct topology in which nodes connect to neighbors along one or more dimensions without wraparound edge links.

Torus Direct topology in which each dimension wraps around so the network forms one ring per dimension.

Gray Code Binary numbering scheme in which adjacent code words differ in exactly one bit.

Bisection Bandwidth Minimum aggregate link bandwidth crossing any equal-halves cut of a network.

Crossbar Indirect switching fabric that can connect any input to any output.

Multi-Stage Interconnect Indirect network built from several stages of small switches rather than one full crossbar.

Perfect Shuffle Fixed wiring permutation commonly used between stages of an omega network.

Omega Network Multi-stage interconnect in which one destination bit guides the routing choice at each stage.

Routing Process of choosing the path a flit or packet takes through the interconnect.

Deterministic Routing Routing policy that always chooses the same path for a given source and destination.

Dimension-Order Routing Deterministic routing policy that traverses dimensions in a fixed order such as x before y.

Adaptive Routing Routing policy that chooses among multiple next hops based on current congestion information.

Oblivious Routing Non-adaptive routing policy that ignores current load but may still use randomized path selection for balance.

Valiant's Algorithm Oblivious routing method that sends traffic first to a randomly chosen intermediate node and then to the final destination.

Flow Control Rules determining when data may advance through the network and what happens when downstream resources are full.

Circuit Switching Communication method that reserves an entire route before data transmission begins.

Packet Switching Communication method in which independently routed packets carry their own destination information.

Packet Message unit that carries routing information and may be divided into multiple flits for transport.

Back Pressure Upstream throttling mechanism used to prevent downstream buffers from overflowing.

Credit-Based Flow Control Back-pressure scheme in which a sender tracks how many downstream buffer slots remain available.

Store-and-Forward Flow-control method that buffers the entire packet at each hop before transmitting it onward.

Cut-Through Routing Packet-forwarding method that begins sending to the next hop as soon as enough of the header has arrived to determine the route.

Wormhole Flow Control Flit-level forwarding method in which one packet may stretch across several routers and links at once.

Head-of-Line Blocking Throughput loss caused when one blocked packet prevents unrelated traffic behind it from advancing.

Virtual Channel Logical subdivision of one physical link into multiple independently buffered and arbitrated channels.

Router Interconnect component that examines incoming traffic, chooses outputs, and manages buffering and arbitration.

Data Race Incorrect unsynchronized overlap of conflicting accesses to shared state.

Embarrassingly Parallel Workload whose useful work is effectively all parallelizable and therefore

scales nearly ideally if resources permit.

Pipelining Temporal parallelism where different stages process different instructions simultaneously.

Superscalar (Multi-Issue) Pipeline organization that can issue more than one instruction per cycle.

Register Renaming Introduction of additional physical register identities so later writes do not falsely block earlier reads or writes.

Op One encoded operation within a VLIW instruction stream.

Multi-Op Group of ops intended to execute in the same cycle in a VLIW design.

Tail Bit Per-op field marking whether the current op ends the current multi-op.

Pause Field Encoded delay field that inserts a specified number of idle cycles before the next multi-op begins.

Iterative Modulo Scheduling (IMS) Practical software-pipelining algorithm that searches for a legal modulo schedule by trying increasing initiation intervals.

Initiation Interval (II) Number of cycles between the start of successive loop iterations in a software-pipelined schedule.

Minimum Initiation Interval (MII) Lower bound on II, equal to $\max(ResII, RecII)$.

Resource-Constrained Initiation Interval (ResII) Lower bound on II imposed by the number of available functional units of each type.

Recurrence-Constrained Initiation Interval (RecII) Lower bound on II imposed by loop-carried dependence cycles.

Prologue Non-repeating prefix of a software-pipelined loop that fills the pipeline before steady state.

Kernel Repeating steady-state portion of a software-pipelined loop.

Epilogue Non-repeating suffix of a software-pipelined loop that drains remaining work after loop exit.

Point Latency Model Scheduling model that treats an op as occupying one issue point and becoming ready after a fixed latency.

Tomasulo's Algorithm Out-of-order scheduling method that combines renaming, reservation stations, and result broadcast to wake dependent instructions when operands become ready.

Reservation Station Scheduling entry that holds an instruction, its operand state, and dependency tags until it can execute.

Common Data Bus (CDB) Shared result-broadcast path used to distribute a completed value and its producer tag to waiting instructions.

Result Bus Generalized form of the CDB; modern designs often use more than one such broadcast path.

Architected Register Register name defined by the ISA and visible to software.

Architected Register File Register file containing the committed architectural state visible to software.

Physical Register Internal hardware register used to hold renamed values beyond the architected register set.

Register Alias Table (RAT) Mapping table from architected register names to the current physical register locations.

Ready Bit Status bit indicating that a register or operand value has already been produced and may be consumed.

Reorder Buffer (ROB) In-order retirement structure that records speculative results and enables precise exceptions.

Future File Register organization that holds the most recent speculative values while a separate architected register file holds committed state.

Messy Register File Informal name for the current speculative register state that may contain out-of-order completions not yet reflected in a coherent sequential state.

Precise Exception Exception model in which all older instructions appear committed and all younger instructions appear not to have changed architectural state.

Imprecise Exception Exception model that reports an error without guaranteeing an exact sequential recovery point.

Checkpoint/Repair Recovery method that periodically captures coherent register state and restores it when an exception or mis-speculation occurs.

Instruction Boundary Dynamic marker used to denote a coherence point or recovery boundary in checkpoint-based schemes.

Hyperthreading (SMT) Simultaneous multithreading in which multiple threads share one core's execution resources.

Speculative Side Channel Information leak caused when squashed speculative work still changes microarchitectural state such as caches or predictors.

Runtime Time to complete one job (response/turnaround time).

Throughput Completed work per unit time.

Amdahl's Law Speedup limit for fixed-size workloads when only a fraction is accelerated.

Gustafson's Law Scaled-speedup model where workload size grows with processor count.

Average Access Time (AAT) Expected memory-access latency, typically modeled as hit time plus miss rate times miss penalty.

SRAM Fast, low-latency, higher-area memory used for on-chip caches.

DRAM Dense, lower-cost, higher-latency memory used for main memory capacity.

Hit Time (HT) Latency to serve an access from the current cache level.

Miss Ratio (MR) Fraction of accesses at a level that miss.

Miss Penalty (MP) Extra latency to service a miss from lower levels.

Compulsory Miss First-reference miss for data not previously loaded.

Capacity Miss Miss caused by working set exceeding effective cache capacity.

Conflict Miss Miss caused by mapping/placement constraints despite sufficient total capacity.

Write-Back Write policy where modified data is propagated on eviction.

Write-Through Write policy where lower level is updated on each write hit.

Write-Allocate Write-miss policy that fetches and installs the block before write.

Write-No-Allocate Write-miss policy that bypasses install and writes lower level directly.

Qubit Quantum bit whose useful state must be isolated and controlled well enough to preserve quantum behavior during computation.

Decoherence Loss of intended quantum behavior when a qubit interacts with the surrounding environment.

NISQ Noisy intermediate-scale quantum computing; near-term machines with limited qubits, noise, and short usable programs.

Logical Qubit Error-corrected qubit represented by many physical qubits and maintained through repeated decoding.

Phase Oracle Reversible quantum operation that encodes a classical function by changing the phase of selected basis states.

Fault-Tolerant Quantum Computing Quantum-computing approach that uses encoded logical qubits and error correction to run longer computations reliably.

Superconducting Logic Logic family that uses superconducting devices and very low-resistance wires, often representing information with pulses.

Rapid Single Flux Quantum (RSFQ) Superconducting pulse-logic family that represents bits using quantized magnetic-flux pulses.

Reciprocal Quantum Logic (RQL) AC-powered superconducting logic family related to RSFQ and used as one candidate technology for energy-efficient processors.

Operand Queue Storage structure that holds produced values until compiler-scheduled consumer instructions need them.

Barrel Processor Multithreaded processor that issues from different resident threads in rotation to keep a deep pipeline occupied.

Cryostat Cooling system used to maintain cryogenic temperatures for superconducting or quantum hardware.

Energy-Delay Product (EDP) Efficiency metric combining energy and runtime; lower EDP indicates a better energy/performance tradeoff.

SUP Superconducting Unit of Performance; proposed organization combining latency-oriented and throughput-oriented superconducting resources.

PIPT Physically indexed, physically tagged cache organization in which TLB translation typically precedes L1 indexing/tag lookup.

VIPT Virtually indexed, physically tagged cache organization that overlaps indexing with translation.

TLB Translation lookaside buffer storing recent virtual-to-physical translations.

Structural Hazard Stall risk from insufficient hardware resources.

Data Hazard Stall risk from operand read/write ordering conflicts.

Control Hazard Stall risk from unresolved branch/jump control flow.

RAW Read-after-write dependence hazard.

WAR Write-after-read dependence hazard.

WAW Write-after-write dependence hazard.

Data Dependence Graph Graph representation of instruction ordering constraints induced by true and false dependences.

Dependence Height Priority heuristic measuring the remaining latency on the longest dependent path below an instruction.

List Scheduling Heuristic compiler scheduling method that repeatedly places the highest-priority ready instructions into the earliest legal cycle.

Memory Disambiguation Compile-time or runtime reasoning used to determine whether two memory references may access the same location.

Forwarding (Bypassing) Sending a produced value directly to a consumer before register write-back.

Branch Target Buffer (BTB) Predictor structure that supplies likely branch targets early in fetch.

Aliasing Predictor or cache entry interference from unrelated addresses mapping to the same slot.

Return-Address Stack (RAS) Small stack predictor for call/return target addresses.

Direction Prediction Branch predictor decision for taken versus not taken.

One-Bit Predictor Dynamic branch predictor that records only the last observed branch outcome.

Smith Predictor Saturating-counter branch predictor, commonly implemented with a two-bit counter per entry.

GShare Global-history predictor that uses an exclusive-or of low PC bits and global history to index the prediction table.

GSelect Global-history predictor that forms its index by concatenating selected PC bits and global-history bits.

Yeh/Patt Predictor Two-level branch predictor that uses recorded branch history to index a table of direction counters.

Perceptron Predictor Branch predictor that computes a weighted sum over branch-history bits and predicts from the sign of that sum.

Hybrid Predictor Predictor that chooses among multiple underlying predictors, often through a tournament-style selector.

BTFNT Backward taken, forward not taken static branch heuristic.

Delay Slot ISA semantic in which the instruction after a branch executes before control transfer takes effect.

Squash Slot Post-branch instruction position that may be conditionally killed based on branch outcome/policy.

WBWA Write-back, write-allocate pairing.

WTWNA Write-through, write-no-allocate pairing.

Blocking Cache Cache behavior where an outstanding miss prevents servicing younger accesses.

Non-Blocking Cache Cache behavior allowing selected younger accesses to proceed during a miss.

Hit-Under-Miss Ability to serve cache hits while a miss is still outstanding.

Acronym List

AAT Average Access Time
ALU Arithmetic Logic Unit
AVX Advanced Vector Extensions
ACK Acknowledgment
AW Address Width
BTFNT Backward Taken, Forward Not Taken
BTB Branch Target Buffer
CDB Common Data Bus
CISC Complex Instruction Set Computer
CPI Cycles Per Instruction
CPU Central Processing Unit
CT Cycle Time
DDG Data Dependence Graph
DSP Digital Signal Processor
DISP Dispatch and Decode
DRAM Dynamic Random-Access Memory
EDP Energy-Delay Product
EX Execute
FICI Fire In Order, Complete In Order
FIFO First In, First Out
FICO Fire In Order, Complete Out of Order
FOCO Fire Out of Order, Complete Out of Order
FP Floating Point
FU Functional Unit
GPU Graphics Processing Unit
HT Hit Time
HTM Hardware Transactional Memory
IC Instruction Count
ID Instruction Decode
IF Instruction Fetch
II Initiation Interval
ILP Instruction-Level Parallelism
IMS Iterative Modulo Scheduling
IPC Instructions Per Cycle
ISA Instruction Set Architecture

LFU Least Frequently Used
LL/SC Load-Link / Store-Conditional
LIP Least Recently Used Insertion Policy
LLC Last-Level Cache
LRU Least Recently Used
MEM Memory
MII Minimum Initiation Interval
MIMD Multiple Instruction, Multiple Data
MIP Most Recently Used Insertion Policy
MISD Multiple Instruction, Single Data
MESI Modified, Exclusive, Shared, Invalid
MP Miss Penalty
MR Miss Ratio
MRU Most Recently Used
MSI Modified, Shared, Invalid
MOESI Modified, Owned, Exclusive, Shared, Invalid
MOESIF Modified, Owned, Exclusive, Shared, Invalid, Forward
MPI Message Passing Interface
NACK Negative Acknowledgment
NISQ Noisy Intermediate-Scale Quantum
NoC Network on Chip
OpenMP Open Multi-Processing
OS Operating System
PC Program Counter
PCIe Peripheral Component Interconnect Express
PE Processing Element
PFN Physical Frame Number
PGAS Partitioned Global Address Space
PIPT Physically Indexed, Physically Tagged
RAS Return-Address Stack
RAT Register Alias Table
RAW Read After Write
RecII Recurrence-Constrained Initiation Interval
ResII Resource-Constrained Initiation Interval
RISC Reduced Instruction Set Computer
ROB Reorder Buffer
RMW Read-Modify-Write

RQL Reciprocal Quantum Logic
RS Reservation Station
RSFQ Rapid Single Flux Quantum
SIMD Single Instruction, Multiple Data
SIMT Single Instruction, Multiple Threads
SISD Single Instruction, Single Data
SMT Simultaneous Multithreading
SRAM Static Random-Access Memory
STM Software Transactional Memory
SSE Streaming SIMD Extensions
SSD Solid-State Drive
SU State Update
SUP Superconducting Unit of Performance
TLP Thread-Level Parallelism
TLB Translation Lookaside Buffer
TM Transactional Memory
VIPT Virtually Indexed, Physically Tagged
VLIW Very Long Instruction Word
VPN Virtual Page Number
WAR Write After Read
WAW Write After Write
WB Write Back
WBWA Write-Back, Write-Allocate
WTWNA Write-Through, Write-No-Allocate

Appendix D

Advanced Topics: Quantum and Superconducting Architectures

This appendix summarizes the advanced-topics guest lecture. The material is not a replacement for the core chapters; it is a short final-review reference for two research directions that use architectural ideas outside conventional CMOS processors.

D.1 Quantum Computing Architecture

Quantum computers are best viewed as accelerators rather than replacements for classical computers. A useful quantum system still needs a classical computer to prepare work, drive the quantum device, interpret measurements, and coordinate repeated trials. The expected advantages are problem-specific. Two motivating examples from the lecture were simulation of physical and molecular systems, which was the original Feynman-style motivation, and integer factoring, which matters because efficient factoring would threaten RSA-style public-key cryptography.

D.1.1 Why the Hardware Is Difficult

A classical transistor contains many atoms and is engineered so that small physical disturbances do not usually change the digital value. A qubit is different: in several implementations the useful state is tied to one atom, ion, neutral atom, or artificial atom-like device. That makes isolation and precise control central architectural concerns.

The lecture grouped quantum errors into two broad classes. *Decoherence* means that the qubit stops behaving like the intended closed quantum system because the environment interacts with it. A stray photon, thermal effect, vibration, or other coupling can destroy the state that the algorithm depends on. *Imperfect control* means that the hardware operation itself is inaccurate: a laser pulse may be mistimed, mis-aimed, too wide, too weak, too strong, or the measurement apparatus may fail to collect the intended signal.

These errors accumulate as a quantum program runs. For that reason, a common architectural and compiler heuristic is:

> fewer quantum instructions is usually better.

Shorter quantum programs reduce the time window for decoherence and reduce the number of imperfect operations that can corrupt the computation. This does not eliminate noise, but it is one of the few levers available to a compiler or architecture researcher when the underlying qubits are unreliable.

D.1.2 NISQ and Fault Tolerance

The near-term quantum era is often called the *NISQ* era: noisy intermediate-scale quantum computing. The lecture's main conclusion was that useful near-term quantum advantage is hard because current machines have too few reliable qubits, high error rates, and only short usable program windows. Many proposed near-term algorithms are simply too long or too qubit-hungry for the machines that exist today.

The longer-term path is *fault-tolerant quantum computing*. Instead of treating one noisy physical qubit as one reliable algorithmic qubit, many noisy physical qubits are entangled and managed together to form a more reliable *logical qubit*. The lecture used roughly a thousand physical qubits per logical qubit as an order-of-magnitude intuition, not as a universal constant. This creates a new architecture problem: the machine must repeatedly extract error information and run a decoder quickly enough to keep the logical qubits usable. Decoder design and topology-aware error correction are therefore architectural topics, not just physics topics.

D.1.3 Quantum Programs and Reversibility

Quantum gates describing the evolution of an isolated system must be reversible (unitary). Measurement and reset are different kinds of operation and need not be reversible on the measured system. The gate requirement is a major difference from ordinary classical functions. For example, a classical function $f(x)$ that maps many inputs to one output may not have an inverse, so it cannot be used directly as a quantum operation.

A Boolean function $f(x) \in \{0, 1\}$ can instead be represented by a reversible phase operation:

$$U_f|x\rangle = (-1)^{f(x)}|x\rangle. \tag{D.1}$$

If $f(x) = 0$, the state is unchanged. If $f(x) = 1$, the state receives a negative phase. Applying the same operation twice cancels the phase change, so this operation is reversible. This is the idea behind the lecture's "sign" version of a classical function.

The algorithmic benefit comes from superposition and interference. A quantum operation can act on a superposition of possible inputs, and later operations can make the desired answers reinforce while undesired answers cancel. This is not generic parallelism: the algorithm must match the mathematical structure of the problem. That is why quantum speedups exist for some problems but not for arbitrary computation.

D.2 Superconducting Processor Architecture

The second guest lecture asked whether processors can be built in a substantially different way from conventional CMOS designs. The motivation is energy. Modern data centers and supercomputers consume large amounts of power, and simply scaling familiar CMOS microarchitecture is increasingly expensive. Superconducting circuits offer extremely low switching energy, but they impose constraints that break many familiar processor structures.

D.2.1 Why Ordinary Processor Designs Do Not Map Cleanly

A superconductor has near-zero resistance, so the usual voltage-and-current intuition changes. Superconducting logic families often represent information using short pulses rather than long-lived

voltage levels. That can be very energy efficient, especially on wires, but it creates a practical problem: a tiny pulse cannot be freely fanned out to many destinations. Every split or copy of a signal may require explicit hardware.

This makes common high-performance structures expensive:

- multi-ported register files need many read and write paths,
- result buses and bypass networks need broad fanout,
- out-of-order schedulers need associative wakeup and tag broadcast,
- branch prediction and recovery need fast tables and cheap mis-speculation recovery,
- and multi-ported caches require duplicated indexing, tags, comparators, and wiring.

These are exactly the structures used heavily in conventional high-IPC CMOS cores. The architectural response is to move more work to the compiler and design the hardware around predictable data movement.

D.2.2 Syndrome: Compiler-Scheduled Single-Thread Performance

The lecture's proposed single-thread core, called Syndrome in the notes, uses a dataflow-like execution model. Instead of every instruction reading operands from and writing results back to one centralized register file, an instruction names later destination instructions that should receive its result. The produced value is held in an *operand queue* until the scheduled consumer needs it.

This resembles VLIW and software scheduling from Chapter 7, but with a different hardware motivation. The compiler knows the timing distance between producers and consumers, so it can encode relative delivery times and set up bypass or crossbar controls before the value arrives. The goal is to keep useful instruction-level parallelism while avoiding the most expensive fanout-heavy structures of out-of-order execution.

Control flow is also exposed earlier to the machine. The lecture described a three-part pattern:

- *prepare* supplies a possible branch target early,
- *compare* computes the condition early,
- *when* commits the control-flow choice when the condition is known.

This separates where the program might go from whether it will go there. For loops, where the exit condition is often available near the top of the loop body, this gives the processor time to prefetch and configure control paths without relying on a large conventional branch predictor.

The cache design follows the same theme. Instead of building a fully multi-ported cache, the design banks the cache by address. For example, odd and even indices can map to different half-sized banks. This gives multiple accesses per cycle when the accesses land in different banks, while avoiding the area cost of a monolithic multi-ported cache.

D.2.3 Stride: Thread-Level Parallelism Through Barrel Processing

The proposed Stride core attacks the problem from the other direction: thread-level parallelism instead of instruction-level parallelism. It is a barrel processor. On each cycle, the pipeline issues an instruction from a different thread, rotating through many hardware threads. If the pipeline is very deep, the core needs many resident threads so that a thread's long-latency operation can make progress before that thread is scheduled again.

The tradeoff is clear. One thread has very low individual IPC because it issues only once every many cycles. The whole core can still sustain high system throughput because many threads are in flight at once. This style is attractive for workloads with abundant independent or embarrassingly parallel work, but it is a poor fit for code that needs high single-thread latency.

D.2.4 SUP: Combining the Two Modes

The proposed larger unit combines the Syndrome-style core and the Stride-style core into a *superconducting unit of performance (SUP)*. The reason is that high-performance computing programs often alternate between phases:

- single-threaded or latency-sensitive setup,
- coarse-grained parallel work with some synchronization,
- and massively parallel phases such as matrix or signal-processing kernels.

Syndrome is intended for the single-thread and coarse-grained phases. Stride is intended for the highly threaded phases. A RISC-V-style register interface provides a software-visible boundary between the two modes: state can be saved into registers when leaving one side and restored when execution returns. A Cilk-like programming model is a natural fit because it already expresses spawn and join structure.

The design also reuses tiles where possible. Data tiles and memory tiles can serve both modes, while execution tiles can share storage structures with different controllers. That matters because superconducting fabrication area is expensive, and every avoided duplicate structure helps.

D.2.5 Cryogenic System Organization

Cooling cost cannot be ignored. The lecture used a rough rule of thumb that removing one watt from the 4 K environment may require hundreds of watts outside the cryostat. Even with that overhead, superconducting logic can still be attractive because the in-cryostat operation energy is extremely low. The relevant metric is not just peak frequency; it is energy-delay product and system throughput per watt.

A full system can use temperature staging. Superconducting logic sits near liquid-helium temperatures. Some CMOS support logic and cache structures may sit at a warmer cryogenic level, such as liquid-nitrogen temperature, where CMOS can still operate efficiently. Multiple cryostats can then be connected using conventional high-performance interconnects such as InfiniBand, Omni-Path, or optical networks.

D.3 Final Review Checkpoints

For final-review purposes, focus on the following claims.

1. Quantum computers are accelerators coordinated by classical computers, not universal replacements for classical processors.
2. Quantum speedups are problem-specific; physics simulation and integer factoring are canonical examples.
3. Decoherence and imperfect control are the two broad error sources emphasized in the lecture.

4. Fewer quantum instructions are preferred because errors accumulate with time and with each imperfect operation.
5. NISQ machines are limited by noise, qubit count, and short useful program lengths.
6. Fault-tolerant quantum computing uses many physical qubits plus decoding to create more reliable logical qubits.
7. Quantum operations must be reversible; phase-oracle forms such as Equation D.1 are one way to embed classical functions.
8. Superconducting logic can be much more energy efficient than CMOS, but cooling overhead and architecture constraints still matter.
9. Near-zero resistance does not make conventional fanout, register files, bypass networks, and branch predictors free.
10. The Syndrome-style core uses compiler-scheduled dataflow and operand queues to avoid centralized multi-ported register-file pressure.
11. The Stride-style core uses barrel processing to trade single-thread latency for high aggregate throughput across many threads.
12. A combined SUP-style design uses the single-thread core for latency-sensitive phases and the threaded core for highly parallel phases.

D.4 Practice True/False Prompts

Mark each statement true or false, then correct the false statements.

1. A quantum computer is expected to replace the classical computer entirely for all workloads.
2. Reducing quantum instruction count can improve reliability because both decoherence and gate/control errors accumulate during execution.
3. NISQ machines are already assumed to be good enough for long, reliable, general-purpose quantum programs.
4. Fault tolerance in quantum computing usually means encoding one logical qubit across many unreliable physical qubits.
5. A quantum implementation of a classical function must respect reversibility.
6. Superconducting processors can use all conventional out-of-order CPU structures cheaply because wire resistance is near zero.
7. Compiler scheduling becomes more important when hardware fanout, multi-ported storage, and associative wakeup are expensive.
8. A barrel processor improves the latency of one thread by issuing that same thread every cycle.
9. Banking a cache can provide multiple accesses per cycle when the accesses map to different banks.
10. A SUP-style design combines single-thread-oriented and throughput-oriented execution resources because real programs often move through different parallel phases.

Appendix E

Comprehensive Final Review Checklist

This appendix is a compact review map for the full course. It is not a substitute for the chapter practice questions; it is a last-pass checklist of the concepts that repeatedly connect worked problems, implementation exercises, and review questions.

E.1 Core Topic Checklist

Architecture and Performance

- Know the grain sizes of parallelism: bit-level, instruction-level, thread-level, and inter-program parallelism.
- Distinguish SIMD, MIMD, packed-word SIMD, and pipelined vector execution.
- Use the CPU-time equation time $= IC \times CPI \times CT$ and know which term an optimization changes.
- Apply Amdahl's law for fixed-size speedup and Gustafson's law for scaled workloads.
- Use arithmetic means for times and harmonic means for rates.

Caches and Memory Hierarchy

- Split addresses into tag, index, and offset fields from (C, B, S) cache notation.
- Classify misses as compulsory, capacity, conflict, or coherence misses when multiprocessors are involved.
- Connect each cache optimization to the term it improves: hit time, miss ratio, or miss penalty.
- Compare write-back/write-through and write-allocate/write-no-allocate behavior.
- Explain why VIPT caches must keep index plus offset bits within the page offset unless extra synonym-handling machinery exists.
- Treat tiling as a working-set transformation: choose tiles so the active data footprint fits in cache and does not create severe set conflicts.

Pipelines and Branch Prediction

- Interpret IF/ID/EX/MEM/WB timing diagrams cycle by cycle.
- Identify structural, data, and control hazards and connect them to inserted stalls or squashes.
- Explain why forwarding fixes many RAW hazards but does not eliminate the classic load-use stall.
- Compare delay slots, squash slots, branch target buffers, return-address stacks, and direction predictors.

- Work through one-bit, two-bit, GSelect, GShare, Yeh/Patt, perceptron, and hybrid predictor lookup/update behavior.

Dynamic and Compiler-Scheduled ILP

- Distinguish FICI, FICO, and FOCO execution models.
- Explain how Tomasulo-style scheduling uses reservation stations, tags, result broadcast, and register renaming.
- Explain why precise exceptions require in-order architectural state update, often through a reorder buffer or checkpoint/repair.
- Draw data dependence graphs and classify pure, anti-, and output dependences.
- Use dependence height as a list-scheduling priority.
- Explain VLIW as a trade: simpler dynamic hardware in exchange for stronger compiler scheduling responsibility.
- For software pipelining, compute $ResII$, $RecII$, and $MII = \max(ResII, RecII)$, then reason about prologue, kernel, and epilogue code.

Multiprocessors, Synchronization, and Consistency

- Distinguish shared-memory and message-passing systems.
- Explain coherence as ordering and propagation for one memory location, not as full synchronization correctness.
- Trace MSI/MESI/MOESI-style state transitions and know what `E`, `O`, and `F` add.
- Distinguish abort/retry from intervene behavior when dirty data is requested by another cache.
- Explain false sharing as unnecessary coherence traffic caused by unrelated variables sharing one cache block.
- Compare snooping and directory-based coherence, including why limited-pointer directories trade space against overflow behavior.
- Explain why test-and-test-and-set reduces coherence traffic compared with repeated test-and-set.
- Use locks for mutual exclusion around critical sections and barriers for regrouping a set of threads at a phase boundary.
- Define a data race and explain why coherence alone does not remove data races.
- Compare strict, sequential, and weak/relaxed consistency models.
- Know what store fences, load fences, full fences, acquire operations, and release operations prevent.

Interconnection Networks

- Distinguish direct topologies, such as meshes and tori, from indirect topologies, such as crossbars and multi-stage networks.
- For k-ary n-dimensional networks, compute node count as k^n.

- Define bisectional bandwidth as the aggregate bandwidth crossing the minimum equal-halves cut.
- Explain why a torus has more bisectional bandwidth than the corresponding mesh.
- Compare deterministic, adaptive, and oblivious routing.
- Explain Valiant's algorithm as randomized load balancing through an intermediate node.
- Distinguish packet-level and flit-level flow control.
- Compare store-and-forward, cut-through, wormhole, and virtual-channel flow control.

E.2 Review Clarifications

Tiling

Tiling is useful when the program has reuse but the untiled working set is too large for the cache. The transformation changes loop order so the program operates on cache-sized pieces before moving on. For review problems, the first check is usually the footprint inequality: active arrays, tile dimensions, and bytes per element must fit within the effective cache capacity. The second check is placement: a tile that fits by byte count can still perform poorly if the active blocks conflict in the same sets.

Iterative Modulo Scheduling

Modulo scheduling is easiest to understand as loop unrolling followed by rerolling. Unrolling exposes overlap among different iterations, but unrolling too far makes code large. Software pipelining keeps the repeating overlapped pattern as a compact kernel and adds a prologue and epilogue to handle startup and drain.

Iterative modulo scheduling begins at the lower bound $MII = \max(ResII, RecII)$. $ResII$ comes from resource pressure. $RecII$ comes from loop-carried dependence cycles. The compiler tries to schedule the loop at that initiation interval; if it cannot, it increments II and tries again. An operation that would appear in cycle x of the unrolled schedule maps to row $x \bmod II$ of the modulo reservation table. Operations that conceptually wrap around the compact kernel explain why prologue and epilogue code are needed.

Memory Consistency

The core consistency trap is that coherence is per-location, while synchronization usually depends on ordering between locations. In the producer/consumer pattern, a coherent machine can still let the consumer observe a flag update before it observes the data update that the flag was meant to publish. The fix is not merely better coherence; the fix is an ordering rule around synchronization, implemented through acquire/release semantics or explicit fences.

Strict consistency ties the visible memory order to actual time. Sequential consistency allows a single legal interleaving that preserves each thread's program order, even if that interleaving is not the real-time order. Weak or relaxed models allow ordinary accesses to move more freely but require synchronization accesses to carry stronger ordering guarantees.

Barriers and Locks

A lock protects a critical section: only one thread should execute that protected code at a time. A barrier is a phase boundary: every participating thread waits until all have arrived, and then all may proceed. Barriers are natural in bulk-synchronous programs such as GPU-style kernel phases. They are expensive when overused because fast threads idle while waiting for slow ones.

Bisectional Bandwidth

Bisectional bandwidth is a topology figure of merit. For a legal equal-halves cut, sum the bandwidths of the links crossing the cut. The bisectional bandwidth is the minimum such aggregate over the allowed equal splits. For the regular square meshes and tori used here, a dimension-aligned equal cut attains the minimum. Other equal-halves partitions are still legal candidates; the definition always takes the minimum over all of them. This value is a potential maximum, not a guarantee: routing, buffering, and traffic conflicts can keep real throughput below it.

E.3 Fast Self-Check Prompts

1. Given a cache geometry, can you compute tag/index/offset bits and explain whether a miss is compulsory, capacity, conflict, or coherence?
2. Given a branch predictor table and branch history, can you show both the prediction and the update?
3. Given a dependence graph, can you identify false dependences and explain how renaming removes them?
4. Given a loop body and functional-unit counts, can you compute $ResII$, $RecII$, and MII?
5. Given a coherence trace, can you count memory reads, memory writes, invalidations, and cache-to-cache transfers?
6. Given a lock implementation, can you explain which part provides atomicity and which part provides ordering?
7. Given two threads accessing two variables, can you decide which outcomes are legal under strict consistency, sequential consistency, and relaxed ordering?
8. Given a network topology, can you compute bisectional bandwidth and explain why routing may still perform worse than that theoretical limit?
9. Given a packet/flit scenario, can you explain whether store-and-forward, cut-through, wormhole, or virtual channels best describes the behavior?
10. Given the advanced-topics appendix, can you state why quantum machines and superconducting processors are architectural accelerators rather than simple CMOS replacements?

Appendix F

Lecture Sources and Historical Notes

The course materials supplied with this project are the primary reference for its scope and terminology. Slide numbers refer to the printed identifiers inside the handouts, which may differ from PDF page numbers because a PDF page can contain two slides.

How Attribution Works

Lecture examples are paraphrased, not reconstructed quotations. They preserve the teaching point while making modeling assumptions explicit. Examples added to connect ideas, rather than recovered from a lecture, are presented as the book's own illustrations.

Examples Used in the Narrative

Independent additions, Chapter 1. `n1_intro.pdf`, INTRO-6, contrasts three dependent additions with three independent additions on three adders. The book states the equal-latency, available-resource assumptions in its diagram.

The Cray vector example, Chapter 1. `n1_intro.pdf`, INTRO-13–14, contrasts streamed vector execution with packed-word arithmetic. The book separates the lecture's simplified pipeline from historical hardware timings.

Buying a jet-simulation machine, Chapter 2. `n2_perf.pdf`, PERF-18–20, uses an extravagant many-processor machine to expose Amdahl's limit, then changes the question to a larger simulation for Gustafson's law. The book uses 100 hours as a convenient baseline rather than the handout's rounded week-to-seconds conversion.

Buying the slower machine, Chapter 2. `n2_perf.pdf`, PERF-9–11, compares rates of 4 and 4 with rates of 2 and 7. The workload must contain equal work of each type for the harmonic-mean comparison.

The supercomputer joke, Chapter 2. January 15, 2026 lecture, Notability note *2. ECE 6100*, *Recording 2*, approximately 06:03–06:32. The cost discussion paraphrases the joke's economic point; it does not reproduce the uncertain personal-name attribution in automatic transcription.

Why renaming mattered, Chapter 6. `n5_ilp_part1.pdf`, ILP-14, connects the System/360 Model 91's long floating-point operations and small architectural register set to dynamic scheduling and renaming.

Primary Historical References

Vector registers and chaining. Cray Research, *CRAY-1 Computer Systems Hardware Reference Manual*, HR-0004F, May 1982, sections on vector registers and chaining. The manual describes

eight vector registers with up to 64 elements each and explains the conditions for forwarding an element stream between functional units. Available from the Computer History Museum's Bitsavers archive.

Dynamic scheduling. R. M. Tomasulo, *An Efficient Algorithm for Exploiting Multiple Arithmetic Units*, IBM Journal of Research and Development, 11(1), 1967, pp. 25–33. This is the original description of the Model 91 floating-point scheduling method. doi:10.1147/rd.111.0025.

Reading the Remaining Course Handouts

The chapter sequence follows `n1_intro.pdf`, `n2_perf.pdf`, `n3_cache_all.pdf`, `n4_pipe_part1.pdf`, `n5_ilp_part1.pdf`, the VLIW/software-pipelining handouts, `n6_multiprocv1.1.pdf`, `n7_synch.pdf`, and `n8_interconn.pdf`. These filenames identify the local course sources; the handouts are not redistributed with the book. Course-specific categories, including the broad use of speculation for locality-based caching, are identified in the text when a narrower definition is also common.

Specifications Behind Implementation Caveats

Atomic operations and reservations. The RISC-V atomic-extension specification provides a concrete example of selectable acquire/release ordering and permitted store-conditional failure. These details motivate the distinction between atomicity and ordering in Chapter 9; other ISAs have their own rules.

Dynamic and guided loop scheduling. The OpenMP worksharing-loop specification distinguishes fixed-size dynamic chunks from guided chunks that decrease as work is consumed. Chapter 9 uses this distinction when describing a fetch-and-add work counter.